TEMA 1: INTRODUCCIÓN A JAVA

Java es un lenguaje de programación de alto nivel, lo que significa que está diseñado para ser comprensible para los seres humanos. No está escrito en código de máquina (como el que entiende directamente una computadora), sino en un conjunto de reglas y estructuras que permiten expresar ideas lógicas y funcionales de forma más clara y abstracta. Como lenguaje de propósito general, Java se puede utilizar para desarrollar casi cualquier tipo de software: desde aplicaciones móviles y de escritorio hasta servidores web, videojuegos, sistemas bancarios, software empresarial y más.

1.1. ¿Por qué aprender Java?

Una de las principales razones por las que Java sigue siendo tan relevante décadas después de su creación es su promesa de portabilidad. Esta idea se resume en una frase muy conocida entre programadores: "Write once, run anywhere", que en español significa “Escríbelo una vez, ejecútalo en cualquier parte”. Esta frase hace referencia al hecho de que un programa escrito en Java puede ejecutarse en diferentes sistemas operativos (como Windows, macOS o Linux) sin necesidad de modificar el código fuente. Esto es posible gracias a un componente fundamental del ecosistema de Java llamado la **máquina virtual de Java**, o JVM.

1.2. La máquina virtual de Java (JVM)

La Máquina Virtual de Java (JVM por sus siglas en inglés, Java Virtual Machine) es un programa que actúa como intermediario entre el código Java que escribe el programador y el sistema operativo sobre el que se ejecuta el programa. Cuando escribimos un programa en Java, este no se convierte directamente en código que entiende la computadora (como ocurre con otros lenguajes como C o ensamblador), sino que se traduce a un formato intermedio llamado **bytecode**.

Este bytecode es un conjunto de instrucciones que no están ligadas a ningún sistema operativo específico. La JVM es la encargada de leer este bytecode y ejecutarlo en tiempo real, traduciéndolo en instrucciones comprensibles para la computadora en la que se encuentra. Gracias a esto, el mismo programa Java puede funcionar en distintas plataformas, siempre que en ellas esté instalada una JVM compatible.

En otras palabras, la JVM actúa como un traductor universal: recibe instrucciones genéricas (el bytecode) y las adapta al lenguaje que entiende la máquina donde se ejecuta. Esta arquitectura hace que Java sea altamente portátil y versátil, lo que lo convierte en una herramienta muy poderosa para los desarrolladores.

1.3. Un poco de historia

Java fue creado a principios de los años 90 por un equipo de ingenieros de la empresa Sun Microsystems, liderado por James Gosling. Su primera versión pública fue lanzada en 1995. Aunque en sus inicios estaba pensado para dispositivos electrónicos como televisores inteligentes, Java pronto se convirtió en un lenguaje clave para el desarrollo de aplicaciones de Internet, gracias a su seguridad, estabilidad y portabilidad. Con el paso del tiempo, Java se consolidó como uno de los lenguajes más utilizados en el mundo empresarial, y también en el desarrollo de aplicaciones móviles, especialmente con la llegada de Android, el sistema operativo más usado en smartphones, que se basa en gran medida en Java.

En 2010, Sun Microsystems fue adquirida por Oracle Corporation, que desde entonces es la empresa responsable de mantener y desarrollar el lenguaje Java.

1.4. Instalando el entorno de desarrollo: ¿Qué necesitas para programar en Java?

Para comenzar a programar en Java, necesitas tener instalado en tu computadora un conjunto de herramientas llamado JDK, que significa Java Development Kit o “Kit de Desarrollo de Java”. El JDK incluye:

- El compilador de Java (javac), que es el programa que traduce el código fuente escrito por el programador a bytecode.
- La JVM, que se encarga de ejecutar el bytecode en la computadora.
- Bibliotecas estándar, que son conjuntos de clases ya listas para usar y que permiten realizar tareas comunes como manejar archivos, conectarse a internet, trabajar con fechas, números, texto, interfaces gráficas y mucho más.
- Herramientas de apoyo para depuración y documentación.

Puedes descargar el JDK desde el sitio web oficial de Oracle (https://www.oracle.com/java/technologies/javase-downloads.html) o desde la web del proyecto OpenJDK (https://jdk.java.net/), que es una implementación libre y abierta de Java. En

la actualidad, existen varias versiones de Java disponibles, pero para comenzar es recomendable elegir una versión LTS (Long-Term Support), como Java 17 o Java 21, ya que estas versiones reciben soporte técnico durante varios años y son ampliamente utilizadas en el mundo profesional.

1.5. Configurar el entorno: ¿cómo escribir y ejecutar tu primer programa?

Una vez que hayas instalado el JDK, puedes programar en Java de dos maneras principales. La primera es utilizando únicamente un editor de texto simple (como el Bloc de notas en Windows, o un editor como Notepad++, Sublime Text o Visual Studio Code) y una terminal o consola de comandos. Esta opción te permite entender bien el funcionamiento interno de Java, ya que deberás compilar y ejecutar los programas de forma manual, lo que es excelente para el aprendizaje inicial.

La segunda opción es utilizar un IDE (Entorno de Desarrollo Integrado), que es una aplicación que reúne en un solo lugar todas las herramientas necesarias para escribir, compilar, ejecutar y depurar programas. Algunos IDEs populares para Java son IntelliJ IDEA, Eclipse y NetBeans. Estos entornos son ampliamente utilizados en el mundo profesional y ofrecen funcionalidades avanzadas como autocompletado de código, navegación rápida entre archivos, control de versiones y herramientas gráficas para diseñar interfaces.

En este texto voy a utilizar Eclipse.

1.6. Eclipse

Como decía, Eclipse es un entorno de desarrollo gratuito y de código abierto basado en Java y diseñado originalmente para escribir programas en este lenguaje. Con el paso de los años, se ha vuelto extensible y modular, por lo que hoy también se puede utilizar para programar en otros lenguajes como C++, Python, PHP o JavaScript, mediante plugins adicionales.

Entre sus principales características destacan una interfaz gráfica intuitiva con múltiples vistas y herramientas, así como una gestión automática de proyectos, carpetas y archivos fuente. También ofrece integración con sistemas de control de versiones como Git y la posibilidad de depurar programas paso a paso. Además, cuenta con una amplia comunidad y abundante documentación en línea.

1.6.1. Cómo instalar Eclipse

- Dirígete a la página oficial: https://www.eclipse.org/downloads/
- Descarga el instalador.
- Ejecuta el instalador. Selecciona *Eclipse IDE for Java Developers*. Se te pedirá seleccionar un directorio de instalación y una carpeta de trabajo (llamada workspace), que es donde Eclipse guardará tus proyectos por defecto.
- Una vez instalado, abre Eclipse. La primera vez que lo hagas te pedirá seleccionar o crear tu espacio de trabajo (workspace).

1.6.2. Crea tu primer proyecto Java en Eclipse

Ahora que tienes Eclipse instalado y funcionando, es momento de crear tu primer proyecto. A continuación, se describe paso a paso cómo escribir, compilar y ejecutar tu primer programa Java, que mostrará en pantalla el clásico mensaje: "¡Hola, mundo!".

Paso 1: Crea un nuevo proyecto Java

- Dependiendo de la versión, al abrir Eclipse puedes tener la opción de crear un nuevo proyecto Java, o bien, en el menú superior, ve a File/New/Java Project.
- Escribe el nombre del proyecto, por ejemplo, PrimerPrograma, y haz clic en Finish.
- Esto creará una carpeta de proyecto que contendrá todos los archivos y configuraciones necesarios.

Paso 2: Crea una clase Java

- En el panel izquierdo, haz clic derecho sobre la carpeta src dentro del proyecto recién creado.
- Selecciona New/Class (ver imagen 1).

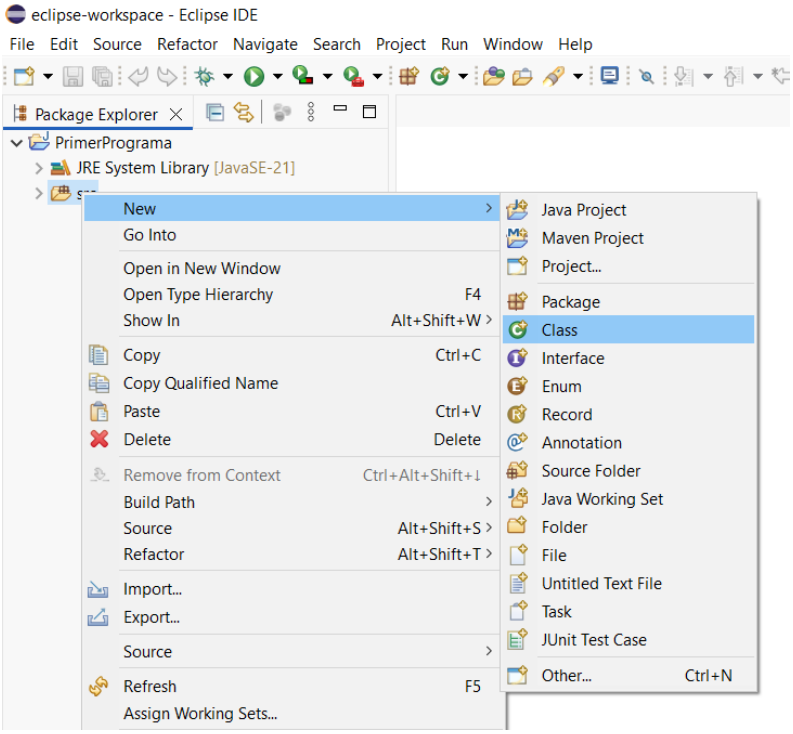

Ilustración 1. Creando una clase nueva en Eclipse. Elaboración propia.

- En la ventana que aparece, escribe el nombre de la clase: HolaMundo (ver ilustración 2).
- Añade el nombre del paquete (package) (ver ilustración 2).
- Marca la casilla que dice public static void main(String[] args) para que Eclipse genere automáticamente el método principal (ver ilustración 2).
- Haz clic en Finish.

1.7. ¿Qué es un paquete en Java?

Un paquete (o package, en inglés) es una forma de organizar las clases y los archivos fuente dentro de un proyecto. Funciona como una carpeta lógica: agrupa clases relacionadas entre sí para mantener el código limpio y ordenado.

Además, los paquetes evitan conflictos de nombres cuando diferentes clases se llaman igual. Por ejemplo, puedes tener una clase Usuario dentro del paquete com.banco.clientes y otra clase también llamada Usuario dentro del paquete com.forum.usuarios, sin que Java se confunda.

En Eclipse, siempre que creas una nueva clase, se recomienda escribir también un nombre de paquete. Si no lo haces, Eclipse colocará la clase en el paquete por defecto, pero esta práctica no es recomendable en proyectos reales porque dificulta la organización del código.

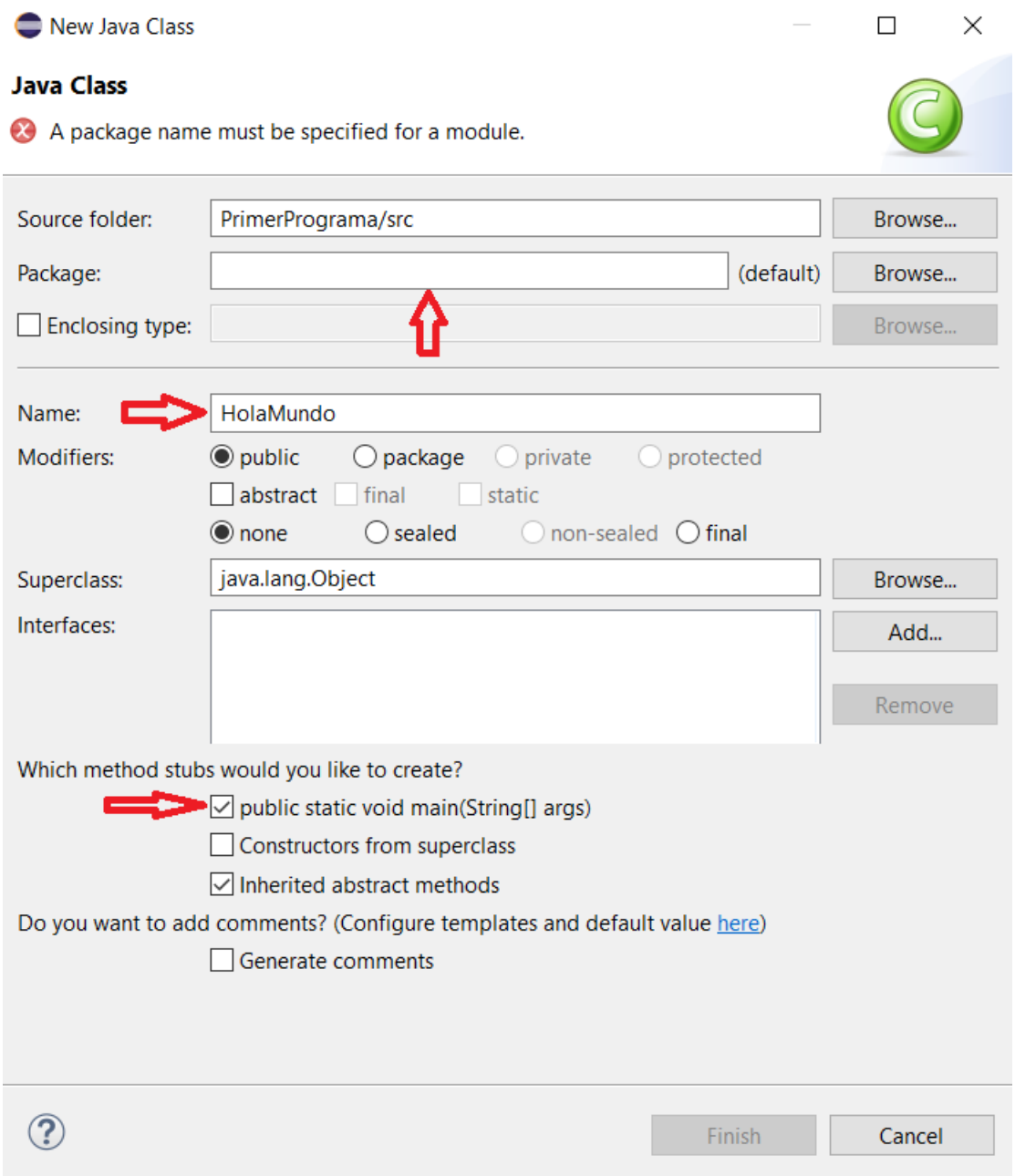

Ilustración 2. Configuración de una clase. Elaboración propia.

Eclipse creará una nueva clase llamada HolaMundo.java con el siguiente contenido básico:

```
public class HolaMundo {


   public static void main(String[] args) {
      // TODO Auto-generated method stub
   }
}
```

Paso 3: Escribir el mensaje de salida

Sustituye la línea que dice // TODO Auto-generated method stub por la siguiente línea de código:

```
System.out.println("¡Hola, mundo!");
```

El código completo se vería así:

```
public class HolaMundo {


   public static void main(String[] args) {
      System.out.println("¡Hola, mundo!");
   }
}
```

Paso 4: Ejecutar el programa

- Guarda el archivo (Ctrl + S o desde el menú File → Save).
- Haz clic derecho sobre el archivo HolaMundo.java en el panel izquierdo.
- Selecciona Run As → Java Application, o bien haz clic en el icono de la ilustración 3.
- En la parte inferior de la pantalla, en la consola de Eclipse, verás aparecer el mensaje: ¡Hola, mundo!

¡Felicidades! Acabas de crear y ejecutar tu primer programa Java desde Eclipse.

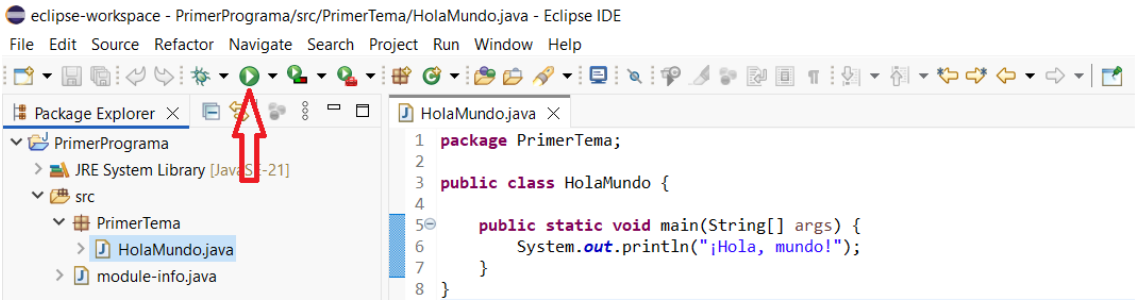

Ilustración 3. Icono para ejecutar el programa. Elaboración propia.

TEMA 2: FUNDAMENTOS DEL LENGUAJE JAVA

Una vez que ya sabes cómo crear y ejecutar un programa básico en Java, es momento de entender cómo se construyen los programas a nivel interno. Java, como cualquier lenguaje de programación, se basa en reglas y estructuras que permiten almacenar, manipular y procesar información. En este tema aprenderás los fundamentos esenciales: qué son las variables, qué tipos de datos existen, cómo se hacen operaciones básicas y cómo puedes pedirle al usuario que introduzca datos desde el teclado.

2.1. ¿Qué es una variable?

Una variable es un espacio en la memoria del ordenador donde puedes guardar información para usarla más adelante. Por ejemplo, si quieres guardar la edad de una persona, puedes crear una variable llamada edad y asignarle un número.

Piensa en una variable como una caja con nombre, donde puedes guardar un valor, cambiarlo, leerlo o usarlo para hacer cálculos.

En Java, para declarar una variable necesitas decir dos cosas: el tipo de dato que va a almacenar (por ejemplo, número, texto, etc.) y el nombre de la variable.

Por ejemplo:

```
int edad = 25;
```

Aquí, int indica que la variable almacena un número entero (sin decimales), edad es el nombre de la variable y 25 es el valor que guarda.

2.2. Tipos de datos en Java

Java es un lenguaje de tipado fuerte, lo que significa que cada variable debe tener un tipo bien definido y no puedes cambiarlo. Los tipos de datos básicos más comunes son:

- int: para números enteros, como 10, -3, 0.
- double: para números con decimales, como 3.14, -7.5, 2.0.
- String: para cadenas de caracteres (palabras, frases, nombres). Se escribe entre comillas dobles, como "Hola" o "Juan Pérez".

- boolean: para representar decisiones o condiciones. Solo puede tener dos valores: true (verdadero) o false (falso).

Existen otros tipos primitivos de datos menos utilizados. En la tabla 2.1 se muestran todos.

Tipo de dato	Tamaño	Rango de valores	Ejemplo de uso
byte	8 bits	–128 a 127	byte edad = 25;
short	16 bits	–32.768 a 32.767	short poblacion = 12000;
int	32 bits	–2.147.483.648 a 2.147.483.647	int anio = 2025;
long	64 bits	–9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	long distancia = 15000000000L;
float	32 bits	±3.4e−38 a ±3.4e+38 (con ~7 cifras decimales)	float precio = 19.99f;
double	64 bits	±1.7e−308 a ±1.7e+308 (con ~15 cifras decimales)	double pi = 3.1415926535;
char	16 bits	0 a 65.535 (caracteres Unicode)	char letra = 'A';
boolean	1 bit (teórico)	true o false	boolean activo = true;
String	(no primitivo)	Ilimitado (depende de la memoria disponible)	String nombre = "Lucía";

Tabla 2.1. Tipos de datos. Elaboración propia.

Los tipos **byte, short y long** también se usan para números enteros, como int. Se elige el tipo según el tamaño que necesitas para almacenarlo.

El tipo **float** es para números con decimales. Pero normalmente se usa double por su mayor precisión (más número de decimales).

El tipo de datos **char** guarda un solo carácter y debe ir entre comillas simples ('A', '5', '#').

2.3. Declaración y uso de variables

Veamos un ejemplo con varios tipos:

```
int edad = 30;
double altura = 1.75;
String nombre = "Lucía";
boolean esMayorDeEdad = true;

System.out.println("Nombre: " + nombre);
System.out.println("Edad: " + edad);
System.out.println("Altura: " + altura);
System.out.println("¿Es mayor de edad? " + esMayorDeEdad);
```

Este programa declara cuatro variables con distintos tipos y muestra su contenido por pantalla. Observa cómo se pueden combinar textos con variables usando el operador +, que en este contexto se llama **operador de concatenación**.

2.4. Operadores básicos

Java permite realizar operaciones matemáticas y lógicas básicas usando operadores. Aquí están los más comunes:

Operadores aritméticos (números)

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/)
- Módulo (resto de una división) (%)

Ejemplo:

```
int x = 10;
int y = 3;

System.out.println("Suma: " + (x + y));
```

```
System.out.println("Resta: " + (x - y));
System.out.println("Multiplicación: " + (x * y));
System.out.println("División: " + (x / y));
System.out.println("Resto: " + (x % y));
```

Nota: Cuando divides dos enteros (int), el resultado también será un entero. Si necesitas decimales, usa el tipo *double*.

Ha llegado el momento de que empieces a perder el miedo. Haz estos ejercicios sencillos. Intenta no mirar la solución.

Ejercicio 2.1. Desarrolla un programa que sume 543 y 653.

Ejercicio 2.2. Desarrolla un programa que divida 658 entre 251 con y sin decimales.

Ejercicio 2.3. Desarrolla un programa que calcule el resto de la división 41 entre 3.

2.5. Operadores de comparación

- Igual a (==)
- Distinto de (!=)
- Mayor que (>)
- Menor que (<)
- Mayor o igual (>=)
- Menor o igual (<=)

Ejemplo:

```
int edad = 20;
System.out.println(edad >= 18);
```

Este programa devuelve true porque el operador de comparación (booleana) entiende que 20 es mayor o igual que 18.

2.6. Comentarios en el código

Los comentarios son líneas que el programa ignora, pero que el programador usa para escribir explicaciones. Son muy útiles para aclarar qué hace cada parte del código. Existen dos formas:

Comentario de una línea:

```
// Esto es un comentario
```

Comentario en bloque (varias líneas)

```
/* Este es un comentario
   que ocupa varias líneas */
```

Puedes usar comentarios para dejar recordatorios, explicaciones o notas para otros programadores (o para ti mismo en el futuro).

En el programa anterior podríamos haber añadido comentarios. Así:

```
import java.util.Scanner; // Importamos la clase Scanner para leer datos del teclado

public class EntradaUsuario {
   public static void main(String[] args) {
      // Creamos un objeto Scanner llamado "teclado" que leerá desde el sistema de entrada
(teclado)
      Scanner teclado = new Scanner(System.in);

      // Mostramos un mensaje al usuario, sin salto de línea
      System.out.print("Introduce tu nombre: ");

      // Leemos la línea que el usuario escribe y la guardamos en la variable "nombre"
      String nombre = teclado.nextLine();

      // Mostramos un saludo personalizado
      System.out.println("Hola, " + nombre);

      // Cerramos el objeto Scanner para liberar recursos
      teclado.close();
   }
}
```

2.7. Entrada de datos por consola

Hasta ahora, todos los datos los hemos escrito directamente en el código. Pero en la mayoría de los programas queremos que el usuario introduzca información en tiempo real. Para eso, en Java usamos una clase llamada Scanner.

Una clase es una plantilla que define objetos: estructuras que pueden contener datos y comportamientos. En este caso, Scanner es una clase incluida en Java que permite leer datos desde el teclado (veremos estos conceptos con mayor profundidad más adelante).

Cómo usar Scanner:

1. Importa la clase al inicio del archivo:

```
import java.util.Scanner;
```

2. Declara un objeto Scanner:

```
Scanner teclado = new Scanner(System.in);
```

En esta orden:

- **teclado** es el nombre del objeto. Puedes llamarlo como quieras. Yo lo he llamado teclado porque me resulta más intuitivo.
- **new** se usa para crear objetos nuevos en Java.
- **System.in** significa “la entrada estándar del sistema”, que es, en la mayoría de los casos, el teclado.

Así que new Scanner(System.in) crea un lector que obtiene datos desde el teclado.

3. Usa los métodos disponibles para leer distintos tipos de datos:

```
import java.util.Scanner;

public class EntradaUsuario {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in); // Creamos el lector

      System.out.print("Introduce tu nombre: "); //Pedimos el nombre al usuario
      String nombre = teclado.nextLine(); //Lo leemos y lo almacenamos en una variable

      System.out.println("Hola, " + nombre); //Escribimos por la pantalla
```

```
        teclado.close(); // Cerramos el lector
    }
}
```

La línea **System.out.print("Introduce tu nombre: ");** muestra un mensaje en pantalla, pero sin dar un salto de línea:

- **System.out** representa la salida estándar del sistema, que normalmente es la consola.
- **print()** escribe el texto en la pantalla, pero no cambia de línea. El cursor se queda al lado del texto.
- Si en lugar de print() usáramos **println(),** sí se saltaría a la línea siguiente después del mensaje.

Con la línea **String nombre = teclado.nextLine();** estamos leyendo el texto que el usuario escribe en la pantalla y guardándolo en una variable. Así:

- **String** indica que la variable nombre es una cadena de texto.
- **teclado** es el objeto que creamos con Scanner.
- **nextLine()** es un método de la clase Scanner que lee una línea completa de texto, hasta que el usuario pulsa Enter.

NOTA: Se verá más adelante lo que significa método de una clase.

En este caso, si el usuario escribe “María López”, todo ese texto se guarda como una sola cadena.

Al igual que antes, la línea **System.out.println("Hola, " + nombre);** imprime un mensaje de saludo en pantalla, combinando texto fijo con una variable.

- En este caso **println()** imprime el texto y luego salta a la siguiente línea.
- El operador **+** se usa aquí para unir textos y variables, lo que se llama **concatenación**, es decir, si la variable nombre tiene asignada la cadena de texto "María", en la consola se mostrará: Hola, María.

La línea **teclado.close();** cierra el objeto Scanner cuando ya no se va a usar. Cuando se crea un objeto que accede a recursos del sistema (como archivos o el teclado), es una buena práctica liberar esos recursos cuando ya no se necesitan. Aunque el programa sea pequeño, esto ayuda a mantener el código limpio y evita posibles errores en programas más grandes.

2.8. Leer números por teclado

Ya sabemos cómo leer textos (cadenas) usando Scanner y el método nextLine(). Ahora vamos a ver cómo leer números enteros (int) o decimales (double). Esto se hace también con Scanner, pero usando los métodos:

- **nextInt()** para leer un número entero (int)
- **nextDouble()** para leer un número con decimales (double)

NOTA: si el usuario escribe algo que no es un número válido (como letras o algún otro carácter del teclado), el programa dará un error. Luego veremos cómo manejar eso, pero por ahora trabajaremos con entradas correctas.

Mira este ejemplo:

```
import java.util.Scanner;


public class DatosUsuario {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in); // Creamos el lector


      // Pedimos la edad
      System.out.print("Introduce tu edad: ");
      int edad = teclado.nextInt(); // Lee un número entero


      // Pedimos la altura
      System.out.print("Introduce tu altura en metros (por ejemplo, 1.75): ");
      double altura = teclado.nextDouble(); // Lee un número decimal


      // Mostramos la información
      System.out.println("Tienes " + edad + " años y mides " + altura + " metros.");


      teclado.close(); // Cerramos el objeto Scanner
   }
}
```

NOTA: es importante que el usuario escriba el número con decimales usando punto y no coma.

Por ejemplo, debe escribir 1.70 y no 1,70. En muchos países de habla hispana usamos coma para los decimales, pero Java (como casi todos los lenguajes) usa el punto decimal al estilo anglosajón.

Ejercicio 2.4. Escribe un programa que le pida al usuario su nombre (texto), su edad (entero) y su peso en kilos (decimal). Luego mostrará por consola un mensaje como: Hola, Carla. Tienes 28 años y pesas 60.5 kg. Añade comentarios a tu código.

OJO: Cuando usamos métodos como nextInt() o nextDouble(), Java lee solamente el número introducido, pero **deja el salto de línea (Enter) sin procesar** en la memoria del teclado. Esto puede causar errores si luego llamas a nextLine(). Mira este ejemplo:

```
System.out.print("Introduce tu edad: ");
int edad = teclado.nextInt(); // el usuario escribe: 25 y pulsa Enter

System.out.print("Introduce tu nombre: ");
String nombre = teclado.nextLine(); // ¡problema! Java lee el "Enter" pendiente, no espera texto
```

En este caso la variable nombre queda vacía ("") porque el programa leyó directamente el Enter que quedó pendiente después del número.

Para evitar este problema, **hay que consumir el salto de línea pendiente** antes de pedir el texto (limpiar el buffer). Lo hacemos añadiendo un teclado.nextLine(); **después de leer el número**. Así:

```
package SegundoTema;

import java.util.Scanner;

public class DatosCorregidos {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce tu edad: ");
      int edad = teclado.nextInt(); // Lee el número

      teclado.nextLine(); // Limpia el "Enter" pendiente

      System.out.print("Introduce tu nombre: ");
```

```
        String nombre = teclado.nextLine(); // Ahora sí, espera correctamente una línea de texto

        System.out.println("Tienes " + edad + " años y te llamas " + nombre);

        teclado.close();
    }
}
```

Este nextLine() extra de limpieza **no guarda nada en ninguna variable**, simplemente se usa para "saltar" el Enter que quedó colgado.

Ejercicio 2.5. Desarrolla un programa que pida al usuario una cantidad de metros y los convierta a millas (1 milla = 1.609m). Usa los dos tipos de comentarios vistos a lo largo del código.

El resto de métodos de la clase Scanner que se utilizan para leer datos de la consola se muestran en la tabla 2.2.

Tipo de dato	**Método de lectura (Scanner)**	**Ejemplo de uso**
int	nextInt()	int edad = teclado.nextInt();
double	nextDouble()	double precio = teclado.nextDouble();
float	nextFloat()	float nota = teclado.nextFloat();
long	nextLong()	long poblacion = teclado.nextLong();
short	No hay método específico. Hay que leer como int y convertir (casting)	short s = (short) teclado.nextInt();
byte	No hay método específico. Hay que leer como int y convertir (casting)	byte b = (byte) teclado.nextInt();
char	No hay método directo. Hay que leer como String y extraer el primer carácter	char letra = teclado.next().charAt(0);
boolean	nextBoolean()	boolean activo = teclado.nextBoolean(); *(debes escribir true o false)*
String	nextLine() (texto completo con espacios) o next() (una palabra)	String nombre = teclado.nextLine();

Tabla 2.2. Métodos de la clase Scanner. Elaboración propia.

Como se ha dicho antes, el método nextLine() lee **toda la línea completa** (incluidos espacios). Sin embargo, el método next() lee **solo una palabra**, hasta el primer espacio o Enter.

Para leer un carácter char, se suele hacer: char letra = teclado.next().charAt(0); es decir, se lee una palabra y toma su primer carácter.

Para los datos short y byte, se usa **casting** ((short), (byte)) para convertir el tipo de dato.

Existen métodos nextByte() y nextShort() en la clase Scanner, pero **no se usan tanto** porque:

- Son **más propensos a errores** si el usuario introduce valores fuera del rango.
- La mayoría de los programadores simplemente leen como int y convierten con casting manual.

Ejemplo de lectura del teclado para todos los tipos de datos:

```
package SegundoTema;
import java.util.Scanner;

public class LecturaDeTipos {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Leer una línea de texto
      System.out.print("Introduce tu nombre (String): ");
      String nombre = teclado.nextLine();

      // Leer un número entero (int)
      System.out.print("Introduce tu edad (int): ");
      int edad = teclado.nextInt();

      // Leer un número con decimales (double)
      System.out.print("Introduce tu peso (double): ");
      double peso = teclado.nextDouble();

      // Leer un número con decimales menos preciso (float)
      System.out.print("Introduce tu altura (float): ");
      float altura = teclado.nextFloat();

      // Leer un número entero grande (long)
```

```
        System.out.print("Introduce tu número de teléfono (long): ");
        long telefono = teclado.nextLong();

        // Leer un número pequeño (short) usando casting
        System.out.print("Introduce tu número favorito pequeño (short): ");
        short favoritoShort = (short) teclado.nextInt(); // cast desde int

        // Leer un número muy pequeño (byte) usando casting
        System.out.print("Introduce tu nivel de energía (byte): ");
        byte energia = (byte) teclado.nextInt(); // cast desde int

        // Leer un carácter (char): no hay método directo, se usa nextLine + charAt(0)
        teclado.nextLine(); // limpiar salto de línea pendiente
        System.out.print("Introduce tu inicial (char): ");
        char inicial = teclado.nextLine().charAt(0); // cast implícito

        // Mostrar los valores ingresados
        System.out.println("\n--- Resumen de datos ---");
        System.out.println("Nombre: " + nombre);
        System.out.println("Edad: " + edad);
        System.out.println("Peso: " + peso);
        System.out.println("Altura: " + altura);
        System.out.println("Teléfono: " + telefono);
        System.out.println("Número favorito (short): " + favoritoShort);
        System.out.println("Energía (byte): " + energia);
        System.out.println("Inicial: " + inicial);

        teclado.close();
    }
}
```

2.9. Casting

Casting significa decirle explícitamente al compilador que quieres convertir un valor de un tipo a otro. En Java, esto se hace escribiendo el tipo destino entre paréntesis delante del valor o variable.

```
tipoDestino nuevaVariable = (tipoDestino) valorOriginal;
```

Existen dos tipos de casting:

- **Casting implícito (ampliación).** Java lo hace automáticamente cuando no hay riesgo de pérdida de información. Por ejemplo, convertir de int a double es seguro:

```
int x = 10;
double y = x;  // conversión automática de 10 a 10.0
```

- **Casting explícito (reducción).** Cuando hay riesgo de pérdida de precisión o datos, tú debes indicarle a Java que quieres hacerlo. Esto es casting explícito:

```
double a = 9.7;
int b = (int) a;  // fuerza la conversión: b = 9
```

¡Ojo! Tienes que tener cuidado con algunas conversiones. Por ejemplo, mira lo que pasaría al convertir un número grande en otro más pequeño:

```
int numeroGrande = 130;
byte pequeño = (byte) numeroGrande;  // pequeño = -126 (¡por desbordamiento!)
```

Es decir, el número 130 no cabe en una variable de tipo byte, por lo que se produce un desbordamiento.

Otro ejemplo de conversión entre números enteros compatibles:

```
short s = 32000;
int i = s;            // implícito (seguro)
byte b = (byte) s;     // explícito (puede perder datos)
```

¿Cuándo usar casting?

- Al leer números pequeños (byte, short) con Scanner (que devuelve int).
- Al hacer operaciones entre tipos distintos, como sumar un double con un int.
- Cuando necesitas convertir tipos manualmente en condiciones específicas (como truncar decimales).

NOTA: El casting **no redondea** ni verifica si la conversión es segura. Solo **recorta el valor** según las reglas del tipo de destino.

```
double pi = 3.99;
int x = (int) pi;  // x = 3, no 4
```

2.10. ¿Cuándo añadir una letra al final de algunos valores?

En Java, cuando escribes un **número con decimales**, por defecto **el compilador lo interpreta como un double**, que es el tipo decimal más preciso y usado. Entonces, si tú declaras una variable float y le das un valor decimal como 3.14, Java te lanza un error. ¿Por qué? Porque 3.14 **es double**, y estás intentando guardarlo en un float, que es más pequeño (menos preciso). Para solucionar este problema tienes que decirle explícitamente al compilador que ese número es un float, **añadiendo una f o F al final del número**. Así:

```
float pi = 3.14f;  // Correcto
```

Lo mismo ocurre con los tipos de datos long, necesitamos añadir una L al final del valor. Cuando escribes un número entero en tu código, Java lo interpreta automáticamente como un *int*, a menos que tú le digas lo contrario.

```
long poblacion = 9000000000L;
```

NOTA: Usa L mayúscula en el tipo de dato long, porque la l minúscula se puede confundir fácilmente con el número uno.

Ejercicio 2.6. Desarrolla un programa que divida 4 entre 3 usando una variable de tipo entero, luego usando variables de tipo real de precisión simple y de precisión doble.

Ejercicio 2.7. Desarrolla un programa que calcule el área de un círculo (área = pi * radio al cuadrado). El radio se le pedirá al usuario.

Ejercicio 2.8. Desarrolla un programa que calcule el volumen de una esfera (volumen = pi * radio al cubo * 4/3). El radio lo introducirá el usuario y se leerá con una variable de tipo double.

Ejercicio 2.9. Desarrolla un programa que pida al usuario una distancia y una velocidad constante (double). El programa calculará el tiempo necesario para recorrer ese espacio en horas, minutos y segundos (int).

2.11. Formateo de números

Las funciones **System.out.printf()** y **String.format()** permiten usar plantillas de formato para controlar la cantidad de decimales.

```
double numero = 3.141592;
System.out.printf("Número con 2 decimales: %.2f\n", numero); //  %.2f redondea a 2 decimales y \n es una secuencia de escape que hace un Enter para ir a la siguiente línea
String texto = String.format("Número con 3 decimales: %.3f", numero);
System.out.println(texto);
```

Cuya salida es:

```
Número con 2 decimales: 3.14
Número con 3 decimales: 3.142
```

Ejercicio 2.10. Desarrolla un programa que pida al usuario que introduzca dos números de doble precisión. El programa dividirá el primero entre el segundo y mostrará el resultado por consola utilizando tres cifras decimales.

NOTA: Si introduces 0 como segundo número, el programa intentará dividir entre cero y mostrará un resultado como "Infinity" o un error. Esto es normal por ahora; aprenderemos a controlar este tipo de situaciones más adelante.

2.12. Secuencias de escape

Las secuencias de escape en Java son combinaciones especiales de caracteres que comienzan con una barra invertida (\) y permiten representar caracteres que no podrías escribir directamente en una cadena de texto, como saltos de línea, tabulaciones, comillas, etc.

Se usan dentro de cadenas de texto (String) para dar formato, mostrar caracteres especiales o controlar la salida por pantalla.

Se llaman de escape porque "escapan" del significado literal de un carácter. Por ejemplo, el carácter " normalmente marca el inicio o fin de una cadena. Si quisieras mostrar unas comillas dentro de la cadena, no puedes escribirlas directamente. Para eso necesitas una secuencia especial: \".

Las principales secuencias de escape se muestran en la tabla 2.3:

Secuencia	Significado	Ejemplo de uso
\n	Salto de línea	"Hola\nMundo"

Secuencia	Significado	Ejemplo de uso
\t	Tabulación	"Nombre:\tCarlos"
\"	Comillas dobles	"Dijo: \"Hola\""
\'	Comillas simples	'\'
\\	Barra invertida	"C:\\archivos\\miFoto.jpg"
\r	Retorno de carro	Rara vez se usa solo
\b	Retroceso	Poco común
\f	Salto de página	Obsoleto en la práctica

Tabla 2.3. Principales secuencias de escape. Elaboración propia.

Ejemplo práctico:

```
public class SecuenciasEscape {
    public static void main(String[] args) {
        // Salto de línea
        System.out.println("Línea 1\nLínea 2");

        // Tabulación
        System.out.println("Nombre:\tAna");

        // Comillas dobles dentro de una cadena
        System.out.println("Él dijo: \"Estoy aprendiendo Java\"");

        // Barra invertida
        System.out.println("Ruta del archivo: C:\\Usuarios\\Documentos");

        // Combinación de varias secuencias
        System.out.println("Primera línea\n\t\"Texto tabulado entre comillas\"\nFin");
    }
}
```

Salida por consola:

```
Línea 1
Línea 2
```

```
Nombre:         Ana
Él dijo: "Estoy aprendiendo Java"
Ruta del archivo: C:\Usuarios\Documentos
Primera línea
        "Texto tabulado entre comillas"
Fin
```

2.13. Atajos para operaciones matemáticas comunes

En Java (y muchos otros lenguajes), hay atajos muy útiles para operaciones matemáticas comunes que hacen el código más corto y limpio. Aquí te muestro los más usados:

- Incrementar en 1

```
i = i + 1;   // forma larga
i += 1;      // abreviada con operador compuesto
i++;         // forma más corta y común
```

- Decrementar en 1

```
i = i - 1;
i -= 1;
i--;         // decrementa en 1
```

- Sumar otro valor

```
i = i + 5;
i += 5;     // suma 5 a i
```

- Restar otro valor

```
i = i - 3;
i -= 3;     // resta 3 a i
```

- Multiplicar

```
i = i * 2;
i *= 2;     // multiplica i por 2
```

- Dividir

```
i = i / 4;
```

```
i /= 4;     // divide i entre 4
```

- Módulo (resto de la división)

```
i = i % 3;
i %= 3;     // asigna a i el resto de dividir i entre 3
```

Ejemplo:

```
int i = 5;


i++;       // ahora i es 6
i += 3;     // ahora i es 9
i *= 2;     // ahora i es 18
i--;       // ahora i es 17
i /= 4;     // ahora i es 4 (división entera)
i %= 3;     // ahora i es 1 (resto de 4 dividido entre 3)
```

2.14. Ejercicios de repaso

Ejercicio 2.11. Pide al usuario que introduzca dos números enteros y muestra la suma de ambos. Usa nextInt() y muestra el resultado con System.out.println().

Ejercicio 2.12. Pide al usuario que introduzca su nombre y apellidos (en líneas separadas) usando nextLine(). Luego muestra el nombre completo concatenado.

Ejercicio 2.13. Pide al usuario una distancia en metros (tipo int) y conviértela a kilómetros (tipo double). Muestra el resultado con dos decimales.

Ejercicio 2.14. Pide al usuario un número entero y guárdalo en una variable int. Luego conviértelo a byte y short usando casting, y muestra ambos valores.

Ejercicio 2.15. Pide dos números double y muestra el resultado de la división con tres cifras decimales usando printf.

Ejercicio 2.16. Declara una variable entera e inicialízala con un valor. Usa i++, i--, i += 5, i *= 2 y muestra el valor después de cada operación.

Ejercicio 2.17. Desarrolla un programa que pida varias letras al usuario y luego las muestre por pantalla formando una única palabra.

Recuerda:

- next().charAt(0) permite leer solo la **primera letra** introducida.
- Las letras se **concatenan** usando +, y se inicia la cadena con "" para que Java entienda que debe unir todo como texto.

Si no ponemos las comillas dobles al principio, Java intentaría hacer una suma entre caracteres, porque l1, l2, ... son variables de tipo char, que internamente son números (códigos Unicode).

Por ejemplo, si escribieras:

```
char l1 = 'a'; // código Unicode 97
char l2 = 'b'; // código Unicode 98
String palabra = l1 + l2 + ...;
```

Java sumaría esos números (97 + 98 + ...) en vez de unirlos como texto.

Sin embargo, si empiezas con "" (cadena vacía), Java entiende que vas a concatenar texto, así que convierte todos los caracteres siguientes en texto y los une uno detrás de otro.

Ejemplo:

```
String texto = "" + l1 + l2 + l3;
```

Ejercicio 2.18. Desarrolla un programa que pida al usuario 5 palabras de una en una. Después se mostrarán en consola como una frase.

Ejercicio 2.19. Pide al usuario una temperatura en grados Celsius y conviértela a Fahrenheit usando la fórmula: F = (C * 9/5) + 32. Muestra el resultado con dos decimales.

Ejercicio 2.20. Pide al usuario tres números double y muestra el promedio con dos decimales.

Ejercicio 2.21. Pide un número de días y muestra cuántas horas, minutos y segundos hay en esa cantidad.

Ejercicio 2.22. Pide nombre y apellido y forma una dirección de email en formato:

nombre.apellido@ejemplo.com.

Ejercicio 2.23. Pide base y altura, luego calcula y muestra perímetro y área.

TEMA 3. ESTRUCTURAS DE CONTROL DE FLUJO

Hasta ahora, nuestros programas ejecutaban las instrucciones de forma lineal, una detrás de otra. Pero en el mundo real, la mayoría de las veces se necesita tomar decisiones (por ejemplo, si el usuario introduce un número negativo, mostrar un mensaje de error), o repetir tareas (como pedir una contraseña hasta que sea válida).

Java, como la mayoría de los lenguajes de programación, nos ofrece estructuras especiales para:

- Tomar decisiones (estructuras condicionales).
- Repetir bloques de código (estructuras repetitivas o bucles).
- Controlar el flujo dentro de estas estructuras (palabras clave como break y continue).

3.1. Estructura condicional if-else

La forma más básica de tomar decisiones en Java es con la sentencia **if**. Su sintaxis es:

```
if (condición) {
   // Código que se ejecuta si la condición es verdadera
}
```

Y si queremos hacer algo cuando la condición no se cumple, añadimos **else**. Así:

```
if (condición) {
   // Código si se cumple
} else {
   // Código si NO se cumple
}
```

Mira este ejemplo:

```
int edad = 20;

if (edad >= 18) {
   System.out.println("Eres mayor de edad.");
} else {
   System.out.println("Eres menor de edad.");
}
```

(edad >= 18) es una expresión lógica que se evalúa como true o false. Si se cumple, se ejecuta el primer bloque; si no, el segundo.

Además, puedes comprobar varias condiciones usando **else if**. Así:

```
if (nota >= 9) {
   System.out.println("Sobresaliente");
} else if (nota >= 7) {
   System.out.println("Notable");
} else if (nota >= 5) {
   System.out.println("Aprobado");
```

```
} else {
   System.out.println("Suspenso");
}
```

Ejercicio 3.1. Desarrolla un programa que pida al usuario dos números reales de simple precisión (float) y diga cuál es el mayor de ellos.

Ejercicio 3.2. Desarrolla un programa que pida al usuario un número entero corto (short). Si es impar pedirá un segundo número entero, pero en esta ocasión largo (long). Ahora mostrará por pantalla si el número largo es impar.

NOTA: Para saber si un número es impar tienes que dividirlo entre dos y comprobar el resto, es decir, si el módulo (resto de la división) es distinto de 0 significa que es impar (num%2 != 0).

Además, podemos combinar condiciones con **operadores lógicos** (ver tabla 3.1):

Operador	Nombre	Significado	Ejemplo
&&	AND	Verdadero solo si **ambas** condiciones son verdaderas	x > 0 && y > 0
||	OR	Verdadero si **al menos una** de las condiciones es verdadera	x < 0 || y > 0
!	NOT	Invierte el valor lógico (niega la condición)	!(x > 0)

Tabla 3.1. Operadores lógicos. Elaboración propia.

Mira este ejemplo:

```
int edad = 25;
boolean tieneCarnet = true;

if (edad >= 18 && tieneCarnet) { //operador lógico AND
   System.out.println("Puede conducir.");
} else {
   System.out.println("No puede conducir.");
}
```

Ejercicio 3.3. Desarrolla un programa que pida un número entero corto (short) al usuario y muestre por pantalla si es múltiplo de 2 o de 5. Después, pedirá un nuevo número y lo restará al primero.

NOTA: Para saber si el número es múltiplo, tienes que dividir el número entre 2 y entre 5. Si el resto de la división (módulo) tiene que ser igual a 0 (num % 2 == 0) || (num % 5 == 0).

Ejercicio 3.4. Desarrolla un programa que pida al usuario un número entero de un byte y que diga si es múltiplo de 2 y de 5.

3.1.1. Operador ternario

El operador ternario o condicional tiene esta forma:

```
condición ? valor_si_verdadero : valor_si_falso;
```

El operador:

- Evalúa una condición.
- Si la condición es verdadera, devuelve el primer valor.
- Si es falsa, devuelve el segundo valor.

Como ves es equivalente a la estructura if-else solo que el operador ternario es más conciso, ahorras líneas de código. Por otro lado, este operador es mucho menos intuitivo.

Ejemplo:

```
package TercerTema;

import java.util.Scanner;

public class OperadorTernarioEjemplo {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce tu edad: ");
      int edad = teclado.nextInt();

      String resultado = (edad >= 18) ? "Eres mayor de edad" : "Eres menor de edad";

      System.out.println(resultado);

      teclado.close();
   }
}
```

Se puede imitar una estructura de if…else if… else tantas veces como necesites metiendo otro operador ternario dentro del segundo valor (valor_si_falso) para evaluar una segunda condición. Así:

```
package TercerTema;

import java.util.Scanner;

public class NotaConTernario {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce una nota del 0 al 10: ");
      int nota = teclado.nextInt();

      String resultado = (nota >= 9) ? "Sobresaliente" :
```

```
                    (nota >= 7) ? "Notable" :
                    (nota >= 5) ? "Aprobado" :
                              "Suspenso";

        System.out.println("Resultado: " + resultado);

        teclado.close();
    }
}
```

Ejercicio 3.5. Usa el operador condicional para calcular el menor de dos números reales de doble precisión introducidos por el usuario.

Ejercicio 3.6. Desarrolla un programa que pida dos números enteros cortos (short) al usuario. Después usará el operador ternario (?) para asignar a una variable booleana el valor “true” si los dos números son iguales, o “false” si no los son.

3.2. Estructura switch

Switch es una estructura de control que evalúa el valor de una expresión (normalmente una variable) y ejecuta un bloque de código según el caso que coincida. Su sintaxis es:

```
switch (expresión) {
   case valor1:
      // código si la expresión vale valor1
      break;
   case valor2:
      // código si la expresión vale valor2
      break;
   default:
      // código si no coincide ningún caso
}
```

Donde:

- **expresión**: puede ser int, byte, short, char, String.
- **case**: compara la expresión con un valor concreto.
- **break**: evita que el código "caiga" al siguiente caso.
- **default**: bloque opcional que se ejecuta si no hay coincidencia.

Ejemplo:

```
package TercerTema;

import java.util.Scanner;

public class EjemploSwitch {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.println("Menú:");
      System.out.println("1. Saludar");
```

```
        System.out.println("2. Despedirse");
        System.out.println("3. Salir");
        System.out.print("Elige una opción (1-3): ");

        int opcion = teclado.nextInt();

        switch (opcion) {
            case 1:
                System.out.println("Hola");
                break;
            case 2:
                System.out.println("Adiós");
                break;
            case 3:
                System.out.println("Saliendo del programa...");
                break;
            default:
                System.out.println("Opción no válida.");
        }

        teclado.close();
    }
}
```

Usa switch para mejorar la legibilidad del código en lugar de usar muchos bloques de if-else if. Normalmente se utilizan para evaluar las opciones de un menú.

Para optimizar código puedes agrupar varios “case” sin escribir código cuando la sentencia es la misma. Mira este ejemplo para aclarar el concepto:

```
package TercerTema;

import java.util.Scanner;

public class NumerosEntre1y5 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce un número del 1 al 10: ");
        int numero = teclado.nextInt();

        switch (numero) {
            case 1:
            case 2:
            case 3:
            case 4:
            case 5:
                System.out.println("El número está entre 1 y 5.");
                break;
            case 6:
            case 7:
```

```
        case 8:
        case 9:
        case 10:
            System.out.println("El número está entre 6 y 10.");
            break;
        default:
            System.out.println("Número fuera de rango.");
    }

    teclado.close();
  }
}
```

Otro ejemplo más: Supón que pedimos una letra al usuario y que las letras A, B y C se consideran "alta prioridad" y D, E "prioridad baja".

```
package TercerTema;

import java.util.Scanner;

public class PrioridadPorLetra {
  public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);

    System.out.print("Introduce una letra (A-E): ");
    String letra = teclado.nextLine().toUpperCase();

    switch (letra) {
        case "A":
        case "B":
        case "C":
            System.out.println("Alta prioridad.");
            break;
        case "D":
        case "E":
            System.out.println("Prioridad baja.");
            break;
        default:
            System.out.println("Letra no reconocida.");
    }

    teclado.close();
  }
}
```

Ejercicio 3.7. Desarrolla un programa que pida al usuario que introduzca un carácter. Después, dirá si se trata de un signo de puntuación (. , ; :), un número u otro carácter.

Ejercicio 3.8. Desarrolla un programa que pida al usuario un número del 1 al 7. El programa mostrará el nombre del día correspondiente (1 = lunes, 2 = martes, ..., 7 = domingo).

3.3. Estructuras repetitivas o bucles

Los bucles nos permiten repetir una o más instrucciones **mientras se cumpla una condición** o durante un número de veces determinado.

3.3.1. Estructura while

La sintaxis de esta estructura es:

```
while (condición) {
   // código que se ejecuta mientras la condición sea verdadera
}
```

El bucle **comprueba primero la condición**, y si es true, ejecuta el cuerpo del bucle hasta que la condición pase a ser false. Sin embargo, si la condición es false desde el principio, el cuerpo **no se ejecuta nunca**.

Mira este ejemplo:

```
int i = 1;
while (i <= 5) {
   System.out.println("i vale: " + i);
   i++;
}
```

En este caso, como 1 es menor que 5, se entra en el bucle. En cada iteración se va incrementando la variable i hasta que llega a 6. En este momento, al no cumplirse la condición del bucle, se sale.

¡OJO! Cuando comparas dos variables de tipo String, no debes usar ==, sino el método equals() porque == comprueba si dos variables apuntan al mismo objeto en memoria (es decir, si son el mismo lugar en la memoria), mientras que equals() compara el contenido real de los Strings, es decir, si las cadenas tienen exactamente los mismos caracteres en el mismo orden.

```
String a = new String("hola");
String b = new String("hola");

System.out.println(a == b);      // false — porque son objetos diferentes en memoria
System.out.println(a.equals(b)); // true  — porque su contenido es igual
```

Ejercicio 3.9. Desarrolla un programa que pida al usuario una contraseña. La contraseña correcta es 1234. Mientras no acierte la contraseña permanecerá el programa dentro del bucle pidiendo que vuelva a introducir la contraseña. Al final del programa se le dará *feedback* al usuario diciendo: “Contraseña acertada.”

Ejercicio 3.10. Desarrolla un programa que calcule el cubo de un número entero corto introducido por el usuario. Esto lo hará de manera repetitiva hasta que el usuario introduzca un 0. Al final del programa se le dará *feedback* al usuario diciendo: “Fin del programa.”

3.3.2. Estructura do-while

La sintaxis de esta estructura es:

```
do {
   // código que se ejecuta al menos una vez
} while (condición);
```

A diferencia de la estructura while, este bucle **siempre se ejecuta al menos una vez**, porque la condición se evalúa después del cuerpo.

Ejemplo:

```
int i = 1;
do {
   System.out.println("i vale: " + i);
   i++;
} while (i <= 5);
```

Ejercicio 3.11. Vuelve a hacer el ejercicio 3.10, pero ahora con la estructura do-while.

Ejercicio 3.12. Desarrolla un programa que pida al usuario que introduzca números enteros positivos y los sume. Cuando el usuario introduzca un número negativo, el programa debe terminar y mostrar la suma total de los números introducidos (sin contar el negativo).

3.3.3. Estructura for

La sintaxis de esta estructura es:

```
for (inicialización; condición; actualización) {
   // código a repetir
}
```

Esta estructura es ideal cuando sabemos de antemano cuántas veces se va a repetir.

Ejemplo:

```
for (int i = 1; i <= 5; i++) {
   System.out.println("i vale: " + i);
}
```

Ejercicio 3.13. Desarrolla un programa que muestre por consola los números del 100 al 200 seguidos y separados por un espacio.

Ejercicio 3.14. Desarrolla un programa que muestre por consola los números del 50 al 90 que sean divisibles entre 5 y entre 3.

Ejercicio 3.15. Desarrolla un programa que muestre por consola los números pares comprendidos entre el 21 y el 37.

Ejercicio 3.16. Desarrolla un programa que muestre por consola los números del 15 al 5, es decir, en lugar de incrementar, deberás decrementar la variable de 1 en 1.

3.3.4. Bucles infinitos

Un bucle infinito es un bucle que nunca termina, porque su condición siempre es verdadera. En algunos casos es útil (por ejemplo, en programas que esperan órdenes del usuario), pero si no se gestiona bien, puede bloquear la ejecución del programa.

Ejemplo con while:

```
while (true) {
   System.out.println("Esto se repite para siempre...");
}
```

Este bucle imprimirá el mensaje una y otra vez, sin fin.

3.3.5. Bucles anidados

Un bucle anidado es un bucle dentro de otro bucle. Esto significa que, por cada iteración del bucle externo, se ejecuta por completo el bucle interno. Su sintaxis es:

```
for (int i = 0; i < n; i++) {            // Bucle externo
   for (int j = 0; j < m; j++) {         // Bucle interno
      // Código que se ejecuta m veces por cada vuelta del bucle externo
   }
}
```

Esto también se puede hacer con while o do-while, pero el ejemplo más típico es con for, porque muchas veces sabemos cuántas veces queremos repetir algo.

Ejemplo:

```
for (int fila = 1; fila <= 3; fila++) {
   for (int columna = 1; columna <= 4; columna++) {
      System.out.print("* ");
   }
   System.out.println();  // Salto de línea al terminar cada fila
}
```

Este código imprime por consola:

```
* * * *
* * * *
* * * *
```

Ejercicio 3.17. Muestra una tabla de multiplicar del 1 al 5.

Ejercicio 3.18. Dibuja un rectángulo de asteriscos de 4 filas y 6 columnas.

3.4. Control del flujo

En Java, las instrucciones **break** y **continue** permiten modificar el comportamiento normal de los bucles. La orden break se utiliza para **salir inmediatamente de un bucle**, aunque no se haya completado. Por otro lado, continue **salta el resto del código dentro del bucle** y pasa directamente a la siguiente iteración. Estas instrucciones son útiles cuando se necesita mayor control sobre la repetición, como interrumpir un proceso al cumplir una condición o evitar ejecutar ciertas partes del código en casos concretos.

Ejemplo con break:

```
int contador = 1;

while (true) {
   if (contador > 5) break;  // Rompe el bucle cuando contador > 5
   System.out.println("Contador: " + contador);
   contador++;
}
```

Salida del programa:

```
Contador: 1
Contador: 2
Contador: 3
Contador: 4
Contador: 5
```

Ejemplo con continue: mostrar los números del 1 al 5 excepto el 3:

```
for (int i = 1; i <= 5; i++) {
   if (i == 3) continue; // Salta el 3
   System.out.println(i);
}
```

Salida del programa:

```
1
2
4
5
```

Ejercicio 3.19. Desarrolla un programa que pida al usuario palabras por teclado. Si el usuario escribe “salir”, el programa terminará con el mensaje “Programa finalizado”. (Usa un bucle while infinito y break).

Ejercicio 3.20. Desarrolla un programa que muestre los números del 1 al 10, excepto el 4 y el 7. (Usa for y continue).

3.5. Ejercicios de repaso

Ejercicio 3.21. Desarrolla un programa que muestre por consola los números impares del 39 al 17 usando while.

Ejercicio 3.22. Desarrolla un programa que muestre por consola los números pares del 86 al 60, usando “do...while”.

Ejercicio 3.23. Desarrolla un programa que escriba indefinidamente “no hay fin” en pantalla sin avanzar de línea.

Ejercicio 3.24. Desarrolla un programa que muestre indefinidamente en pantalla los números pares empezando por el 2.

Ejercicio 3.25. Desarrolla un programa que muestre 4 veces los números pares del 10 al 20, en una misma línea.

Ejercicio 3.26. Desarrolla un programa que muestre por consola los números comprendidos entre el 100 y el 130 que sean divisibles entre 5, 6 o 7.

Ejercicio 3.27. Desarrolla un programa que dibuje un cuadrado hueco de arrobas de lado 3. Así:

```
@@@
@   @
@@@
```

Ejercicio 3.28. Desarrolla un programa que pida un número entero de tamaño byte al usuario y muestre por consola los múltiplos de 7 que haya entre 1 y el número introducido.

Ejercicio 3.29. Desarrolla un programa que pida al usuario dos números enteros de 32 bits y busque su MCM (mínimo común múltiplo), es decir, que busque el número más bajo que al dividirlo entre los números que introduce el usuario dé de resto 0. Interrumpe la búsqueda cuando lo hayas encontrado.

Ejercicio 3.30. Desarrolla un programa que muestre por consola los números impares del 85 al 17, excepto los múltiplos de 5. Hazlo con y sin la orden “continue”.

TEMA 4. ARRAYS

Un **array** (o arreglo) es una estructura de datos que permite almacenar varios valores del mismo tipo en una única variable.

Por ejemplo, en lugar de declarar 5 variables int para guardar 5 números, podemos usar un solo array de 5 posiciones. Así:

```
int[] numeros = new int[5];
```

Donde:

- **int[]** indica que estamos creando un **array de enteros**.
- **numeros** es el nombre que le damos a la variable.
- **new int[5]** es donde realmente se **crea el array**. int[5] indica que queremos un array de **5 enteros**. Inicialmente, todos los elementos del array se rellenan con el valor **0** (el valor por defecto de int).

Así, este array tiene 5 posiciones, **numeradas desde el 0 hasta el 4** (siempre empieza en 0). Puedes asignar un valor a cada posición usando el nombre del array seguido del índice entre corchetes:

```
numeros[0] = 10;  // primera posición
numeros[1] = 2;  // segunda posición
numeros[2] = 34;  // tercera posición
numeros[3] = 15;  // cuarta posición
numeros[4] = 51;  // quinta posición
```

En el siguiente ejemplo, vamos a declarar un array de 5 números enteros, le asignaremos valores uno a uno, y luego los mostraremos por pantalla:

```
package CuartoTema;

public class Ejemplo {
   public static void main(String[] args) {
      // Crear un array de 5 enteros
      int[] numeros = new int[5];

      // Asignar valores a cada posición
      numeros[0] = 10;
      numeros[1] = 20;
      numeros[2] = 30;
      numeros[3] = 40;
      numeros[4] = 50;

      // Mostrar los valores
      System.out.println("Contenido del array:");
      System.out.println(numeros[0]);
      System.out.println(numeros[1]);
      System.out.println(numeros[2]);
      System.out.println(numeros[3]);
```

```
        System.out.println(numeros[4]);
    }
}
```

Podemos crear arrays de cualquier tipo de datos y, además, inicializar sus posiciones añadiendo llaves al crearlo. Así:

```
// Array de floats
float[] numerosReales = { 32.3f, 85.6f, 741.325f, 2.6f, 85.35f };

// Array de Strings
String[] capitalesEuropeas = { "madrid", "parís", "londres", "roma" };
```

Si, en lugar de asignar directamente los valores, queremos leer los datos introducidos por el usuario:

```
import java.util.Scanner;

public class Ejemplo2 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int[] numeros = new int[4]; // Declaramos un array de 4 enteros

        // Leemos uno a uno
        System.out.print("Introduce el primer número: ");
        numeros[0] = teclado.nextInt();

        System.out.print("Introduce el segundo número: ");
        numeros[1] = teclado.nextInt();

        System.out.print("Introduce el tercer número: ");
        numeros[2] = teclado.nextInt();

        System.out.print("Introduce el cuarto número: ");
        numeros[3] = teclado.nextInt();

        // Mostramos los valores
        System.out.println("Los números introducidos son:");
        System.out.println(numeros[0]);
        System.out.println(numeros[1]);
        System.out.println(numeros[2]);
        System.out.println(numeros[3]);

        teclado.close();
    }
}
```

Ejercicio 4.1. Desarrolla un programa que declare un array de tipo entero de 32 bits. Después, pedirá al usuario 4 números y los almacenará dentro del array. Terminará mostrando por consola la media aritmética de los números introducidos.

Ejercicio 4.2. Desarrolla un programa que declare un array de números reales de simple precisión (float). Después, pedirá al usuario 5 números y los almacenará dentro del array. Terminará mostrando por consola los números en orden inverso al que se introdujeron.

Ejercicio 4.3. Desarrolla un programa que declare un array de números enteros de tamaño byte con el número de días que tiene cada mes. Después, mostrará por consola los datos almacenados.

Como ya hemos visto en el tema anterior, el bucle for es una estructura de control que nos permite repetir un bloque de código un número determinado de veces. Es especialmente útil cuando queremos recorrer arrays o ejecutar una acción un número fijo de veces.

```
int[] numeros = new int[5]; // Declaro un array de 5 posiciones de enteros

for (int i = 0; i < 5; i++) {  // i representa cada una de las posiciones del array
   System.out.println(numeros[i]); // Escribo por consola todas las posiciones del array
}
```

Como ves, se agiliza mucho el proceso de leer y escribir datos en un array.

El array tiene un tamaño fijo, en este caso 5 posiciones (índices del 0 al 4). Para saber cuántos elementos tiene un array, usamos la propiedad **length**. Esta propiedad nos dice el número total de posiciones que tiene el array.

Esto es muy útil porque, si queremos recorrer todas las posiciones del array, en lugar de escribir números fijos (como i < 5), podemos escribir:

```
for (int i = 0; i < numeros.length; i++) {
   // hacemos algo con numeros[i]
}
```

Así, si cambiamos el tamaño del array, no tenemos que cambiar el número en el bucle.

Ejercicio 4.4. Realiza el ejercicio 4.1 utilizando bucles for y la propiedad length.

Ejercicio 4.5. Realiza el ejercicio 4.2. utilizando bucles for y la propiedad length.

Ejercicio 4.6. Realiza el ejercicio 4.3. utilizando bucles for y la propiedad length.

Ejercicio 4.7. Desarrolla un programa que pida al usuario 6 números enteros largos y luego los muestre por consola en orden inverso al que se introdujeron.

Ejercicio 4.8. Desarrolla un programa que pida al usuario la longitud de un array de float (es decir, cuántos números quiere introducir). Después, recorrerá el array para calcular la media y mostrarla por consola.

Ejercicio 4.9. Desarrolla un programa que pida al usuario 10 números reales de doble precisión. Se guardarán esos valores en un array, calculará la media y luego mostrará por consola todos los valores que estén por encima de la media.

Ejercicio 4.10. Desarrolla un programa que declare un array para almacenar 100 nombres. Se pedirá al usuario que introduzca nombres de manera repetitiva hasta que se pulse la tecla "intro" (se representa con una cadena vacía). Después, el programa mostrará los nombres que se han introducido hasta ese momento.

4.1. Operaciones básicas con arrays

Hasta ahora, hemos aprendido a declarar arrays, introducir datos en ellos y recorrerlos con bucles. Ahora vamos a dar un paso más: **manipular el contenido del array**.

En muchos programas, necesitamos **buscar datos, insertar nuevos elementos, eliminar valores o modificar contenido**. Aunque Java tiene estructuras más avanzadas como ArrayList (se verá más adelante), por ahora aprenderemos a hacer estas operaciones **manualmente con arrays simples**, para entender mejor cómo funciona la lógica interna.

Recuerda que los arrays tienen un tamaño fijo, es decir, son estáticos. Esto significa que el array tendrá **exactamente las posiciones con las que se declara**. No podemos añadir más, ni reducir su tamaño. Podemos **modificar los valores** de esas posiciones, pero no cambiar cuántas hay.

Las operaciones básicas que podemos realizar son (ver tabla 4.1):

Operación	¿Qué hace?
Buscar	Encontrar un valor dentro del array
Añadir	Insertar un nuevo valor al final del array (si hay espacio disponible)
Eliminar	Quitar un valor y reorganizar el array para llenar el hueco
Insertar	Añadir un valor en una posición concreta (desplazando los demás)

Tabla 4.1. Operaciones básicas en arrays. Elaboración propia.

Vamos a ver un ejemplo de cómo **buscar** información dentro de un array:

```
package CuartoTema;

import java.util.Scanner;

public class BuscarElemento {
   public static void main(String[] args) {
      int[] numeros = {10, 20, 30, 40, 50};
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un número para buscar: ");
      int buscado = teclado.nextInt();

      boolean encontrado = false;
```

```
        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] == buscado) {
                System.out.println("Número encontrado en la posición " + i);
                encontrado = true;
                break;
            }
        }

        if (!encontrado) {
            System.out.println("Número no encontrado.");
        }
    }
}
```

En este ejemplo creamos un array con 5 valores y pedimos al usuario un número. Luego recorremos el array con un bucle for. Si encontramos ese número (numeros[i] == buscado), mostramos su posición y usamos break para salir del bucle, pero si el número no está, mostramos un mensaje indicando que no se encontró.

Ahora un ejemplo para ver cómo **añadir** datos al final del array:

```
package CuartoTema;

import java.util.Scanner;

public class AnadirElemento {
    public static void main(String[] args) {
        int[] numeros = new int[5];
        int contador = 0;
        Scanner teclado = new Scanner(System.in);

        while (contador < numeros.length) {
            System.out.print("Introduce un número (o -1 para terminar): ");
            int n = teclado.nextInt();

            if (n == -1) break;

            numeros[contador] = n;
            contador++;
        }

        System.out.println("Contenido del array:");
        for (int i = 0; i < contador; i++) {
            System.out.print(numeros[i] + " ");
        }
    }
}
```

En este ejemplo, creamos un array de tamaño 5 (vacío al principio), luego usamos un bucle while para permitir que el usuario introduzca hasta 5 números. El bucle se detiene cuando el usuario introduce -1 o se llenan todas las posiciones. Al final, mostramos por consola los valores introducidos.

El siguiente ejemplo es un poco más complejo. Vamos a **eliminar** un dato dentro del array. Para ello hay que restructurar el array. Observa:

```
package CuartoTema;

import java.util.Scanner;

public class EliminarElemento {
   public static void main(String[] args) {
      int[] numeros = {10, 20, 30, 40, 50};
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un número para eliminar: ");
      int eliminar = teclado.nextInt();

      int nuevoTam = numeros.length;
      boolean encontrado = false;

      for (int i = 0; i < nuevoTam; i++) {
         if (numeros[i] == eliminar) {
            // Mover todo a la izquierda desde esa posición
            for (int j = i; j < nuevoTam - 1; j++) {
               numeros[j] = numeros[j + 1];
            }
            nuevoTam--;
            encontrado = true;
            break;
         }
      }

      if (encontrado) {
         System.out.println("Array después de eliminar:");
         for (int i = 0; i < nuevoTam; i++) {
            System.out.print(numeros[i] + " ");
         }
      } else {
         System.out.println("Número no encontrado.");
      }
   }
}
```

En este ejemplo, el usuario introduce el valor a eliminar (no el índice). Entonces recorremos el array y cuando lo encontramos desplazamos hacia la izquierda todos los elementos a partir de esa posición. Luego reducimos el tamaño virtual del array (nuevoTam--).

Para finalizar, mostramos el array actualizado y, si el valor no se encuentra, se informa al usuario.

La última operación que veremos es un poco más compleja. Se trata de **insertar** un dato en la posición que quiera el usuario. Para ello, hay que desplazar el resto de las posiciones del array. Observa este ejemplo:

```
package CuartoTema;

import java.util.Scanner;

public class InsertarElemento {
   public static void main(String[] args) {
      int[] numeros = new int[6]; // array con espacio extra
      int[] originales = {10, 20, 30, 40, 50};
      int elementos = 5;

      // Copiamos los originales al nuevo array
      for (int i = 0; i < elementos; i++) {
         numeros[i] = originales[i];
      }

      Scanner teclado = new Scanner(System.in);
      System.out.print("Introduce el número a insertar: ");
      int nuevo = teclado.nextInt();

      System.out.print("Introduce la posición (0 a 5): ");
      int pos = teclado.nextInt();

      if (pos < 0 || pos > elementos) {
         System.out.println("Posición inválida.");
      } else {
         for (int i = elementos; i > pos; i--) {
            numeros[i] = numeros[i - 1];
         }
         numeros[pos] = nuevo;
         elementos++;

         System.out.println("Array después de insertar:");
         for (int i = 0; i < elementos; i++) {
            System.out.print(numeros[i] + " ");
         }
      }
   }
}
```

En este ejemplo, creamos un array con espacio para un elemento adicional, copiamos los valores originales en él, pedimos al usuario un número y la posición donde quiere insertarlo, **desplazamos los valores desde esa posición hacia la derecha para hacer hueco**, insertamos el nuevo valor y mostramos el array actualizado.

Ejercicio 4.11. Desarrolla un programa que declare un array que guarde 5 palabras. Después pedirá al usuario una palabra que quiera buscar dentro del array. El programa avisará si el dato está o no y, además, en el caso de encontrarse en el array, dirá cuántas veces aparece.

Ejercicio 4.12. Desarrolla un programa que declare un array de 5 números enteros cortos. El array estará vacío y le pedirá al usuario que introduzca un número para añadirlo al array.

Ejercicio 4.13. Desarrolla un programa que declare un array para guardar 5 caracteres (char). El array tendrá de inicio 3 posiciones ocupadas con los caracteres 'r', 's' y 'd'. Después le pedirá al usuario que introduzca un nuevo carácter y la posición en la que lo quiere insertar. El programa insertará el nuevo carácter y mostrará el resultado final por consola.

Ejercicio 4.14. Desarrolla un programa que declare un array de 5 números enteros de simple precisión. El array tendrá de inicio 4 posiciones ocupadas con los valores que quieras. Después preguntará al usuario qué posición del array quiere borrar. El programa, tras borrar la posición, mostrará el contenido del array por consola.

NOTA: anteriormente vimos que, cuando queremos comparar **dos cadenas de texto** (Strings), **no podemos usar ==**, porque eso compara si están en la misma **posición de memoria**, no si tienen el mismo contenido. Para comparar su **contenido**, usamos el método equals.

Este método **distingue mayúsculas y minúsculas**, es decir, "Hola" y "hola" se consideran **diferentes**. Pero si queremos **comparar sin importar las mayúsculas o minúsculas**, usamos el método equalsIgnoreCase(). Así:

```
cadena1.equalsIgnoreCase(cadena2)
```

Ejemplo:

```
String palabra1 = "Madrid";
String palabra2 = "madrid";

if (palabra1.equalsIgnoreCase(palabra2)) {
   System.out.println("Las palabras son iguales (ignorando mayúsculas/minúsculas).");
} else {
   System.out.println("Las palabras son diferentes.");
}
```

4.2. Recorriendo el array: Bucle foreach

El bucle foreach, también llamado **enhanced for loop**, se usa para **recorrer colecciones de datos** (como arrays) de una forma más sencilla y legible que el bucle for tradicional. Su sintaxis es:

```
for (Tipo elemento : array) {
   // Acciones con cada elemento
}
```

Mira este ejemplo:

```
public class EjemploForeach1 {
   public static void main(String[] args) {
      int[] numeros = { 10, 20, 30, 40, 50 };

      for (int n : numeros) {
```

```
        System.out.println("Número: " + n);
    }
  }
}
```

Como ves, utilizar el bucle foreach en lugar del for clásico tiene varias ventajas:

- Más claro y limpio que el bucle for clásico.
- Evita errores al manejar los índices.
- Ideal para **leer o mostrar** elementos de un array.

Es decir, es conveniente usarlo si tienes que recorrer todos los elementos del array para mostrarlos por consola o para buscar un valor dentro.

Ejercicio 4.15. Desarrolla un programa que tenga un array de 4 notas (números reales de tipo float). El programa recorrerá el array usando un bucle foreach para **calcular la suma de todas las notas**. Finalmente, mostrará por consola el resultado de la suma.

Ejercicio 4.16. Desarrolla un programa que tenga un array con 4 nombres de colores (String). Después, se pedirá al usuario que escriba un color. Usando un bucle foreach, el programa buscará si ese color se encuentra en el array (ignorando mayúsculas y minúsculas). Si lo encuentra, dirá **"Color encontrado"**; si no, dirá **"Color no encontrado"**.

4.3. Arrays bidimensionales

Hasta ahora hemos trabajado con arrays unidimensionales, que se pueden imaginar como una fila de casillas numeradas donde se almacena información (números, cadenas, caracteres, etc.). Pero ¿qué pasa si queremos trabajar con tablas o matrices de datos? Ahí es donde entran en juego los arrays bidimensionales.

Un array bidimensional es un array con filas y columnas, como una tabla de Excel.

Por ejemplo, imagina que tienes esta tabla:

```
1  2  3
4  5  6
7  8  9
```

Esto es una matriz de 3 filas y 3 columnas. En Java, esto se puede representar como:

```
int[][] matriz = {
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}
};
```

Donde:

- matriz[0][0] es 1 (primera fila, primera columna),

- matriz[1][2] es 6 (segunda fila, tercera columna),
- matriz[2][1] es 8 (tercera fila, segunda columna), etc.

Hay dos formas principales de declarar un array bidimensional:

1. Con tamaño fijo:

```
int[][] tabla = new int[3][4]; // 3 filas y 4 columnas
```

Esto crea un array de enteros donde cada fila contiene 4 columnas (valores por defecto: 0).

2. Inicializado con valores directamente:

```
int[][] datos = {
   {10, 20, 30},
   {40, 50, 60}
};
```

Aquí el array tiene 2 filas y 3 columnas. Para acceder a una celda concreta, usamos dos índices:

```
int valor = datos[1][2]; // accede al valor 60
datos[0][0] = 100;      // cambia el primer valor a 100
```

Para recorrer un array bidimensional normalmente se usan dos bucles for anidados: uno para las filas, y otro para las columnas. Así:

```
for (int i = 0; i < datos.length; i++) { // Recorre filas
   for (int j = 0; j < datos[i].length; j++) { // Recorre columnas
      System.out.print(datos[i][j] + " ");
   }
   System.out.println(); // Salto de línea tras cada fila
}
```

Los arrays bidimensionales son útiles cuando queremos representar datos organizados en filas y columnas, como por ejemplo:

- Tablas de multiplicar
- Matrices matemáticas
- Tableros de juegos (tres en raya, ajedrez)
- Datos estructurados (horarios, calificaciones, mapas)

Ejercicio 4.17. Desarrolla un programa que declare un array bidimensional de 2 bloques de 10 números enteros. Después, pedirá al usuario datos para rellenar el array. El programa terminará mostrando el mayor dato que se ha introducido.

NOTA: Integer.MIN_VALUE es una constante predefinida que representa el valor mínimo que puede almacenar una variable de tipo int. Esto es útil cuando necesitamos inicializar una variable con el valor más bajo para hacer comparaciones, como en este caso, que queremos encontrar el número **mayor** en una lista de enteros. Así:

```
int mayor = Integer.MIN_VALUE;
```

De esta forma, nos aseguramos de que **cualquier número que introduzca el usuario será mayor** que ese valor inicial, y por tanto se actualizará correctamente la variable mayor.

Usar Integer.MIN_VALUE no es obligatorio. También podemos inicializar la variable mayor con el primer número que introduce el usuario, y luego seguir comparando el resto.

4.4. Arrays tridimensionales

Un array tridimensional puede entenderse como una **colección de matrices**. Si los arrays bidimensionales representan **tablas** (como una hoja de cálculo), los arrays tridimensionales representan algo así como un **cubo de datos** o una **serie de tablas** apiladas.

Ejemplo:

```
int[][][] cubo = new int[2][3][4];
```

Esto se puede visualizar como:

- 2 bloques o "capas" de matrices,
- cada bloque tiene 3 filas,
- cada fila tiene 4 columnas.

Para acceder a un elemento de este array haríamos: cubo[0][1][2] (significa: **bloque 0**, **fila 1**, **columna 2**).

Podríamos necesitar usar un array de 3 dimensiones:

- En juegos en 3D: por ejemplo, en un juego tipo Minecraft, donde necesitas almacenar información en un espacio 3D: alto, ancho y profundidad.
- Para almacenar datos temporales o secuenciales. Supón que guardas datos de temperatura por días, horas, y ciudades. Podrías tener:

```
float[][][] temperatura = new float[dias][horas][ciudades];
```

- Imágenes o gráficos en capas: en procesamiento de imágenes, podrías tener arrays tridimensionales que representen:
    - Capas de color (RGB)
    - Píxeles (filas y columnas)
    - Capas o frames
- Simulaciones científicas o físicas: cuando simulas volúmenes, como una red cúbica con propiedades físicas en cada punto (presión, temperatura, velocidad...).

NOTA: Los arrays de **más dimensiones** (int[][][][], etc.) se vuelven muy difíciles de visualizar y manipular. Normalmente, a partir de 3 dimensiones, se empieza a considerar alternativas más avanzadas, como:

- Objetos personalizados
- Estructuras de datos más flexibles (listas, mapas, etc.)
- Librerías especializadas (como matrices multidimensionales en bibliotecas científicas)

4.5. Array de arrays

Un array de arrays es una estructura en la que cada elemento del array principal contiene otro array. Es una forma flexible de organizar datos en "filas" de **diferente longitud**.

A diferencia de un array bidimensional regular (como int[][] matriz = new int[3][4]), en un array de arrays **cada fila puede tener una longitud distinta**.

Su sintaxis es:

```
int[][] datos = new int[3][]; // 3 filas, sin columnas aún definidas

// Asignar longitud distinta a cada fila
datos[0] = new int[2]; // Fila 0 con 2 columnas
datos[1] = new int[4]; // Fila 1 con 4 columnas
datos[2] = new int[3]; // Fila 2 con 3 columnas
```

Los arrays de arrays se usan cuando trabajas con datos que naturalmente tienen estructuras irregulares. Por ejemplo:

- Notas de estudiantes donde cada uno hizo diferente cantidad de exámenes.
- Tableros de juego de forma variable.
- Listas de tareas con diferente número de subtareas por elemento.
- Horarios donde no todos los días tienen la misma cantidad de clases.

Mira este ejemplo:

```
public class ArrayDeArrays {
   public static void main(String[] args) {
      int[][] calificaciones = new int[3][]; // 3 alumnos

      // Asignamos diferente número de notas a cada alumno
      calificaciones[0] = new int[]{8, 7};          // Alumno 1 tiene 2 notas
      calificaciones[1] = new int[]{6, 9, 8};        // Alumno 2 tiene 3 notas
      calificaciones[2] = new int[]{10, 7, 6, 8};     // Alumno 3 tiene 4 notas

      for (int i = 0; i < calificaciones.length; i++) {
         System.out.print("Alumno " + (i + 1) + ": ");
         for (int j = 0; j < calificaciones[i].length; j++) {
            System.out.print(calificaciones[i][j] + " ");
```

```
        }
        System.out.println();
      }
    }
}
```

Cuya salida por consola sería:

```
Alumno 1: 8 7
Alumno 2: 6 9 8
Alumno 3: 10 7 6 8
```

Ejercicio 4.18. Desarrolla un programa que pida al usuario 2 números enteros de tamaño byte. Luego declarará un array de 2 arrays con la longitud de los números que introdujo el usuario. Más tarde, se le pedirá al usuario que rellene ambos bloques de números. Finalmente deberá mostrar la media de cada uno de los bloques.

4.6. Ejercicios de repaso

Ejercicio 4.19. Declara un array de 6 números enteros. Pide al usuario que introduzca los valores. Después, usando un bucle foreach, muestra el número más pequeño del array.

Ejercicio 4.20. Declara un array de 5 palabras (String). Pide al usuario que las introduzca una a una. Después, muestra todas las palabras en orden inverso.

Ejercicio 4.21. Declara un array bidimensional de 2 filas y 3 columnas con números enteros. Pide al usuario que rellene el array. Después, muestra por consola la suma de cada fila por separado.

Ejercicio 4.22. Declara un array de arrays de tipo float, donde haya 2 bloques de 4 elementos cada uno. Pide al usuario que introduzca los datos. Calcula y muestra la media de cada bloque utilizando un bucle foreach.

Ejercicio 4.23. Declara un array bidimensional de String (3x2) con nombres de frutas. Pide al usuario que introduzca una palabra. Usando un bucle foreach, di si esa palabra se encuentra en el array (sin importar mayúsculas o minúsculas).

Ejercicio 4.24. Crea un array de enteros con los valores {10, 20, 30, 40, 50}. Recorre el array primero con for y luego con foreach, mostrando todos los valores por consola en ambos casos.

Ejercicio 4.25. Declara un array de 10 enteros. Pide al usuario que lo rellene. Después, pide un número y muestra cuántas veces aparece ese número en el array.

Ejercicio 4.26. Declara un array de 5 caracteres (char) con los valores 'a', 'e', 'i', 'o', 'u'. Pide al usuario un carácter y una posición (de 0 a 4) y reemplaza el carácter en esa posición. Muestra el array resultante con foreach.

Ejercicio 4.27. Declara dos arrays de 4 enteros cada uno. Pide al usuario que introduzca los valores de ambos arrays. El programa debe decir cuántos elementos coinciden en la misma posición.

Ejercicio 4.28. Declara un array bidimensional de 3 filas y 10 columnas. Cada fila contendrá la tabla de multiplicar del 2, 3 y 4 respectivamente. Después, muestra las tablas por consola usando foreach.

Ejercicio 4.29. Crea un array de 5 números reales que representen temperaturas en grados Celsius. Transforma todas a Fahrenheit y muestra el resultado. Fórmula: $F = C \times 9/5 + 32$.

Ejercicio 4.30. Declara un array de 6 palabras (String). Pide al usuario que introduzca las palabras. Luego, usando foreach, muestra la palabra más larga de todas.

TEMA 5: MANIPULACIÓN DE CADENAS DE TEXTO

Como hemos visto, las cadenas de texto (String) en Java son uno de los tipos de datos más usados y fundamentales para cualquier programa que interactúe con el usuario o maneje información textual. Técnicamente, una cadena es una secuencia de caracteres, lo que la hace conceptualmente similar a un array de caracteres (char[]). Sin embargo, a diferencia de los arrays, las cadenas en Java son objetos inmutables que ofrecen muchas facilidades para manipular texto de forma sencilla, segura y eficiente.

Habiendo trabajado ya con arrays unidimensionales y multidimensionales, estamos preparados para entender cómo podemos acceder, recorrer y transformar cadenas usando métodos específicos, muchos de los cuales internamente trabajan con arrays de caracteres, pero con una interfaz mucho más amigable.

5.1. Variables de tipo String

Una cadena de texto es un objeto de la clase String que representa una secuencia de caracteres. Por ejemplo, "Hola Mundo" es una cadena de texto que contiene 10 caracteres (contando el espacio).

Las cadenas de texto String son:

- **Inmutables:** Una vez creada, una cadena no puede ser modificada. Por ejemplo, si concatenas algo a una cadena, en realidad se crea una nueva cadena con el resultado.
- **Objetos:** Aunque podemos usar cadenas fácilmente, internamente son objetos de la clase String, que ofrece muchas “herramientas” para trabajar con ellas. Es decir, no son variables de tipo primitivo.
- **Secuencia de caracteres:** Conceptualmente, es similar a un array de char, pero con ventajas significativas en manejo y seguridad.

Ya hemos visto en un tema anterior cómo se declaran:

```
String texto = "Hola";
```

Podemos acceder a cada carácter de una cadena usando charAt(int índice). Por ejemplo:

```
String saludo = "Hola";
char letra = saludo.charAt(1);  // 'o', porque los índices empiezan en 0
```

Los índices empiezan en 0, así que charAt(0) es la primera letra.

Para saber cuántos caracteres tiene una cadena, usamos length():

```
int longitud = saludo.length();  // 4
```

¡OJO! No confundir con la propiedad length que usábamos en los bucles for para recorrer arrays.

Recuerda que para comparar cadenas no se debe usar ==, ya que compara referencias de objetos, no su contenido.

En su lugar usamos:

- **equals(String otraCadena):** devuelve true si las cadenas son iguales en contenido.
- **equalsIgnoreCase(String otraCadena):** igual que equals pero ignorando si son mayúsculas o minúsculas.

Ejemplo:

```
String a = "hola";
String b = "Hola";

System.out.println(a.equals(b));          // false
System.out.println(a.equalsIgnoreCase(b)); // true
```

Ejercicio 5.1. Desarrolla un programa que pida una palabra al usuario y después la muestre en consola separando los caracteres.

Ejercicio 5.2. Desarrolla un programa que pida una frase al usuario y después la muestre por consola en orden inverso.

Como decía, las cadenas de texto objetos inmutables, lo que significa que **no se pueden modificar directamente**. Sin embargo, Java proporciona herramientas para obtener una **nueva cadena modificada** a partir de la original.

Una de estas herramientas es la concatenación vista anteriormente (operador +):

```
String saludo = "Hola";
String nombre = "Carlos";
String mensaje = saludo + ", " + nombre + "!";
System.out.println(mensaje);  // Resultado: Hola, Carlos!
```

Para obtener este mismo resultado, también se puede usar **concat()**. Así:

```
String mensaje = saludo.concat(", ").concat(nombre).concat("!");
System.out.println(mensaje);  // Resultado: Hola, Carlos!
```

Para cambiar de mayúsculas a minúsculas y viceversa, podemos usar **toUpperCase()** y **toLowerCase()**. Así:

```
String texto = "Java es genial";
System.out.println(texto.toUpperCase()); // JAVA ES GENIAL
System.out.println(texto.toLowerCase()); // java es genial
```

Podemos eliminar espacios sobrantes al principio y al final de la cadena con **trim().** Así:

```
String entrada = "   texto con espacios   ";
```

```
String limpio = entrada.trim();  // "texto con espacios"
```

También podemos convertir la cadena en array con **toCharArray():**

```
String palabra = "hola";
char[] letras = palabra.toCharArray();

for (int i = 0; i < letras.length; i++) {
   System.out.println(letras[i]);
}
```

Esta conversión es muy útil cuando quieres trabajar con cada letra de una palabra como si fuera un elemento de un array.

Ejercicio 5.3. Desarrolla un programa que pida al usuario su nombre y sus apellidos. Después realizará las siguientes operaciones:

1. Eliminará espacios al principio y al final de cada dato si los hubiera.
2. Unirá el nombre y los apellidos en una sola cadena utilizando concat().
3. Mostrará el nombre completo en mayúsculas y en minúsculas.
4. Mostrará uno a uno los caracteres del nombre completo usando toCharArray().

Para extraer una parte de una cadena utilizamos **substring(int inicio)** y **substring(int inicio, int fin)**.

- substring(inicio) devuelve desde esa posición hasta el final.
- substring(inicio, fin) devuelve desde la posición inicio (incluida) hasta fin (excluida).

```
String texto = "programación";
String parte = texto.substring(0, 5); // "progr"
```

Para buscar una cadena dentro de otra podemos usar contains(). Devuelve true o false, es decir, sí está o no está. Así:

```
String frase = "Hola mundo";
boolean tieneHola = frase.contains("Hola"); // true
```

Otra herramienta para buscar cadenas dentro de otras es **indexOf(String texto)** que devuelve la **posición de la primera aparición** de una subcadena si encuentra lo que está buscando. En caso contrario devuelve -1. Así:

```
String frase = "Hola mundo";
int posicion = frase.indexOf("mundo"); // devuelve la posición 5
```

startsWith(String texto) devuelve true si la cadena **empieza con la secuencia de caracteres** dada. Así:

```
String saludo = "Hola mundo";
boolean empieza = saludo.startsWith("Hola"); // true
```

endsWith(String sufijo) comprueba si una cadena termina con el texto indicado. Devuelve true o false.

```
String archivo = "informe_final.pdf";
if (archivo.endsWith(".pdf")) {
   System.out.println("Es un archivo PDF.");
}
```

Si queremos reemplazar un fragmento de cadena por otro podemos utilizar replace(). Tanto para **caracteres** como para **subcadenas**. Así:

```
String texto = "banana";
String resultado = texto.replace('a', 'o');
System.out.println(resultado);  // Salida: bonono

String frase = "Me gusta el helado de fresa";
String nuevaFrase = frase.replace("fresa", "chocolate");
System.out.println(nuevaFrase);  // Salida: Me gusta el helado de chocolate
```

Y, si queremos que reemplace a lo largo de todo el texto, podemos usar **replaceAll(String antigua, String nueva)**. Así:

```
String texto = "hoy es lunes, lunes soleado";
String resultado = texto.replaceAll("lunes", "martes");
System.out.println(resultado);  // hoy es martes, martes soleado
```

Ejercicio 5.4. Desarrolla un programa que pida al usuario una frase y una palabra. Después mostrará:

1. Si la frase contiene esa palabra.
2. En qué posición empieza esa palabra (si está).
3. Si la frase comienza con esa palabra.
4. Además, mostrará desde el inicio de la palabra hasta el final de la frase (substring()).

Ejercicio 5.5. Desarrolla un programa que pida al usuario una frase. Luego, aplicará las siguientes operaciones y mostrará el resultado por consola:

1. Elimina los espacios al principio y al final.
2. Convierte toda la frase a mayúsculas.

3. Convierte toda la frase a minúsculas.
4. Muestra el primer carácter.
5. Muestra los primeros 5 caracteres.
6. Cuenta cuántas veces aparece la letra 'a'.

Ejercicio 5.6. Desarrolla un programa que pida al usuario 5 frases que se guardarán en un array. Luego pedirá cadenas y las buscará por todo el array para ver si se encuentran. No es necesario saber la posición. El programa terminará cuando el usuario escriba “fin”.

Ejercicio 5.7. Desarrolla una versión del ejercicio anterior en la que el programa, en caso de encontrar la cadena en el array, avise si se encuentra en la primera posición de alguna de las frases.

Para descomponer una cadena en fragmentos usamos **split(separador)**. Esto divide una cadena en partes utilizando un **separador**. Devuelve un array de String con las partes. Así:

```
String[] partes = texto.split(" ");
```

Ejemplo:

```
String frase = "Hola qué tal estás";
String[] palabras = frase.split(" "); //Cuando vea un espacio en blanco, corta y separa
System.out.println("Número de palabras: " + palabras.length);
```

Ejercicio 5.8. Pide al usuario una frase y muestra cuántas palabras tiene. Utiliza split(" ").

Ejercicio 5.9. Desarrolla un programa que pida al usuario una frase y la descomponga en palabras. El delimitador será el espacio en blanco. Después, mostrará por consola los fragmentos extraídos en orden inverso.

Ejercicio 5.10. Desarrolla un programa que pida al usuario varios números enteros separados por espacios en blanco (recuerda que los números son caracteres salvo que los conviertas a números). Después se fragmentará la cadena cuando se encuentre un espacio y se guardará cada uno de los números enteros en un array. El programa terminará mostrando por consola la suma de todos los números del array (recuerda que si el número entero no se convierte al tipo int o cualquier otro no podrás operar con él).

NOTA: Aunque ya hemos visto el *casting* entre tipos numéricos (como de float a int), es importante saber que **no se puede convertir un texto directamente a un número usando (int)**. En su lugar, necesitamos usar la “herramienta” Integer.parseInt(String), que **convierte una cadena con contenido numérico (por ejemplo, "45") en un número entero real (45)**. Esto se debe a que el texto ("45") y el número (45) son tipos de datos completamente distintos y no compatibles para el casting directo.

Hay más formas de conversión de texto a número (ver tabla 5.1):

Método	Devuelve un...	Ejemplo de uso
Integer.parseInt(String)	int	int n = Integer.parseInt("25");
Double.parseDouble(String)	double	double d = Double.parseDouble("3.14");
Float.parseFloat(String)	float	float f = Float.parseFloat("5.6");
Long.parseLong(String)	long	long l = Long.parseLong("123456789");
Short.parseShort(String)	short	short s = Short.parseShort("10");
Byte.parseByte(String)	byte	byte b = Byte.parseByte("100");

Tabla 5.1. Métodos de conversión de texto a número. Elaboración propia.

5.2. Variables de tipo StringBuilder

Como hemos dicho, Java trata las cadenas (String) como inmutables, lo que significa que cada vez que modificas una cadena, se crea un nuevo objeto en memoria.

Esto puede ser **ineficiente si haces muchas modificaciones**, como concatenaciones en bucles o inserciones.

StringBuilder es una clase (como Scanner) que posee "herramientas" para **modificar directamente el contenido** sin crear nuevos objetos cada vez, es decir, es más eficiente que String para múltiples modificaciones. Estas "herramientas" son append(), insert(), replace(), delete(), reverse() o toString().

Lo único que tenemos que hacer es crear una nueva cadena StringBuilder. Así:

```
StringBuilder sb = new StringBuilder();            // Vacío
StringBuilder sb2 = new StringBuilder("Hola mundo"); // Con contenido inicial
```

- **append(String texto)** agrega texto al final:

```
StringBuilder sb = new StringBuilder("Hola");
sb.append(" mundo");
System.out.println(sb);  // Hola mundo
```

- **insert(int posicion, String texto)** inserta texto en una posición específica:

```
StringBuilder sb = new StringBuilder("Hola mundo");
sb.insert(5, "querido ");
System.out.println(sb);  // Hola querido mundo
```

- **replace(int inicio, int fin, String texto)** reemplaza los caracteres entre dos posiciones por un nuevo texto:

```
StringBuilder sb = new StringBuilder("Hola mundo cruel");
```

```
sb.replace(11, 16, "Java");
System.out.println(sb);  // Hola mundo Java
```

RECUERDA: Para las variables de tipo String también existe un **replace(),** pero funciona de forma distinta.

- **setCharAt(int index, char nuevo)** cambia un carácter por otro en una posición:

```
StringBuilder sb = new StringBuilder("luz");
sb.setCharAt(0, 'C');
System.out.println(sb);  // Cuz
```

- **delete(int inicio, int fin)** elimina caracteres entre las posiciones indicadas:

```
StringBuilder sb = new StringBuilder("Hola querido mundo");
sb.delete(5, 13);
System.out.println(sb);  // Hola mundo
```

- **deleteCharAt(int index)** elimina el carácter en la posición indicada:

```
StringBuilder sb = new StringBuilder("gato");
sb.deleteCharAt(1);  // elimina 'a'
System.out.println(sb);  // gto
```

- **reverse()** invierte todo el contenido:

```
StringBuilder sb = new StringBuilder("Hola");
sb.reverse();
System.out.println(sb);  // aloH
```

- **toString()** convierte el contenido a un String si necesitas manipularlo como tal o imprimirlo de forma más convencional:

```
StringBuilder sb = new StringBuilder("Texto");
String resultado = sb.toString();
```

NOTA 1: Un objeto StringBuilder se puede imprimir porque println() acepta cualquier objeto y convierte su contenido a cadena mediante toString().

NOTA 2: Al contrario que hacíamos con la clase Scanner, no hay que importar el paquete dentro de nuestro proyecto, ya que la clase StringBuilder pertenece al paquete **java.lang** y todo lo que está en java.lang **se importa automáticamente** en todos los programas Java.

Ejercicio 5.11. Desarrolla un programa que pida al usuario una frase y sustituya todas las “a” que contenga la cadena por “o”.

Ejercicio 5.12. Desarrolla un programa que pida al usuario una palabra. Luego realizará las siguientes operaciones usando la clase StringBuilder:

1. Insertar un asterisco * al principio y al final de la palabra.
2. Reemplazar la primera letra por una #.
3. Eliminar el tercer carácter (posición 2).
4. Mostrar el texto final.

5.3. Ejercicios de repaso

Ejercicio 5.13. Pide una frase al usuario. Elimina todos los espacios con replaceAll() y muestra la frase sin espacios. Utiliza StringBuilder.

Ejercicio 5.14. Pide al usuario un nombre de archivo. Si termina en ".jpg" o ".png", muestra “Es una imagen”. Usa endsWith().

Ejercicio 5.15. Pide una frase al usuario. Usa StringBuilder para eliminar el primer carácter y el último usando deleteCharAt().

Ejercicio 5.16. Pide una frase al usuario. Luego cambia el carácter del medio por un asterisco usando setCharAt() (si es par, cambia el de la izquierda).

Ejercicio 5.17. Pide una frase al usuario y muestra su versión invertida utilizando reverse().

Ejercicio 5.18. Pide una frase al usuario. Muestra la misma frase en mayúsculas, minúsculas y sin espacios al principio ni al final.

Ejercicio 5.19. Pide una palabra y muestra si comienza con “pre” y si termina con “ción”.

Ejercicio 5.20. Pide una frase y muestra las 3 primeras letras y las 3 últimas.

Ejercicio 5.21. Pide una frase y un carácter. Luego reemplaza ese carácter en la posición central por un guion.

Ejercicio 5.22. Pide al usuario varios números separados por espacios. Suma los números e indica cuál es el mayor.

Ejercicio 5.23. Pide una frase y usa StringBuilder para invertirla y mostrar el resultado.

Ejercicio 5.24. Pide una frase y elimina los dos primeros y los dos últimos caracteres.

Ejercicio 5.25. Pide una frase y una palabra. Muestra si la palabra aparece en la frase, sin importar mayúsculas o minúsculas.

Ejercicio 5.26. Desarrolla un programa que simule el juego del ahorcado:

1. El programa tendrá una **palabra oculta** que el jugador debe adivinar.
2. Se mostrará al jugador una versión oculta de la palabra con guiones bajos (_) por cada letra.
3. El jugador irá introduciendo letras, una por una.
4. Si la letra está en la palabra, se revelará en su(s) posición(es) correspondiente(s).

5. Si no está, se contará un fallo.
6. El jugador tendrá un máximo de **6 intentos fallidos**.
7. El juego termina cuando se adivina la palabra o si se superan los 6 fallos.

TEMA 6. CONTROL DE EXCEPCIONES

Una **excepción** es un evento que ocurre durante la ejecución de un programa que interrumpe el flujo normal. Este evento suele indicar que algo inesperado o un error ha ocurrido, como intentar dividir por cero, acceder a una posición inválida de un array o leer un archivo que no existe.

Si no se gestionan las excepciones, el programa se detendrá repentinamente con un error en tiempo de ejecución (runtime error).

Por este motivo es importante controlar las posibles excepciones del programa, no solo para evitar que el programa termine de forma inesperada, sino para informar al usuario de forma clara qué ha ocurrido.

La estructura básica para el manejo de excepciones se realiza a través de dos bloques específicos: **try** y **catch**. Así:

```
try {
   // Código que puede lanzar una excepción
} catch (TipoDeExcepcion e) {
   // Código para manejar la excepción
}
```

Donde:

- **try** es el bloque donde colocamos el código que puede generar una excepción y
- **catch** es el bloque que captura la excepción y la maneja, evitando que el programa se detenga.

Además, se podría añadir un tercer bloque **finally** (opcional). Así:

```
try {
   // Código
} catch (Exception e) {
   // Manejo excepción
} finally {
   // Código que siempre se ejecuta, haya o no excepción (como cerrar recursos)
}
```

El bloque finally se ejecuta siempre, y es útil para liberar recursos (cerrar archivos, conexiones...).

6.1. Tipos de excepciones

Java clasifica las excepciones en dos grandes grupos: **checked exceptions** (excepciones comprobadas) y **unchecked exceptions** (excepciones no comprobadas).

Checked Exceptions son las excepciones que el compilador obliga a tratar, es decir, que el programador debe manejar explícitamente usando un bloque try-catch.

Estas excepciones pueden ser previstas y por tanto deben ser anticipadas y controladas para evitar que el programa termine repentinamente.

Las excepciones comprobadas se detectan en tiempo de compilación y, al controlarlas, permiten al programa recuperarse de un error o informar claramente del fallo que ha habido.

Generalmente estas excepciones están relacionadas con recursos externos fuera del control del programa (como archivos, redes, bases de datos).

Ejemplos comunes de estas excepciones son:

- IOException: Error relacionado con entrada/salida (lectura/escritura de archivos, red, etc.)
- FileNotFoundException: Archivo no encontrado.
- SQLException: Error en operaciones con bases de datos.
- ClassNotFoundException: Clase no encontrada durante la carga dinámica.

Aunque no hayamos visto todavía la lectura y escritura de ficheros, te dejo un ejemplo de cómo se controlaría una excepción de entrada/salida (IOException):

```
import java.io.FileReader;
import java.io.IOException;

public class EjemploArchivo {
   public static void main(String[] args) {
      try {
         // Intentamos abrir un archivo para leer
         FileReader lector = new FileReader("datos.txt");

         int caracter;
         // Leemos y mostramos cada carácter hasta el final del archivo (-1)
         while ((caracter = lector.read()) != -1) {
            System.out.print((char) caracter);
         }

         lector.close(); // Cerramos el archivo
      } catch (IOException e) {
         // Capturamos errores como archivo no encontrado o error de lectura
         System.out.println("Error: el archivo no existe o no se puede leer.");
      }
   }
}
```

Unchecked Exceptions son excepciones que no es obligatorio capturar ni declarar. Estas excepciones suelen surgir por errores de lógica o programación que no se pueden anticipar fácilmente, como acceder a una posición inválida en un array o dividir por cero.

Se llaman también **runtime exceptions** porque se detectan en tiempo de ejecución (runtime) y el compilador no fuerza su manejo.

Como decía, estas excepciones no hay que capturarlas obligatoriamente, pero si no se capturan, el programa termina con un error en tiempo de ejecución.

Las excepciones no comprobadas surgen típicamente por errores del programador o condiciones no previstas y no suelen recuperarse en tiempo de ejecución porque normalmente son fallos de lógica que deben corregirse en el código.

Ejemplos comunes de este tipo de excepción son:

- ArithmeticException — Error aritmético, por ejemplo, división por cero.
- NullPointerException — Acceso a un objeto que es null.
- ArrayIndexOutOfBoundsException — Acceso fuera del límite del array.
- ClassCastException — Conversión inválida de tipos.
- IllegalArgumentException — Argumento ilegal pasado a un método.

Ejemplo de excepción aritmética (en este caso, al tratar de dividir entre cero):

```
public class EjemploExcepciones {
   public static void main(String[] args) {
      try {
         int a = 10;
         int b = 0;
         int resultado = a / b;  // Aquí ocurre ArithmeticException
         System.out.println("Resultado: " + resultado);
      } catch (ArithmeticException e) {
         System.out.println("Error: No se puede dividir por cero.");
      }
      System.out.println("Programa continúa...");
   }
}
```

Otro ejemplo de excepción no comprobada (NumberFormatException) al tratar de convertir un texto en número:

```
import java.util.Scanner;

public class EjemploConversion {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Introduce un número entero: ");
      String entrada = sc.nextLine(); // Leemos una línea como texto

      try {
         // Intentamos convertir la cadena a entero
         int numero = Integer.parseInt(entrada);
         System.out.println("El número es: " + numero);
      } catch (NumberFormatException e) {
         // Si el texto no es numérico, se lanza esta excepción
         System.out.println("Error: no has introducido un número válido.");
      }
   }
}
```

Otro ejemplo de excepción (ArrayIndexOutOfBoundsException) al tratar de acceder a una posición de un array que no existe:

```
public class EjemploArray {
   public static void main(String[] args) {
      int[] numeros = {10, 20, 30}; // Array con 3 elementos

      try {
         // Intentamos acceder a una posición inválida (la posición 5 no existe)
         System.out.println("Elemento en posición 5: " + numeros[5]);
      } catch (ArrayIndexOutOfBoundsException e) {
         // Captura el error si nos salimos del rango del array
         System.out.println("Error: estás accediendo a una posición fuera del array.");
      }
   }
}
```

La distinción entre tipos de excepciones se produce porque Java quiere forzar al programador a pensar y manejar adecuadamente los errores "previsibles" que pueden surgir por factores externos (como archivos o red) mediante las checked exceptions. Esto promueve un código más seguro y robusto.

Por otro lado, no fuerza a manejar errores derivados de fallos en la lógica de programación (unchecked exceptions), porque a menudo se espera que el programador corrija el código y no los maneje en tiempo de ejecución.

6.2. Varios bloques catch

Cuando dentro de un bloque try pueden lanzarse distintos tipos de excepciones, Java permite poner varios bloques catch para capturar cada tipo de excepción de forma específica. Así, usar múltiples bloques catch correctamente y entender la jerarquía de excepciones puede ayudarte a escribir código más robusto, legible y preciso.

Ejemplo:

```
try {
   // Código que puede lanzar varias excepciones
} catch (NumberFormatException e) {
   System.out.println("No has introducido un número válido.");
} catch (ArithmeticException e) {
   System.out.println("Error al realizar una operación matemática.");
} catch (Exception e) {
   System.out.println("Ha ocurrido un error inesperado.");
}
```

Los bloques catch deben ordenarse desde el más específico al más general para que el compilador pueda determinar de manera adecuada qué excepción debe manejar cada bloque. Esto se debe a que las excepciones en Java siguen una jerarquía de clases: algunas son subclases

de otras. Por ejemplo, NumberFormatException es una subclase de IllegalArgumentException, que a su vez es una subclase de RuntimeException, que hereda de Exception.

Cuando se escriben múltiples bloques catch, el compilador los evalúa en orden. Si primero se coloca un bloque que captura una excepción más general, este atrapará todas las excepciones que también sean subtipos, haciendo que los bloques posteriores más específicos sean inalcanzables. Esto generará un error de compilación, ya que el compilador detecta que hay código que nunca se ejecutará.

Por lo tanto, **siempre se deben ordenar los bloques catch colocando primero aquellos que capturan excepciones más concretas y específicas, y dejando al final los bloques que capturan clases más generales, como Exception**. Esto asegura que cada excepción tenga su oportunidad de ser atrapada en el bloque correspondiente y que el programa compile y se comporte correctamente.

Ejercicio 6.1. Desarrolla un programa que pida al usuario dos números de doble precisión (tipo double). Después los dividirá y mostrará el resultado por consola. Piensa en los posibles errores que pueden aparecer en el programa y contrólalos con varios bloques "catch".

En la tabla 6.1 puedes ver algunas de las excepciones más frecuentes en Java, indicando a qué paquete pertenecen, si requieren o no ser importadas de forma explícita, y si son comprobadas o no por el compilador. Esta información resulta especialmente útil para saber cuándo es necesario importar una clase de excepción y cómo debe tratarse adecuadamente en el código.

Excepción	Descripción breve	Nombre del paquete	¿Hay que importar?	Tipo de excepción
ArithmeticException	División por cero o error aritmético	java.lang	No	Unchecked
NullPointerException	Acceso a un objeto que es null	java.lang	No	Unchecked
ArrayIndexOutOfBoundsException	Índice fuera de rango en un array	java.lang	No	Unchecked
NumberFormatException	Conversión fallida de texto a número	java.lang	No	Unchecked
IllegalArgumentException	Argumento no válido pasado a un método	java.lang	No	Unchecked
IllegalStateException	Operación realizada en estado inválido	java.lang	No	Unchecked
InputMismatchException	Tipo de dato introducido no	java.util	Sí	Unchecked

Excepción	Descripción breve	Nombre del paquete	¿Hay que importar?	Tipo de excepción
	coincide con el esperado			
IOException	Error general de entrada/salida	java.io	Sí	Checked
FileNotFoundException	Archivo no encontrado al intentar leerlo	java.io	Sí	Checked
ClassNotFoundException	Clase solicitada no existe	java.lang	No	Checked
IndexOutOfBoundsException	Índice inválido en estructuras de datos	java.lang	No	Unchecked
StringIndexOutOfBoundsException	Índice inválido en una cadena de texto	java.lang	No	Unchecked
InterruptedException	Interrupción durante la ejecución de un hilo	java.lang	No	Checked
SQLException	Error relacionado con acceso a base de datos	java.sql	Sí	Checked
NoSuchElementException	Cuando no hay más elementos disponibles (como con Scanner.next())	java.util	Sí	Unchecked

Tabla 6.1. Excepciones frecuentes y sus paquetes de importación. Elaboración propia.

NOTA: Como se dijo en otro tema anterior, el paquete java.lang se importa por defecto en nuestro proyecto, por lo que no es necesario volver a hacerlo.

Ejercicio 6.2. Pide al usuario un número entero y muéstralo por consola. Si introduce algo que no sea un número, muestra un mensaje de error. Usa un solo bloque catch.

Ejercicio 6.3. Pide al usuario un número entero. Después, muestra la tabla de multiplicar de ese número del 1 al 10. Controla que no introduzca letras ni otros símbolos. Usa un solo bloque catch.

Ejercicio 6.4. Crea un array de 4 elementos. Pide al usuario un índice (posición) y muestra el valor del array en esa posición. Controla posibles errores. Usa varios bloques catch para diferenciar InputMismatchException e IndexOutOfBoundsException.

Ejercicio 6.5. Pide al usuario dos números enteros. Divide el primero entre el segundo. Controla la división por cero y errores de entrada. Usa varios bloques catch y un finally que muestre un mensaje final de agradecimiento.

Ejercicio 6.6. Crea un programa que siempre intente ejecutar una operación de suma. Usa el bloque finally para mostrar por consola "Operación completada", pase lo que pase.

Ejercicio 6.7. Declara una variable de tipo String como nula. Intenta usar length() sobre ella y captura el error adecuadamente. Usa un solo bloque catch.

NOTA: El objeto nulo significa que no existe la cadena, por lo tanto, no se puede mostrar nada sobre ella.

Ejercicio 6.8. Pide al usuario que introduzca 5 números separados por espacios. Luego, muestra la suma de esos números. Controla errores de entrada. Usa split, conversión con Integer.parseInt() y control de errores con catch múltiple.

Las excepciones son objetos que derivan de la clase Throwable (o de alguna subclase como Exception).

Cuando haces esto:

```
catch (InputMismatchException e) {
   ...
}
```

e es una **variable de referencia** a un objeto de tipo InputMismatchException que Java lanza automáticamente cuando ocurre un error de entrada, es decir, **son variables** como cualquier otro objeto. Solo que representan errores.

Fíjate que la variable e no se usa, pero darle nombre tiene varias ventajas:

- Puedes usarlo para **debugging** (depurar el código, este concepto se verá en un tema más adelante), **logging** (registrar información sobre lo que está ocurriendo), o para **mostrar mensajes más útiles al usuario**.
- Permite acceder a **información más detallada del error**: el mensaje, la traza, etc. Así:

```
catch (InputMismatchException e) {
   System.out.println("Error: " + e.getMessage());
}
```

e.getMessage() devuelve un texto descriptivo sobre lo que pasó.

También puedes imprimir toda la traza con:

```
e.printStackTrace();
```

Aunque eso suele usarse más en entornos de desarrollo, no en aplicaciones para el usuario final.

NOTA: La **traza de la pila** (*stack trace*) es el **rastro de llamadas** que se produce cuando ocurre una excepción. Esto te permite ver **en qué clase y línea exacta del código** ocurrió el error. Es muy útil para **localizar y corregir errores** en el código.

Por ejemplo:

```
public class MiPrograma {
   public static void main(String[] args) {
      try {
         int a = 5 / 0; // Línea 5: aquí ocurre la excepción
      } catch (ArithmeticException e) {
         // Muestra el mensaje de error
         System.out.println("Se produjo un error: " + e.getMessage());

         // Muestra la traza completa del error (incluye clase y línea)
         e.printStackTrace();
      }
   }
}
```

La salida del programa sería:

```
java.lang.ArithmeticException: / by zero
          at MiPrograma.main(MiPrograma.java:5)
```

Esto indica:

- Qué tipo de excepción ocurrió (ArithmeticException)
- Qué lo causó (/ by zero)
- Dónde ocurrió exactamente (en la línea 5 de la clase MiPrograma)

Ejercicio 6.9. Desarrolla un programa que pida al usuario dos números enteros. Después los dividirá y mostrará el resultado. Si el usuario introduce un valor no numérico o intenta dividir entre cero, el programa mostrará un mensaje de error y la traza de la excepción.

Ejercicio 6.10. Crea un programa que declare un array de 5 enteros. Luego pide al usuario una posición para mostrar el valor almacenado en esa posición. Si el índice está fuera de rango, el programa debe mostrar el mensaje del error y la traza correspondiente.

6.3. Volviendo al programa

Una vez hemos conseguido detectar los errores, **es conveniente volver al programa después,** para hacer que los programas sean más robustos y amigables con el usuario. Esto lo conseguimos **envolviendo el código dentro de un bucle (while, do-while, etc.)** que permite volver a intentarlo si ocurre un error. Al combinar esto con try-catch, puedes capturar la excepción, mostrar un mensaje al usuario, y repetir la operación. Así:

```
import java.util.Scanner;

public class EjemploReintento {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int numero = 0;
      boolean valido = false;

      while (!valido) {
         try {
            System.out.print("Introduce un número entero: ");
            numero = sc.nextLine();
            valido = true; // si no lanza excepción, es válido
         } catch (NumberFormatException e) {
            System.out.println("Error: Debes introducir un número válido.");
         }
      }

      System.out.println("Número introducido: " + numero);
      sc.close();
   }
}
```

NOTA: Cuando trabajemos con la excepción InputMismatchException, es necesario limpiar el buffer con sc.nextLine(); para no entrar en un bucle infinito. Esto ocurre porque **no se consume la entrada del buffer**, por lo que, en la siguiente iteración del bucle, vuelve a intentar leer ese mismo valor erróneo.

```
catch (InputMismatchException e) {
            System.out.println("Entrada no válida. Intenta de nuevo.");
            sc.nextLine(); //Aquí limpiamos el buffer
         }
```

Ejercicio 6.11. Desarrolla un programa que pida dos números enteros: el dividendo y el divisor. El programa debe asegurarse de que el divisor no sea cero y que ambos valores sean números válidos. Si hay algún error, volverá a pedir ambos valores.

Ejercicio 6.12. Desarrolla un programa que tenga un array de nombres. El usuario debe introducir un índice, y el programa mostrará el nombre correspondiente. Si el índice es incorrecto (fuera de rango o no es un número), se mostrará un error y se volverá a pedir el índice.

6.4. La instrucción throw

La instrucción **throw** se usa para **lanzar una excepción** desde un punto específico del programa. Cuando se ejecuta, **interrumpe el flujo normal del programa** y busca un bloque catch que pueda manejar ese tipo de excepción. Mira este ejemplo:

```
If (numero<0)
```

```
throw new IllegalArgumentException("Número negativo.");
```

IllegalArgumentException es una clase de excepción no comprobada (unchecked exception) que indica que **se ha pasado un dato no válido**.

En este caso estás **creando una nueva instancia de esa excepción**, con un **mensaje personalizado** que se podrá recuperar más tarde usando e.getMessage().

Se usa cuando tú como programador detectas una situación que **no es aceptable** para tu programa. En este caso **Java no la detectaría automáticamente,** porque el sistema no tiene problema en leer números negativos. Es decir, si tú no quieres que se introduzcan números negativos, por ejemplo, lanza tú mismo la excepción.

Una vez se lanza una excepción, **no se ejecuta ningún código que venga después** en ese mismo bloque. El control pasa inmediatamente al primer bloque catch compatible con el tipo de excepción lanzada.

Mira este ejemplo completo:

```
import java.util.Scanner;

public class VerificarTexto {

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce tu nombre: ");
         String nombre = sc.nextLine();

         // Si la cadena está vacía lanzamos una excepción
         if (nombre.isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío.");
         }

         System.out.println("Hola, " + nombre + "!");

      } catch (IllegalArgumentException e) {
         System.out.println("Error: " + e.getMessage());
      } finally {
         System.out.println("Programa finalizado.");
      }

      sc.close();
   }
}
```

NOTA: isEmpty() devuelve un booleano diciendo true o false, es decir, si la cadena está vacía o no.

Ejercicio 6.13. Desarrolla un programa que pida al usuario su edad. Si la edad no es un número válido o es menor que 0 o mayor que 110, el programa debe lanzar una excepción con throw new IllegalArgumentException("Edad no válida").

Ejercicio 6.14. Desarrolla un programa que pida al usuario una contraseña. La contraseña debe cumplir con los siguientes requisitos:

- Tener al menos 8 caracteres.
- Contener al menos un dígito.

Si no cumple con los requisitos, **lanza una excepción** con throw new SecurityException("Contraseña no segura").

NOTA: isDigit() comprueba si hay algún dígito en una cadena de caracteres. Así:

```
Character.isDigit(cadena.charAt(posicion))
```

6.5. Ejercicios de repaso

Ejercicio 6.15. Pide una palabra al usuario y luego una posición. Muestra el carácter en esa posición. Captura excepciones si el número no es válido o está fuera de los límites (StringIndexOutOfBoundsException).

Ejercicio 6.16. Pide al usuario dos números enteros y muestra el resultado de su división. Si ocurre cualquier tipo de error (formato, división por cero, etc.), usa **un bloque catch genérico** para capturar cualquier excepción. Luego muestra e.getClass().getSimpleName() y e.getMessage().

NOTA: Si al usar un catch genérico (Exception e) se quiere saber qué tipo de excepción ha surgido, puedes usar e.getClass().getSimpleName(). Así:

```
try {
   int resultado = 10 / 0;
} catch (Exception e) {
   System.out.println("Excepción capturada: " + e.getClass().getSimpleName());
   System.out.println("Mensaje: " + e.getMessage());
}
```

Y la salida sería:

```
Excepción capturada: ArithmeticException
Mensaje: / by zero
```

Ejercicio 6.17. Pide al usuario una cadena y luego un índice. Muestra la letra que se encuentra en esa posición de la cadena. Captura cualquier excepción que pueda producirse y muestra el tipo de excepción y su mensaje.

Ejercicio 6.18. Pide una fecha al usuario en formato dd/mm/aaaa. Si el usuario no introduce el formato correcto (por ejemplo, separadores incorrectos o campos vacíos), lanza una excepción con un mensaje personalizado que indique el error. Captura cualquier posible error.

Ejercicio 6.19. Desarrolla un programa que pida al usuario una contraseña dos veces. Si las contraseñas no coinciden, lanza una excepción con throw new IllegalArgumentException("Las contraseñas no coinciden"). Usa un bloque finally para mostrar un mensaje de cierre.

Ejercicio 6.20. Solicita al usuario un número entero positivo. Si el número es negativo, lanza una excepción personalizada con un mensaje adecuado. Si el número es correcto, calcula su raíz cuadrada y muéstrala. Controla cualquier error con bloques try-catch.

Ejercicio 6.21. Pide al usuario un número decimal. Si la entrada no es un número válido, captura la excepción y vuelve a pedir el número hasta que el usuario lo introduzca correctamente. Utiliza un bucle while para repetir hasta tener una entrada válida.

TEMA 7. FUNCIONES

Una función es un **conjunto de instrucciones agrupadas que realizan una tarea específica**. Estas funciones permiten organizar el código de manera más clara, facilitando su lectura, mantenimiento y reutilización.

Toda función debe definirse dentro de una clase. La más conocida es la función principal llamada ***main***, que actúa como punto de inicio de cualquier programa, aunque podemos definir muchas otras funciones para dividir el trabajo en partes más pequeñas y comprensibles.

Una función se compone de varios elementos clave: un nombre que la identifica, una lista de parámetros entre paréntesis (aunque puede estar vacía), un tipo de valor que devuelve (o la palabra *void* si no devuelve nada), y un bloque de instrucciones encerrado entre llaves.

Aprender a crear y utilizar funciones es un paso fundamental en la programación, ya que nos permite construir programas más organizados, escalables y fáciles de depurar.

7.1. Organización modular de los programas: División en partes

La idea de dividir los problemas en partes más pequeñas y manejables, múltiples beneficios:

- Cada una de estas partes, que se desarrollan de forma independiente, será más sencilla de programar, ya que se centra en una tarea específica y limitada.
- El cuerpo principal del programa resultará más claro y fácil de entender, ya que no tendrá que incluir todos los detalles de cómo se realiza cada acción.
- Se reduce notablemente la repetición de código.
- El trabajo se puede distribuir entre varias personas, asignando a cada una parte concreta del programa. Luego, se unirán todas esas partes para formar el sistema completo.

7.2. Fundamentos sobre funciones

Para crear una función básica, simplemente se le da un nombre, se colocan paréntesis al lado del nombre (aunque no reciba ningún dato), y se utilizan unas palabras clave como "public static void" al principio. Estas palabras tienen significados específicos que se explicarán más adelante. Dentro de un bloque de llaves se escriben todas las instrucciones que debe ejecutar esa función. Esta es su sintaxis:

```
[modificadores] tipoDeRetorno nombreFuncion(parámetros) {
   // instrucciones o código que ejecuta la función
}
```

Donde:

- **modificadores** son palabras opcionales como public, private, static, que indican el nivel de acceso y otras propiedades de la función (se verá más adelante en profundidad).

- **tipoDeRetorno** indica qué tipo de dato devuelve la función (por ejemplo, int, String, boolean). Si no devuelve ningún valor, se usa la palabra clave **void**.
- **nombreFuncion** es el identificador que se le da a la función.
- **parámetros** es una lista de variables que recibe la función para trabajar, encerradas entre paréntesis. Si no necesita parámetros, se dejan los paréntesis vacíos (este concepto se verá en profundidad más adelante).
- **cuerpo de la función** es el bloque de instrucciones que se ejecutan cuando se llama a la función, encerrado entre llaves {}.

Ejemplo:

```
public static void saludar() {
   System.out.println("Hola, bienvenido al programa.");
}
```

Esta función llamada saludar no recibe parámetros y no devuelve ningún valor (por eso su tipo de retorno es void). Simplemente muestra un mensaje por pantalla cuando se llama.

Para llamar a una función, por ahora, solo vamos a escribir su nombre. Así:

```
package SeptimoTema;

public class EjemploSaludar {

   // Definición de la función saludar
   public static void saludar() {
      System.out.println("Hola, bienvenido al programa.");
   }

   // Método principal que llama a la función saludar
   public static void main(String[] args) {
      saludar();
   }
}
```

NOTA: El orden en que se escriben las funciones dentro de una clase **no influye** en el comportamiento del programa, siempre que estén correctamente definidas antes de ser ejecutadas. Esto se debe a que el compilador de Java analiza todo el contenido de la clase antes de ejecutarla, por lo que reconoce todas las funciones sin importar su ubicación.

Ejercicio 7.1. Crea una función llamada "tablaMultiplicar", que muestre la tabla de multiplicar del 5.

Ejercicio 7.2. Crea una función llamada "tablaMultiplicarUsuario", que muestre la tabla de multiplicar del número que indique el usuario.

Ejercicio 7.3. Crea una función llamada "DibujarCuadrado3x3", que dibuje un cuadrado formato por 3 filas con 3 asteriscos cada una.

7.3. Parámetros en una función

Muchas veces necesitamos que una función trabaje con datos concretos que le enviamos desde otra parte del programa. A esos datos los llamamos **parámetros**, y se escriben dentro del paréntesis que aparece justo después del nombre de la función, separados por comas si hay más de uno.

Cada parámetro debe tener un tipo de dato (como int, double, String, etc.) seguido de un nombre identificador. Gracias a los parámetros, las funciones se vuelven más flexibles y reutilizables, ya que no dependen de valores fijos escritos dentro del cuerpo de la función.

Por ejemplo, si queremos que una función reciba dos números y muestre su suma, podemos hacerlo así:

```
package SeptimoTema;

public class SumaConParametros {

   public static void main(String[] args) {
      int num1 = 4;
      int num2 = 8;
      sumar(num1, num2); // Llamamos a la función pasando los valores como argumentos
   }

   public static void sumar(int n1, int n2) {
      int resultado = n1 + n2;
      System.out.println("La suma es: " + resultado);
   }
}
```

En este ejemplo, los valores num1 y num2 se pasan como **argumentos** al llamar a la función sumar, que los recibe como **parámetros** llamados n1 y n2.

Algunos puntos importantes a tener en cuenta:

- Como las funciones realizan acciones, sus nombres suelen ser **verbos**, como sumar, mostrar, calcular, etc.
- En Java es habitual escribir los nombres de las funciones con **minúscula inicial**, y usar mayúsculas para separar palabras si el nombre es compuesto, por ejemplo: mostrarResultado.

Usar parámetros permite que nuestras funciones sean más generales y puedan adaptarse a distintos valores sin necesidad de duplicar código.

Ejercicio 7.4. Crea una función "dibujarRectanguloHueco" que dibuje en pantalla un rectángulo hueco del ancho y alto que se indiquen como parámetros, formado por una letra que también se indique como parámetro. Completa el programa con un "*main*" que pida esos datos al usuario y dibuje el rectángulo.

7.4. Valor devuelto por una función y el uso de return

En Java, cuando definimos una función, podemos especificar si esta va a devolver un valor o no. Cuando una función **no debe devolver ningún valor**, indicamos que su tipo de dato es **void**. Este es el caso de funciones como main o nuestra función saludar del ejemplo anterior, que simplemente ejecutan acciones sin entregar un resultado para que se use en otra parte.

Sin embargo, muchas funciones sí devuelven valores, especialmente cuando realizan cálculos o procesos que producen un resultado, similar a las funciones matemáticas que conocemos (como la raíz cuadrada, que devuelve un número). Para esas funciones, en lugar de void, indicamos el tipo de dato que van a devolver, por ejemplo, int, double o String.

El mecanismo que permite devolver un valor desde una función se llama **return**. La palabra reservada return se usa dentro de la función para **enviar un resultado al lugar donde fue llamada**. El proceso interno es el siguiente: cuando el return se ejecuta, la función **termina su ejecución** y el programa "vuelve" al punto donde fue llamada. Desde el punto de vista de main, ahora la llamada a la función **se comporta como si hubiera sido reemplazado directamente por el valor devuelto**.

Por ejemplo, imagina una función que calcula la raíz cuadrada de un número entero y devuelve un número decimal:

```
package SeptimoTema;

public class DevuelveRaizCuadrada {

   public static void main(String[] args) {
      int n = 16;
      System.out.println("La raíz de 16 es: " + calcularRaiz(n));
   }

   public static double calcularRaiz(int n) {
      return Math.sqrt(n);  // Devuelve el resultado de Math.sqrt
   }
}
```

NOTA: La clase **Math** forma parte del paquete java.lang, por lo que no necesitas importarla para poder utilizarla. Proporciona una amplia variedad de funciones que te permiten realizar operaciones matemáticas complejas de forma sencilla, como potencias, raíces cuadradas, logaritmos, funciones trigonométricas, redondeos, generación de números aleatorios, entre otras.

Fíjate que, la llamada a la función calcularRaiz() se hace desde dentro del println() para imprimir directamente el resultado devuelto: System.out.println("La raíz de 16 es: " + **calcularRaiz**(n));

Podemos devolver cualquier tipo de dato, no solo números. Por ejemplo, esta función devuelve el mayor de dos números enteros:

```
package SeptimoTema;
```

```
public class CalcularMayor {

   public static void main(String[] args) {
      int n1 = 16;
      int n2 = 4;
      System.out.println("El mayor de los números es: " + calcularMayor(n1, n2));
   }

   public static int calcularMayor(int n1, int n2) {
      if (n1 < n2) {
         return n2;  // Si n2 es mayor, devolver n2
      } else {
         return n1;  // Si no, devolver n1
      }
   }
}
```

En este ejemplo, la función tiene más de un return porque puede devolver dos valores diferentes dependiendo de una condición. Tan pronto se ejecuta un return, la función termina y no se ejecuta código después de ese punto.

Esto es importante porque un mal uso puede causar errores o advertencias en el compilador, por ejemplo, cuando se escribe código después de un return que nunca se ejecutará (código inalcanzable):

```
public static String inalcanzable() {
   return "Aquí sí llegamos";
   String ejemplo = "Aquí no llegamos";  // Esto es código inalcanzable
   return ejemplo;                       // También inalcanzable
}
```

El compilador detecta ese código y lanza una advertencia porque nunca será ejecutado.

Ejercicio 7.5. Crea una función "*calcularCubo*" que calcule el cubo de un número real (float) que se indique como parámetro. El resultado deberá ser otro número real. Prueba esta función para calcular el cubo de 3.2 y el de 5.

Ejercicio 7.6. Crea una función "*calcularMenor*" que calcule el menor de dos números enteros que recibirá como parámetros. El resultado será otro número entero.

Ejercicio 7.7. Crea una función llamada "*obtenerSigno*", que reciba un número real, y devuelva un número entero con el valor: -1 si el número es negativo, 1 si es positivo o 0 si es cero.

Ejercicio 7.8. Crea una función "*extraerInicial*", que devuelva la primera letra de una cadena de texto. Prueba esta función para calcular la primera letra de la frase "Hola".

Ejercicio 7.9. Crea una función "*ultimaLetra*", que devuelva la última letra de una cadena de texto. Prueba esta función para calcular la última letra de la frase "Hola".

Ejercicio 7.10. Crea una función "*mostrarPerimSuperfCuadrado*" que reciba un número entero y calcule y muestre en pantalla el valor del perímetro y de la superficie de un cuadrado que tenga como lado el número que se ha indicado como parámetro.

7.5. La palabra reservada static

Hasta ahora, hemos escrito nuestras funciones (como main, saludar, sumar, etc.) de esta manera:

```
public static void saludar() {
   System.out.println("Hola");
}
```

Observa que hemos puesto la palabra static al declarar estas funciones. Cuando una función o una variable se marca como static, **quiere decir que se puede usar directamente desde el programa principal**, sin ningún paso intermedio.

Imagina que cada función es una herramienta. Las funciones static están **sobre la mesa**, listas para usarse cuando las necesites. Es decir, si una función o una variable **no es static**, "no está sobre la mesa todavía", y no puedes acceder a ella directamente desde main.

Además, la función principal (main) es static y, en Java, **una función static solo puede llamar a otras funciones o usar variables que también sean static**.

NOTA: Profundizaremos en este concepto en el siguiente tema.

7.6. Variables locales y de clase

Hasta este momento hemos declarado todas las variables dentro del método main. Ahora que nuestros programas comienzan a dividirse en varias funciones o bloques, el lugar donde declaramos una variable determina cómo y desde dónde puede usarse.

Si una variable se declara dentro de un bloque, como puede ser una función o un bucle for, **solo podrá ser utilizada dentro de ese mismo bloque**. A estas variables se las conoce como **variables locales**, ya que su ámbito está limitado al lugar donde fueron declaradas. En cambio, si una variable se declara fuera de todos los bloques, al comienzo del programa y dentro de la clase, se convierte en una **variable de clase ("globales")**. Esto significa que **puede ser utilizada desde cualquier parte del programa**. En Java, para que una variable de clase funcione correctamente, necesitamos declararla con la palabra clave **static**. Esto se debe a que, si desde funciones estáticas (static) como main quieres acceder a una variable que está fuera de él, **esa variable debe llevar también la palabra static**.

Aunque es posible usar variables de clase, **lo más recomendable es que la mayoría de nuestras variables sean locales**. Esto ayuda a que cada parte del programa funcione de forma independiente, con sus propios datos, lo que reduce la probabilidad de errores causados por interferencias entre distintas funciones. Es decir, en lugar de usar variables de clase para compartir información entre funciones, lo correcto es hacerlo a través de parámetros y valores de retorno.

Para ver en práctica cómo funcionan las variables locales, podemos observar este ejemplo que calcula la potencia de un número entero.

```
import java.util.Scanner;

public class CalcularPotencia {
   public static void main(String[] args) {
      Scanner in = new Scanner(System.in);
      System.out.print("Introduce la base: ");
      int base = in.nextInt();
      System.out.print("Introduce el exponente: ");
      int exp = in.nextInt();
      System.out.println("El resultado es: " + calcularPotencia(base, exp));
   }

   public static int calcularPotencia(int base, int exp) {
      int temporal = 1;
      for (int i = 1; i <= exp; i++) {
         temporal *= base;
      }
      return temporal;
   }
}
```

En este programa, las variables temporal e i son locales a la función calcularPotencia(), por lo que el método main no puede usarlas ni acceder a ellas. De la misma forma, las variables base y exp que se declaran en main son locales a ese método y no son visibles dentro de calcularPotencia().

Ejemplo de variable de clase:

```
public class EjemploVariableDeClase {
   // Esta es una variable de clase (static, fuera de los métodos)
   static int contador = 0;

   public static void main(String[] args) {
      saludar();
      saludar();
      saludar();
      System.out.println("Se llamó a la función saludar " + contador + " veces.");
   }

   public static void saludar() {
      System.out.println("Hola, ¿cómo estás?");
      // Como la variable es de clase, podemos modificarla desde aquí
      contador++;
   }
}
```

Observa que **contador** es una **variable de clase ("global")** porque está declarada **fuera de todos los métodos** y es **static**, por lo tanto, puede usarse en cualquier método static dentro de la

misma clase (como main o saludar). Aemás, retiene su valor entre llamadas, lo que no ocurre con variables locales.

Este ejemplo cuenta cuántas veces se llamó la función saludar, sin necesidad de usar parámetros ni valores de retorno.

RECUERDA: En Java moderno y sobre todo en POO, **NO se recomienda abusar de las variables de clase**. Es preferible usar parámetros y valores de retorno para intercambiar información entre métodos. Sin embargo, en etapas iniciales de aprendizaje o para ciertos ejercicios, este tipo de variables ayuda a ilustrar cómo compartir datos entre funciones.

Ejercicio 7.11. Crea una función "saberSiEsPrimo", que reciba un número y devuelva el valor booleano "true" si es un número primo o "false" en caso contrario.

Ejercicio 7.12. Crea una función "contarLetra", que reciba una cadena y una letra, y devuelva la cantidad de veces que dicha letra aparece en la cadena. Por ejemplo, si la cadena es "Barcelona" y la letra es 'a', debería devolver 2 (porque la "a" aparece 2 veces).

Ejercicio 7.13. Crea una función "sumarCifras" que reciba un numero cualquiera y que devuelva como resultado la suma de sus dígitos. Por ejemplo, si el número fuera 123 la suma sería 6.

Ejercicio 7.14. Crea una función "dibujarTriangulo" que reciba una letra y un número, y escriba un "triángulo" formado por esa letra, que tenga como anchura inicial la que se ha indicado. Por ejemplo, si la letra es * y la anchura es 4, debería escribir:

****

***

**

*

7.7. Modificando parámetros

Podemos intentar modificar el valor de un dato recibido como parámetro dentro de una función, pero **el cambio no afectará a la variable original**. Vamos a ver por qué ocurre esto con un ejemplo:

```
public class VariablesLocales {
	public static void duplicar(int n) {
		n *= 2;
		System.out.println("Ahora, en la función \"duplicar\" el valor de n es: " + n);
	}

	public static void main(String[] args) {
		int n = 5;
		System.out.println("El valor de n es: " + n);
		duplicar(n);
		System.out.println("Y ahora, otra vez en la función \"main\" el valor de n es:
" + n);
```

```
	}
}
```

Este programa muestra por pantalla:

```
El valor de n es: 5
Ahora, en la función "duplicar" el valor de n es: 10
Y ahora, otra vez en la función "main" el valor de n es: 5
```

Aunque dentro de la función duplicar el valor de n se ha multiplicado por 2, **ese cambio no afecta a la variable original del main**. Esto se debe a que los parámetros en Java se pasan **por valor**, es decir, **se trabaja con una copia del valor original**. Modificar el parámetro dentro de la función solo cambia esa copia, no el valor original que estaba en main.

Como ya sabemos, una función puede devolver **un único valor** usando return. Pero a veces necesitaremos que una función nos devuelva **más de un dato**. Como en Java los tipos simples (como int, float, etc.) se pasan siempre por valor, no podemos modificar sus valores directamente desde la función.

Una alternativa es **usar estructuras que permitan contener varios datos**, como **arrays**. Estos sí pueden ser modificados dentro de una función porque, aunque también se pasan por valor, **lo que se copia es la referencia al mismo objeto en memoria**.

Mira este ejemplo:

```
public class DevolverArray {
	public static int[] sumar(int n1, int n2, int n3, int n4, int[] resultados) {
		int suma1 = n1 + n2;
		int suma2 = n3 + n4;
		resultados[0] = suma1;
		resultados[1] = suma2;
		return resultados;
	}

	public static void main(String[] args) {
		int n1 = 5, n2 = 7, n3 = 2, n4 = 12;
		int[] resultados = new int[2];
		sumar(n1, n2, n3, n4, resultados);
		System.out.println("El resultado de sumar " + n1 + " y " + n2 + " es: " +
resultados[0]);
		System.out.println("El resultado de sumar " + n3 + " y " + n4 + " es: " +
resultados[1]);
	}
}
```

Ejercicio 7.15. Crea una función "modificar", que modifique el valor de dos números enteros que se le indiquen como parámetro.

Ejercicio 7.16. Crea una función "extraerIniciales", que reciba una cadena como "Antonio Barba" y devuelva las letras A y B (primera letra, y letra situada tras el primer espacio).

Ejercicio 7.17. Crea una función que devuelva en un array la tabla de multiplicar del número que indique el usuario.

7.8. Recursividad

La **recursividad** es una técnica para resolver un problema dividiéndolo en casos más simples o más pequeños del mismo problema. Una función recursiva es una función que **se llama a sí misma**, trabajando paso a paso hasta llegar a un caso básico o trivial que pueda resolverse directamente.

Un ejemplo clásico y muy usado en matemáticas y programación es el **factorial de un número**.

Por si no lo recuerdas, el factorial de un número entero positivo **n** (se escribe como *n!*) es el producto de todos los números enteros desde *n* hasta 1, es decir:

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 3 \cdot 2 \cdot 1$$

Por ejemplo, el factorial de 4 es: $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$

Podemos observar que el factorial de un número se puede calcular usando el factorial de un número menor:

$$n! = n \cdot (n-1)!$$

Esta fórmula es justamente la **definición recursiva** del factorial: para calcular *n!*, multiplicamos *n* por el factorial de *(n-1)*.

El código sería:

```
import java.util.Scanner;

public class EjercicioFactorial {
	public static void main(String args[]) {
		Scanner sc = new Scanner(System.in);
	System.out.print("Introduce un número entero: ");
	int num = sc.nextInt();
	System.out.print("Su factorial es: " + factorial(num));
	}

	public static long factorial(int num) {
		if (num == 1) // Caso base: condición que detiene la recursión
			return 1;
		return num * factorial(num - 1); // Paso recursivo: la función se llama a sí
misma
	}
}
```

Caso base: Toda función recursiva debe tener un caso base o trivial, que sea el punto donde se **detiene la llamada recursiva**. Esto es imprescindible para evitar que el programa entre en un

ciclo infinito y termine bloqueando el ordenador o la ejecución. En el ejemplo, el caso base es cuando num == 1, donde simplemente devolvemos 1 sin llamar más a la función.

Otra cosa a tener en cuenta son las **limitaciones de los tipos de datos:** Los factoriales crecen muy rápido. Por ejemplo, el factorial de 16 es 20.041.189.184, un número que ya no cabe en un entero normal (int). Por eso, en el código usamos long, que soporta valores más grandes, pero incluso así, a partir de 20 o 21 puede haber problemas por desbordamiento. Si se necesitan factoriales de números aún más grandes, se usan otros tipos o librerías especiales.

Vamos a ver paso a paso qué hace el ejercicio para que lo entiendas mejor:

Primera llamada: factorial(4)

- Como 4 != 1, no entra en el caso base.
- Calcula 4 * factorial(3).
- Pero para eso, primero debe calcular factorial(3).

Segunda llamada (desde la primera): factorial(3)

- 3 != 1, sigue recursión.
- Calcula 3 * factorial(2).
- Debe calcular factorial(2).

Tercera llamada (desde la segunda): factorial(2)

- 2 != 1, sigue recursión.
- Calcula 2 * factorial(1).
- Debe calcular factorial(1).

Cuarta llamada (desde la tercera): factorial(1)

- Aquí sí entra en el caso base: num == 1.
- Devuelve 1 sin llamar más funciones.

Ahora que hemos llegado al **caso base**, empezamos a **volver hacia atrás**:

Volviendo a la tercera llamada. factorial(2) ahora puede calcularse:

- 2 * factorial(1) = 2 * 1 =2
- Devuelve 2.

Volviendo a la segunda llamada. factorial(3) se calcula:

- 3 * factorial(2) = 3 * 2 = 6
- Devuelve 6.

Volviendo a la primera llamada. factorial(4) se calcula:

- 4 * factorial(3) = 4 * 6 = 24
- Devuelve 24.

Finalmente, el programa imprime:

```
Su factorial es: 24
```

Es decir:

- La función se llama a sí misma reduciendo num en 1 en cada llamada.
- Cada llamada se "pausa" esperando el resultado de la siguiente llamada recursiva.
- Cuando llega al caso base (num == 1), devuelve 1 y empieza a "desenrollar" las llamadas, multiplicando los valores conforme regresa.
- Así, aunque parezca que la función se repite muchas veces, en realidad va resolviendo el problema paso a paso desde lo más simple hasta lo complejo.

Muchos algoritmos importantes, como el cálculo de números de Fibonacci, búsqueda en estructuras de datos tipo árbol o recorrido de laberintos, se basan en recursividad.

Ejercicio 7.18. Crea una función que calcule el valor de elevar un número entero a otro número entero (por ejemplo, 5 elevado a 3 = 53= 5 ·5 ·5 = 125). Esta función se debe crear de forma recursiva. Piensa cuál será el caso base (qué potencia se puede calcular de forma trivial) y cómo pasar del caso "n-1" al caso "n" (por ejemplo, si sabes el valor de 54, cómo hallarías el de 55 a partir de él).

Ejercicio 7.18. Crea de forma recursiva una función que diga si una cadena de caracteres es palíndroma (se lee igual de izquierda a derecha que viceversa). Por ejemplo, "reconocer" o “luz azul”.

Ejercicio 7.20. Crear un programa que encuentre el máximo común divisor de dos números usando el algoritmo de Euclides: Dados dos números enteros positivos m y n, tal que m > n, para encontrar su máximo común divisor, es decir, el mayor entero positivo que divide a ambos:

a. Dividir m por n para obtener el resto r ($0 \leq r < n$);
b. Si r = 0, el MCD es n;
c. Si no, el máximo común divisor es MCD (n, r).

7.9. Ejercicios de repaso

Ejercicio 7.21. Crea una clase con una función que declare 2 variables de tipo entero y realice una división. Desde el método main, solo se llamará a esta función.

Ejercicio 7.22. Crea una clase llamada OperacionesSimples. Implementa las siguientes funciones:

- sumar(): Declara dos variables enteras llamadas numero1 y numero2. Calcula la suma y muestra el resultado por la consola.
- restar(): Declara dos variables enteras llamadas numero1 y numero2. Calcula la resta y muestra el resultado por la consola.

Ejercicio 7.23. Crea una clase que contenga una función que salude por consola. Solicitarás el nombre del usuario desde la función main y lo pasarás como parámetro a esta función.

Ejercicio 7.24. Crea una clase llamada FigurasGeometricas. Implementa las siguientes funciones:

- calcularAreaCirculo(double radio): Calcula el área de un círculo (π x radio^2) y muestra el resultado en la consola.

- calcularCircunferencia(double radio): Calcula la circunferencia de un círculo ($2 \times \pi \times$ radio) y muestra el resultado en la consola.
- calcularAreaRectangulo(double base, double altura): Calcula el área de un rectángulo y muestra el resultado en la consola.
- calcularPerimetroRectangulo(double base, double altura): Calcula el perímetro de un rectángulo y muestra el resultado en la consola.

Dentro del método main, pide datos al usuario y envíalos como parámetros a las funciones.

NOTA: Puedes obtener el número pi dentro de la clase Math, así: **Math.PI**

Ejercicio 7.25. Crea una clase llamada Calculadora. Implementa las siguientes funciones:

- sumar(int a, int b): Calcula la suma de dos números enteros y devuelve el resultado.
- restar(int a, int b): Calcula la resta de dos números enteros y devuelve el resultado.
- multiplicar(int a, int b): Calcula el producto de dos números enteros y devuelve el resultado.
- dividir(double a, double b): Calcula el cociente de dos números. Si el divisor es 0, muestra un mensaje de error.

En el método main, solicita al usuario dos números y elige la operación que desea realizar llamando a las funciones. Después, al recibir el resultado de la operación, lo mostrará por la consola.

Ejercicio 7.26. Crea una clase llamada ConversorUnidades. Implementa las siguientes funciones:

- celsiusAFahrenheit(double celsius): Convierte grados Celsius a Fahrenheit usando la fórmula: $F=C\times(9/5)+32$. Después devolverá el resultado.
- fahrenheitACelsius(double fahrenheit): Convierte grados Fahrenheit a Celsius usando la fórmula: $C=(F-32)\times 5/9$. Después devolverá el resultado.
- kilometrosAMillas(double kilometros): Convierte kilómetros a millas usando la fórmula: $millas=kilometros\times 0.621371$. Después devolverá el resultado.
- millasAKilometros(double millas): Convierte millas a kilómetros usando la fórmula: $kilometros=millas/0.621371$. Después devolverá el resultado.
- kilogramosALibras(double kilogramos): Convierte kilogramos a libras usando la fórmula: $libras=kilogramos\times 2.20462$. Después devolverá el resultado.
- librasAKilogramos(double libras): Convierte libras a kilogramos usando la fórmula: $kilogramos=libras/2.20462$. Después devolverá el resultado.

En el método main, solicita al usuario que elija una conversión y un valor (que se pasará como parámetro). Después, mostrará el resultado por la consola.

TEMA 8. PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

Hasta ahora, hemos escrito programas donde todo el código estaba concentrado en una única parte: el método main. Hemos usado variables, estructuras de control, funciones y excepciones. Este enfoque es suficiente para programas pequeños o ejercicios simples, pero se queda corto cuando queremos construir aplicaciones más grandes, mantenibles y organizadas.

Aquí es donde entra la **Programación Orientada a Objetos (POO)**. Este estilo de programación propone una forma distinta de pensar en nuestros programas: en lugar de centrarnos solo en acciones (funciones que hacen cosas), **empezamos a pensar en "objetos" que tienen un comportamiento propio y un estado interno**.

Un objeto puede representar cualquier cosa: una persona, una cuenta bancaria, una ventana, un coche o incluso un botón en una aplicación. Cada uno de estos objetos combina **datos** (atributos) y **funciones** (métodos) que trabajan juntos. Por ejemplo, un objeto "Coche" puede tener como datos su velocidad o su marca, y como acciones métodos como arrancar, frenar o acelerar.

NOTA: Aunque **función** y **método** a veces se usan como sinónimos, en programación orientada a objetos, se hace una pequeña distinción. Así, como sabes, una función es un bloque de código independiente que realiza una tarea y puede devolver un valor. **No está ligada a ningún objeto.** Sin embargo, un método, **es una función que forma parte de una clase y normalmente se invoca sobre un objeto.**

La idea es construir programas que se parezcan más al mundo real, donde las cosas (objetos) actúan, cambian, interactúan entre sí, y tienen sus propias responsabilidades.

Para lograr esto, en Java trabajamos con **clases**, que son como plantillas o moldes a partir de los cuales se crean los objetos. Una clase define qué sabe hacer un objeto (sus métodos) y qué características tiene (sus atributos).

Veamos un ejemplo sencillo:

```
public class Puerta {
   int ancho;
   int alto;
   int color;
   boolean abierta;

   public void abrir() {
      abierta = true;
   }

   public void cerrar() {
      abierta = false;
   }

   public void mostrarEstado() {
      System.out.println("Ancho: " + ancho);
      System.out.println("Alto: " + alto);
      System.out.println("Color: " + color);
      System.out.println("Abierta: " + abierta);
   }
```

```
}
```

En este ejemplo, los objetos creados a partir de la clase Puerta tendrán una anchura, altura, color y un estado (abierta o cerrada). Además, pueden abrirse o cerrarse mediante los métodos definidos, y también pueden mostrar su estado actual en consola.

Para crear un objeto de esta clase, usamos la palabra clave new, igual que con los arrays:

```
Puerta p = new Puerta();
```

Aquí estás indicando que quieres un nuevo objeto de la clase Puerta llamado p. Este objeto tendrá las características que hemos definido en su clase, es decir, un alto, un ancho, un color y, además, se podrá abrir o cerrar y mostrar su estado.

Veamos ahora un programa completo que utiliza este objeto:

```
public class ejemploClases {
   public static void main(String[] args) {
      Puerta p = new Puerta();
      System.out.println("Valores iniciales...");
      p.mostrarEstado();
      System.out.println("Vamos a abrir...");
      p.abrir();
      p.mostrarEstado();
   }
}
```

Observa cómo a través del objeto p puedo acceder a los métodos abrir() y mostrarEstado().

Este código contiene dos clases:

- La clase Puerta, que define el objeto con el que trabajamos.
- La clase ejemploClases, que es el punto de entrada de nuestra aplicación (con el método main).

El resultado en consola será:

```
Valores iniciales...
Ancho: 0
Alto: 0
Color: 0
Abierta: false
Vamos a abrir...
Ancho: 0
Alto: 0
Color: 0
Abierta: true
```

Por defecto las clases deben estar en el **mismo paquete.** En este caso pueden usarse entre ellas sin necesidad de importar otro paquete externo al proyecto **(**import package.class). Tampoco es necesario escribir al comienzo de la clase package nombre_paquete.

Para añadir clases en Eclipse, sitúate sobre el paquete que quieras de tu proyecto, pulsa el botón secundario del ratón y elige "new/class" (ver ilustración 8.1).

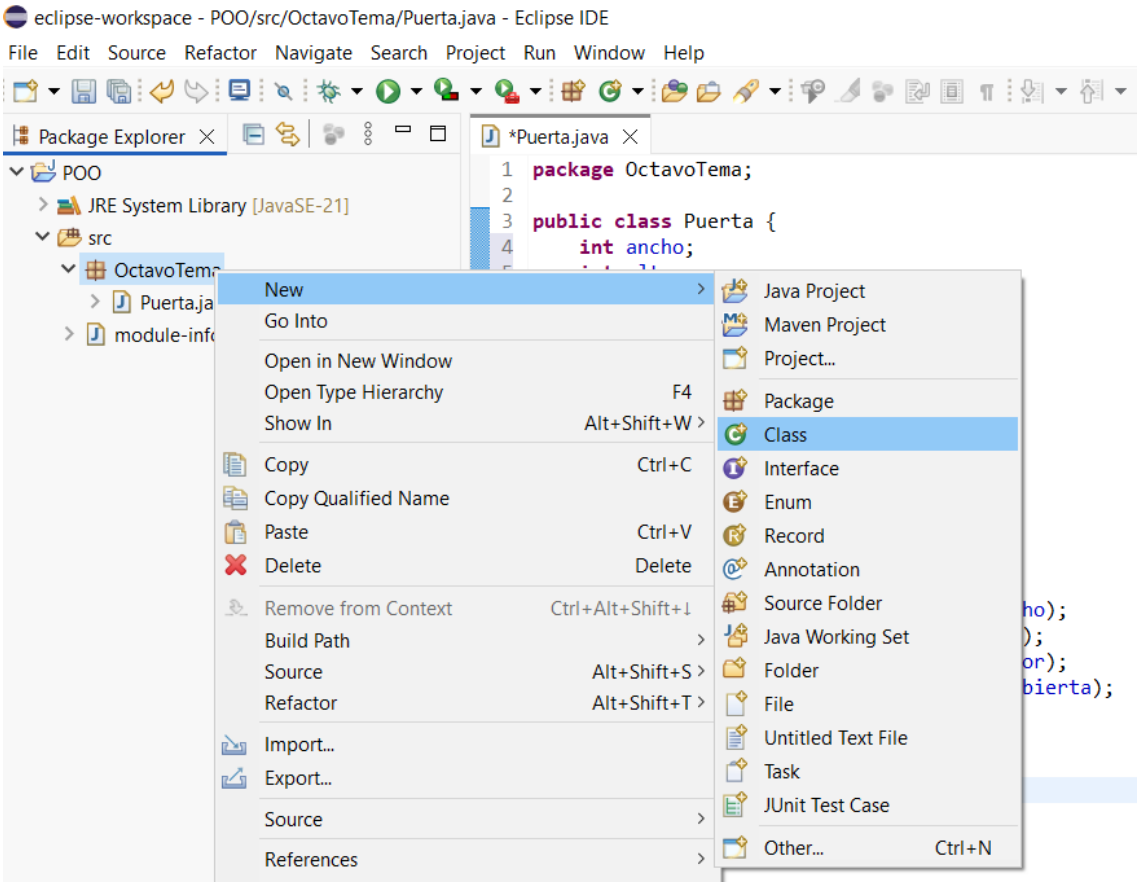

Ilustración 8.1. Añadiendo una nueva clase a nuestro proyecto. Elaboración propia.

Observa en la columna izquierda de la ilustración 8.1 (package explorer) que nuestro proyecto se llama POO y dentro de éste se encuentra una carpeta llamada "src" (source) que es donde se va a guardar todo nuestro código fuente del proyecto. Dentro de esta carpeta puedes crear tantos paquetes como quieras y, dentro de cada paquete, tantas clases como quieras.

Nosotros vamos a trabajar añadiendo clases dentro del mismo paquete.

8.1. Encapsulación

Aunque técnicamente podríamos acceder directamente a los atributos (por ejemplo, cambiar p.abierta = true desde el programa principal), **la Programación Orientada a Objetos recomienda NO hacerlo**. En su lugar, se aconseja **proteger los datos internos** de una clase y manipularlos solo a través de métodos especiales. A este principio se le llama **encapsulación**.

Encapsular significa que los detalles internos de un objeto (por ejemplo, cómo se representa si una puerta está abierta o cerrada) **quedan ocultos al exterior**. El resto del programa no necesita

saber cómo funciona por dentro, solo necesita usar los métodos que ofrece ese objeto. Esto nos da mucha más flexibilidad: si mañana cambiamos cómo funciona la clase internamente, no hará falta reescribir todo el programa.

Para seguir este principio, es común crear **métodos específicos para leer y modificar los atributos**. En inglés, se les suele llamar get y set. Por ejemplo, para acceder al ancho de una puerta podemos usar un método getAncho(), y para cambiarlo, setAncho(int nuevoValor).

Así conseguimos que los objetos tengan un comportamiento más controlado, claro y seguro. Observa cómo quedaría ahora la clase Puerta con los métodos get y set para obtener o modificar el valor del atributo Ancho y cómo se modifica el valor desde el main:

```
public class Puerta {
   int ancho;
   int alto;
   int color;
   boolean abierta;

   public void abrir() {
      abierta = true;
   }

   public void cerrar() {
      abierta = false;
   }

   public int getAncho() {
      return ancho;
   }

   public void setAncho(int nuevoValor) {
      ancho = nuevoValor;
   }

   public void mostrarEstado() {
      System.out.println("Ancho: " + ancho);
      System.out.println("Alto: " + alto);
      System.out.println("Color: " + color);
      System.out.println("Abierta: " + abierta);
   }
}
```

```
public class ejemploClases {
   public static void main(String[] args) {
      Puerta p = new Puerta();
      System.out.println("Valores iniciales...");
      p.mostrarEstado();
      System.out.println("Vamos a abrir...");
      p.abrir();
      p.setAncho(80);
      p.mostrarEstado();
```

```
    }
}
```

Ejercicio 8.1. Crea una clase llamada Persona. Esta clase deberá tener un atributo "nombre", de tipo string. También deberá tener un método "setNombre", de tipo void y con un parámetro string, que permita cambiar el valor del nombre. Finalmente, también tendrá un método "saludar", que escribirá en pantalla "Hola, soy " seguido de su nombre. Crea también una clase llamada PruebaPersona. Esta clase deberá contener solo la función main, que creará dos objetos de tipo Persona, les asignará un nombre a cada uno y les pedirá que saluden.

Ejercicio 8.2. Para guardar información sobre libros, vamos a comenzar por crear una clase llamada "Libro", que contendrá atributos "autor", "titulo", "ubicacion" (todos ellos strings) y métodos Get y Set adecuados para leer su valor y cambiarlo. Prepara también un main en otra clase llamada “pruebaLibro”, que cree un objeto de la clase Libro, dé valores a sus tres atributos y luego los muestre.

Ejercicio 8.3. Crea la clase Coche, que tendrá atributos "marca" (texto), "modelo" (texto), "cilindrada" (número entero) y potencia (número real). Crea también la clase PruebaDeCoche, que contendrá la función "main", que creará un objeto de tipo coche, pedirá al usuario su marca, modelo, cilindrada y potencia, y luego mostrará en pantalla el valor de esos datos.

8.2. Herencia

En la Programación Orientada a Objetos, uno de los conceptos más importantes es la **herencia**. Este mecanismo permite crear nuevas clases basadas en otras ya existentes. Es decir, una clase puede heredar los atributos y métodos de otra clase, lo que evita repetir código y permite establecer relaciones entre tipos de objetos.

Por ejemplo, ya tenemos una clase llamada Puerta que representa puertas normales, con atributos como ancho, alto, color y si están abiertas o cerradas. Ahora queremos crear una nueva clase llamada Porton, que será una especie de puerta más grande o especial. Un portón también tendrá ancho, alto, color y podrá abrirse o cerrarse, igual que cualquier puerta, **pero además podrá bloquearse**.

Para evitar volver a escribir todo lo que ya tiene una puerta, hacemos que la clase Porton **herede** de Puerta. En Java esto se hace con la palabra clave extends:

```
public class Porton extends Puerta {
   boolean bloqueada;

   public void bloquear() {
      bloqueada = true;
   }

   public void desbloquear() {
      bloqueada = false;
   }
}
```

Esto significa que Porton automáticamente tiene todos los atributos y métodos públicos que ya tiene Puerta. Por eso, no es necesario volver a declarar ni el ancho, ni el método abrir(), ni nada que ya esté en la clase Puerta.

Además, podemos **redefinir** métodos heredados para que hagan algo distinto. Por ejemplo, si queremos que al mostrar el estado del Porton se indique también si está bloqueado, podemos sobrescribir el método mostrarEstado():

```
public void mostrarEstado() {
   System.out.println("Portón.");
   System.out.println("Bloqueada: " + bloqueada);
}
```

Aquí estamos **modificando el comportamiento** del método que ya existía en la clase Puerta. Esto se llama **sobrescritura de métodos** (override), y es una herramienta muy poderosa cuando se trabaja con herencia (veremos con profundidad la palabra “override” más adelante).

Veamos un programa de prueba que crea primero una puerta normal y luego un portón:

```
public class PruebaPuerta {
   public static void main(String[] args) {
      Puerta p = new Puerta();
      System.out.println("Valores iniciales...");
      p.mostrarEstado();
      System.out.println("Vamos a abrir...");

      p.abrir();
      p.setAncho(80);
      p.mostrarEstado();

      System.out.println("Ahora el portón...");
      Porton p2 = new Porton();
      p2.bloquear();
      p2.mostrarEstado();
   }
}
```

Y la clase puerta quedaría:

```
public class Puerta {
   int ancho;
   int alto;
   int color;
   boolean abierta;

   public void abrir() {
      abierta = true;
   }

   public void cerrar() {
      abierta = false;
```

```
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int nuevoValor) {
        ancho = nuevoValor;
    }

    public void mostrarEstado() {
        System.out.println("Ancho: " + ancho);
        System.out.println("Alto: " + alto);
        System.out.println("Color: " + color);
        System.out.println("Abierta: " + abierta);
    }
}
```

Y la clase Porton:

```
public class Porton extends Puerta {
    boolean bloqueada;

    public void bloquear() {
        bloqueada = true;
    }

    public void desbloquear() {
        bloqueada = false;
    }

    public void mostrarEstado() {
        System.out.println("Portón.");
        System.out.println("Bloqueada: " + bloqueada);
    }
}
```

Al ejecutar el programa veríamos esto por la consola:

```
Valores iniciales...
Ancho: 0
Alto: 0
Color: 0
Abierta: false
Vamos a abrir...
Ancho: 80
Alto: 0
Color: 0
Abierta: true
Ahora el portón...
Portón.
```

Bloqueada: true

En próximos apartados veremos cómo podemos acceder también desde Porton a los atributos heredados de Puerta, como el ancho o el estado de apertura, y cómo combinar correctamente ambos comportamientos. La herencia nos permitirá construir sistemas más ricos, reutilizando código y representando relaciones entre objetos del mundo real.

Ejercicio 8.4. Crea una variante ampliada del ejercicio 8.1. En ella, la clase Persona no cambia. Se creará una nueva clase PersonaInglesa. Esta clase deberá heredar las características de la clase "Persona", y añadir un método "TomarTe", de tipo void, que escribirá en pantalla "Estoy tomando té". Ahora, la clase PruebaPersona creará dos objetos de tipo Persona y uno de tipo PersonaInglesa, les asignará un nombre, les pedirá que saluden y pedirá a la persona inglesa que tome té.

Ejercicio 8.5. Amplía el ejercicio 8.2. Crea una clase "Documento", de la que Libro heredará todos sus atributos y métodos. Ahora la clase Libro contendrá sólo un atributo "paginas", número entero, con sus correspondientes get y set.

Ejercicio 8.6. Amplía el proyecto de la clase Coche (ejercicio 8.3). Crea una clase "Vehiculo", de la que heredarán "Coche" y una nueva clase "Moto". La clase Vehiculo contendrá todos los atributos y métodos que antes estaban en Coche, y tanto Coche como Moto heredarán de ella.

8.3. Modificadores de acceso

En Java, los modificadores de acceso nos permiten controlar desde dónde se puede acceder a los atributos y métodos de una clase. Esta es una parte fundamental del principio de encapsulación, que busca proteger los datos internos de una clase y permitir que otras partes del programa interactúen solo con lo necesario. Encapsular bien el código nos ayuda a evitar errores y facilita el mantenimiento a largo plazo.

Existen cuatro niveles de acceso en Java: public, private, protected y el acceso por defecto (sin ningún modificador). Cuando algo se declara como public, puede ser accedido desde cualquier clase, esté o no en el mismo paquete. Si se declara como private, solo puede utilizarse dentro de la misma clase en la que fue definido, lo que lo convierte en el nivel de protección más estricto. Esto es útil para proteger datos sensibles o internos que no deben ser modificados directamente desde fuera de la clase.

Por otro lado, el modificador protected ofrece un punto intermedio. Permite el acceso desde la propia clase, desde cualquier clase del mismo paquete y también desde las subclases, incluso si estas se encuentran en paquetes distintos. Es especialmente útil cuando se trabaja con herencia, ya que permite que una subclase acceda a ciertos métodos o atributos de su clase padre sin necesidad de que sean públicos. Sin embargo, por motivos de buena práctica, se suele aplicar este nivel de acceso a métodos más que a atributos, para no debilitar la encapsulación.

Finalmente, si no se indica ningún modificador de acceso, el comportamiento por defecto permite el acceso únicamente desde otras clases del mismo paquete. Esto se conoce como acceso "package-private".

Entender bien los modificadores de acceso es esencial para diseñar clases seguras y reutilizables. Por ejemplo, si tenemos una clase Puerta con un atributo abierta, podríamos declararlo como

private para que no pueda modificarse directamente. En su lugar, ofreceríamos métodos abrir() y cerrar() públicos que modifiquen el estado de ese atributo de forma controlada. Si creamos una subclase como Porton, podríamos usar protected si queremos que esta subclase tenga acceso directo a ciertos métodos o atributos sin exponerlos completamente al exterior.

Gracias a estos niveles de acceso, Java nos da flexibilidad para organizar mejor nuestro código y proteger la lógica interna de nuestras clases.

En la tabla 8.1 puedes ver todos los modificadores de acceso:

Modificador de acceso	Clase	Paquete	Subclase	Todos
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
default	Sí	Sí	No	No
private	Sí	No	No	No

Tabla 8.1. Modificadores de acceso. Elaboración propia.

Ejercicio 8.7. Desarrolla un programa con una superclase Profesor (clase base o padre) y una subclase ProfesorEmerito (clase hija o derivada). Crea el código para estas clases cumpliendo los siguientes requisitos:

- Como atributos de la superclase tendremos nombre (String), edad (int) y añosConsolidados (int) declarados como protected.
- En la subclase se trabajará con el campo adicional añosEmerito declarado como private.
- Un método de la subclase será double obtenerSalarioBase () que obtendrá el salario base como (925 + añosConsolidados * 33.25 + 47.80 * añosEmerito).

¿Puedes acceder al campo añosConsolidados desde la subclase? ¿Por qué? ¿Qué ocurre si el atributo en la superclase lo declaras private?

Ejercicio 8.8. Continua con el ejercicio de la clase Persona añadiendo las siguientes características:

- La clase Persona no cambia.
- La clase PersonaInglesa se modificará para que redefina el método "Saludar", para que escriba en pantalla "Hi, I am " seguido de su nombre.
- Se creará una nueva clase PersonaItaliana, que deberá heredar las características de la clase "Persona", pero redefinir el método "saludar", para que escriba en pantalla "Ciao".
- La clase PruebaPersona deberá contener solo la función main, y creará un objeto de tipo Persona, dos de tipo PersonaInglesa, uno de tipo PersonaItaliana, les asignará un nombre, les pedirá que saluden y pedirá a la persona inglesa que tome té.

Ejercicio 8.9. Continua el ejercicio de la clase Libro. Haz que los atributos de la clase Documento y de la clase Libro sean "protegidos".

Ejercicio 8.10. Mejora el ejercicio de la clase Coche, donde todos los atributos serán "protegidos" y los métodos serán "públicos".

8.4. Constructores

Hasta ahora hemos visto que, al declarar una clase en Java, los atributos reciben automáticamente valores por defecto: los números se inician en 0, los booleanos en false y las referencias (como los objetos o textos) se inicializan como null. Sin embargo, es frecuente que necesitemos dar a esos atributos unos valores iniciales específicos desde el momento en que se crea el objeto. Para ello, Java proporciona una herramienta fundamental: los **constructores**.

Un **constructor** es un método especial que se ejecuta automáticamente en el momento en que se crea un nuevo objeto. Su función principal es **inicializar los atributos** del objeto con valores determinados. No devuelve ningún tipo de dato, ni siquiera void, y **debe llamarse exactamente igual que la clase**.

Características de un constructor:

- Tiene el mismo nombre que la clase.
- Se ejecuta automáticamente al crear un objeto.
- No devuelve datos.
- Se ejecuta una única vez por cada objeto.
- Sirve para dar valores iniciales a los atributos, reservar recursos, leer archivos, etc.

Veamos un ejemplo con la clase Puerta. Supongamos que queremos que cada nueva puerta tenga un ancho de 100, un alto de 200, un color predeterminado y esté cerrada por defecto. Podemos definir el siguiente constructor:

```
public Puerta() {
   ancho = 100;
   alto = 200;
   color = 0xFFFF;
   abierta = false;
}
```

NOTA: El color se inicializa en hexadecimal por comodidad y claridad al representar valores de color, ya que el sistema hexadecimal se ajusta perfectamente al modelo de colores RGB que usan muchos entornos gráficos y lenguajes de programación, incluido Java.

También es posible definir **constructores con parámetros**, lo que nos permite crear objetos personalizados indicando, por ejemplo, el ancho y el alto al momento de la creación:

```
public Puerta(int an, int al) {
   ancho = an;
   alto = al;
   color = 0xFFFF;
   abierta = false;
}
```

De esta forma, si escribimos:

```
Puerta p = new Puerta();
```

el objeto p tendrá ancho 100 y alto 200. Pero si usamos:

```
Puerta p2 = new Puerta(90, 220);
```

el objeto p2 tendrá los valores que hemos indicado como parámetros.

Mira este ejemplo, vamos a hacer un programa que utiliza ambos constructores para crear dos puertas diferentes y luego trabaja con ellas:

```
public class PruebaPuerta {
   public static void main(String[] args) {
      Puerta p = new Puerta();
      Puerta p2 = new Puerta(90, 220);

      System.out.println("Valores iniciales...");
      p.mostrarEstado();

      System.out.println("Vamos a abrir...");
      p.abrir();
      p.mostrarEstado();

      System.out.println("Ahora la segunda puerta...");
      p2.mostrarEstado();

      System.out.println("Ahora el portón...");
      Porton por = new Porton();
      por.bloquear();
      por.mostrarEstado();
   }
}
```

Y la clase Puerta quedaría así:

```
public class Puerta {
   int ancho;
   int alto;
   int color;
   boolean abierta;

   public Puerta() {
      ancho = 100;
      alto = 200;
      color = 0xFFFF;
      abierta = false;
   }

   public Puerta(int an, int al) {
```

```
        ancho = an;
        alto = al;
        color = 0xFFFF;
        abierta = false;
    }

    public void abrir() {
        abierta = true;
    }

    public void cerrar() {
        abierta = false;
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int nuevoValor) {
        ancho = nuevoValor;
    }

    public void mostrarEstado() {
        System.out.println("Ancho: " + ancho);
        System.out.println("Alto: " + alto);
        System.out.println("Color: " + color);
        System.out.println("Abierta: " + abierta);
    }
}
```

De esta forma, el uso de constructores nos permite una mayor flexibilidad y control a la hora de crear objetos, y es una herramienta fundamental dentro de la programación orientada a objetos.

Observa que, tanto un **setter** (método auxiliar set) como un **constructor con parámetros** pueden utilizarse para asignar valores a los atributos de un objeto. Sin embargo, **no se usan igual ni se prioriza uno sobre otro automáticamente**. Depende del contexto, del diseño del programa y de cuándo necesitas establecer esos valores usarás uno u otro.

El constructor con parámetros se usa **al crear el objeto**, para asegurarte de que **desde el principio tiene todos los valores necesarios**. Es ideal cuando **ciertos atributos son obligatorios** y el objeto no tiene sentido sin ellos.

Los setters se usan **después de haber creado el objeto**, si necesitas modificar sus valores. Son útiles cuando los atributos pueden **cambiar durante el ciclo de vida del objeto**.

A veces se pueden combinar: un constructor da valores iniciales, y los setters permiten ajustes posteriores.

Ejercicio 8.11. Continua el ejercicio de las clases Persona para que todas ellas contengan constructores. Los constructores de casi todas las clases estarán vacíos, excepto del de

PersonaInglesa, que prefijará su nombre a "John". Crea también un constructor alternativo para esta clase que permita escoger cualquier otro nombre.

Ejercicio 8.12. Continua el ejercicio de la clase Libro. Ahora tendrá un constructor que permitirá dar valores al autor, el título y la ubicación.

Ejercicio 8.13. Continua el ejercicio de la clase Coche. Añade un atributo "cantidadDeRuedas" a la clase Vehiculo, junto con sus get y set. El constructor de la clase Coche le dará el valor 4 y el constructor de la clase Moto le dará el valor 2.

8.5. Polimorfismo y sobrecarga

El **polimorfismo** es uno de los pilares fundamentales de la Programación Orientada a Objetos. Este concepto significa que un **mismo método puede tener diferentes comportamientos**, dependiendo del contexto en el que se utilice. Es decir, una misma función puede tener "muchas formas".

Existen dos tipos principales de polimorfismo:

8.5.1. Polimorfismo por sobrecarga (o *ad-hoc*)

La **sobrecarga** ocurre cuando varios métodos tienen el mismo nombre, pero se diferencian por el **número o tipo de parámetros** que reciben. En estos casos, es el **compilador quien decide en tiempo de compilación** cuál de las versiones debe usar, en función de los argumentos proporcionados.

Por ejemplo, en la clase Puerta, tenemos dos constructores:

```
public Puerta() {
   // Constructor sin parámetros
}

public Puerta(int an, int al) {
   // Constructor con dos parámetros
}
```

Ambos métodos se llaman igual (Puerta), pero tienen diferentes parámetros. Esto permite crear objetos con distintas inicializaciones sin necesidad de usar nombres diferentes para cada constructor o método.

Este mismo mecanismo puede aplicarse a cualquier método, no solo a los constructores:

```
public void abrir() { ... }

public void abrir(int grados) { ... } // Sobrecargado: acepta un parámetro
```

8.5.2. Polimorfismo puro (o *de inclusión*)

Este tipo de polimorfismo está relacionado con la **herencia**. Se da cuando un método se declara en una superclase y luego se **redefine (override)** en una subclase. El método puede llamarse igual, pero el comportamiento depende del tipo real del objeto.

Por ejemplo, en el método mostrarEstado() que aparece tanto en Puerta como en Porton, el método se ejecuta de forma distinta dependiendo del tipo de objeto:

```
Puerta p = new Porton(); // Referencia de tipo Puerta, objeto real tipo Porton
p.mostrarEstado();      // Se ejecuta la versión de Porton, no la de Puerta
```

Aquí, aunque la variable p es de tipo Puerta, el objeto es realmente un Porton, y por eso se ejecuta el método mostrarEstado de Porton. Esto se llama **polimorfismo en tiempo de ejecución**, y es el compilador quien delega esta decisión al momento de ejecutar el programa.

Ejercicio 8.14. Continúa con el ejercicio de la clase "Persona" y añade un nuevo método Saludar, que reciba un parámetro, que será el texto que debe decir esa persona cuando salude.

Ejercicio 8.15. Continúa el ejercicio de la clase Libro. Ahora, la clase Libro tendrá un segundo constructor que permita dar valores al autor y el título, pero no a la ubicación, que tomará el valor por defecto "No detallada". Crea un nuevo objeto para probar este constructor.

Ejercicio 8.16. Continúa el ejercicio de la clase Coche. Crea dos nuevos métodos en la clase Vehiculo: uno llamado Circular, que fijará su "velocidad" (un nuevo atributo) a 50, y otro Circular(v), que fijará su velocidad al valor que se indique como parámetro.

8.6. Orden de llamada de los constructores

Cuando trabajamos con **herencia** y creamos un objeto de una clase derivada (subclase), el **orden de ejecución de los constructores** sigue siempre la misma lógica: **primero se ejecutan los constructores de las clases más generales (superclases)** y, luego, **los de las clases más específicas (subclases)**.

Es decir, cuando una clase hereda de otra, el objeto resultante **no es solo la subclase**, sino **una combinación de la subclase más todas sus superclases**. Por lo tanto, **cada nivel de la jerarquía debe estar correctamente construido antes de avanzar al siguiente**.

Cada clase puede tener atributos propios que necesitan ser configurados. Si primero se ejecuta el constructor de la superclase, nos aseguramos de que esa parte del objeto (los atributos heredados) esté correctamente inicializada **antes** de añadir o modificar cosas en la subclase.

Vamos a ver un ejemplo para aclarar el concepto. Supongamos una jerarquía de clases como la siguiente:

- Animal (la más general)
- Gato (hereda de Animal)
- GatoSiames (hereda de Gato)

```
public class Animal {
        public Animal() {
                System.out.println("Ha nacido un animal.");
        }
}
```

```
public class Gato extends Animal {
	public Gato() {
		System.out.println("Ha nacido un gato.");
	}
}

public class GatoSiames extends Gato {
	public GatoSiames() {
		System.out.println("Ha nacido un gato siamés.");
	}
}
```

Y un programa que crea objetos de esas clases:

```
public class PruebaGatos {

	public static void main(String[] args) {
		Animal a1 = new Animal();
		System.out.println();

		Gato a2 = new Gato();
		System.out.println();

		GatoSiames a3 = new GatoSiames();
	}
}
```

La salida por consola es:

```
Ha nacido un animal.

Ha nacido un animal.
Ha nacido un gato.

Ha nacido un animal.
Ha nacido un gato.
Ha nacido un gato siamés.
```

Esto demuestra que, al crear un objeto GatoSiames, **primero se ejecuta el constructor de Animal, luego el de Gato, y por último el de GatoSiames**.

Este comportamiento ocurre automáticamente, a menos que la subclase indique de forma explícita qué constructor de la superclase desea invocar (usando la palabra reservada “super”. Se verá este tema más adelante).

Ejercicio 8.17. Crea un proyecto que contenga:

- Una clase Trabajador, cuyo constructor escriba en pantalla "Soy un trabajador".

- Una clase Programador, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy programador".
- Una clase Analista, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy analista".
- Una clase Ingeniero, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy ingeniero".
- Una clase IngenieroInformatico, que derive de Ingeniero, cuyo constructor escriba en pantalla "Soy ingeniero informático".
- Una clase "PruebaDeTrabajadores", que cree un objeto perteneciente a cada una de esas clases.

Ejercicio 8.18. Continúa con el proyecto Libro en la que el constructor de Documento escriba en pantalla "Creando documento" y el constructor de Libro escriba en pantalla "Creando libro". Comprueba su funcionamiento.

Ejercicio 8.19. Crea una versión alternativa de las clases Vehiculo, Coche y Moto, para que el programa te avise del momento en que se entra a cada constructor. Crea un programa de prueba que defina un coche y una moto, y comprueba su funcionamiento.

8.7. Arrays de objetos

En muchos casos, no nos basta con tener un único objeto de una clase, sino que necesitamos **manipular varios objetos del mismo tipo**. Para ello, usamos **arrays**.

Un array de objetos no crea automáticamente los objetos, sino que sólo reserva el espacio para referencias. Por eso debemos usar new dos veces: una vez para **crear el array** y otra vez para **crear cada objeto dentro del array**.

Mira este ejemplo:

```
Perro[] misPerros = new Perro[5];

for (int i = 0; i < 5; i++) {
   misPerros[i] = new Perro();
}
```

Ahora mira un ejemplo completo:

```
public class PruebaAnimal {
   public static void main(String[] args) {
      Perro[] misPerros = new Perro[5];
      for (int i = 0; i < 5; i++) {
         misPerros[i] = new Perro();
      }
   }
}
```

```
public class Animal {
   public Animal() {
      System.out.println("Ha nacido un animal.");
```

```
    }
}
```

```
public class Perro extends Animal {
    public Perro() {
        System.out.println("Ha nacido un perro.");
    }
}
```

Y la salida sería:

```
Ha nacido un animal.
Ha nacido un perro.
(repetido 5 veces)
```

Ejercicio 8.20. Continúa con el ejercicio de la clase Trabajador. Ahora hay que instanciar 3 objetos de cada clase en arrays.

Ejercicio 8.21. Amplía el proyecto Libro de modo que permita guardar hasta 1.000 libros. La función *main* mostrará un menú que permita añadir un nuevo libro o ver los datos de los ya existentes.

Un array también puede estar compuesto por **objetos de distintas subclases**, siempre que todas hereden de una misma clase padre.

```
Animal[] misAnimales = new Animal[3];

misAnimales[0] = new Perro();
misAnimales[1] = new Gato();
misAnimales[2] = new GatoSiames();
```

Esto es posible porque **un objeto de una subclase puede almacenarse en una variable de tipo de su superclase**. Es un ejemplo de **polimorfismo**.

Ejemplo completo:

```
public class PruebaAnimales {
    public static void main(String[] args) {
        Animal[] misAnimales = new Animal[8];

        misAnimales[0] = new Perro();
        misAnimales[1] = new Gato();
        misAnimales[2] = new GatoSiames();

        for (int i = 3; i < 7; i++) {
            misAnimales[i] = new Perro();
        }
```

```
        misAnimales[7] = new Animal();
    }
}
```

```
public class Animal {
    public Animal() {
        System.out.println("Ha nacido un animal.");
    }
}
```

```
public class Perro extends Animal {
    public Perro() {
        System.out.println("Ha nacido un perro.");
    }
}
```

```
public class Gato extends Animal {
    public Gato() {
        System.out.println("Ha nacido un gato.");
    }
}
```

```
public class GatoSiames extends Gato {
    public GatoSiames() {
        System.out.println("Ha nacido un gato siamés.");
    }
}
```

Y la salida:

```
Ha nacido un animal.
Ha nacido un perro.
Ha nacido un animal.
Ha nacido un gato.
Ha nacido un animal.
Ha nacido un gato.
Ha nacido un gato siamés.
Ha nacido un animal.
Ha nacido un perro.
Ha nacido un animal.
Ha nacido un perro.
Ha nacido un animal.
Ha nacido un perro.
Ha nacido un animal.
Ha nacido un perro.
Ha nacido un animal.
```

Ejercicio 8.22. En el ejercicio de la clase Trabajador, crea un array de trabajadores en el que no sean todos de la misma clase.

Ejercicio 8.23. Amplía el proyecto Libro de modo que permita guardar 1000 documentos de cualquier tipo. A la hora de añadir un documento, se preguntará al usuario si desea guardar un documento genérico o un libro, para usar el constructor adecuado.

8.8. Sobrescribir un método en una subclase

En programación orientada a objetos, sobrescribir un método significa **redefinir un método heredado** en una subclase para que se comporte de manera diferente.

Vamos a usar el ejemplo previo de animales: Animal, Perro y Gato. Todos tienen un método llamado hablar(), pero **cada uno "habla" de forma diferente**.

```
public class Animal {
   public Animal() {}

   public void hablar() {
      System.out.println("Estoy comunicándome...");
   }
}

public class Perro extends Animal {
   public Perro() {}

   public void hablar() {
      System.out.println("Guau!!");
   }
}

public class Gato extends Animal {
   public Gato() {}

   public void hablar() {
      System.out.println("Miau!!");
   }
}
```

Vamos a probarlo de dos formas distintas: una instanciando objetos directamente de su clase correspondiente y la otra a través de un array de objetos Animales:

```
public class PruebaAnimales {
   public static void main(String[] args) {
      Perro perro = new Perro();
      Gato gato = new Gato();
      Animal animal = new Animal();

      perro.hablar();
      gato.hablar();
      animal.hablar();
```

```
        System.out.println();

        Animal[] misAnimales = new Animal[3];
        misAnimales[0] = new Perro();
        misAnimales[1] = new Gato();
        misAnimales[2] = new Animal();

        misAnimales[0].hablar();
        misAnimales[1].hablar();
        misAnimales[2].hablar();
    }
}
```

Y la salida sería:

```
Guau!!
Miau!!
Estoy comunicándome...

Guau!!
Miau!!
Estoy comunicándome...
```

Observa que el resultado es el mismo. Esto ocurre porque cuando usamos un array de tipo Animal[] almacenando objetos de Perro o Gato, Java usa **polimorfismo dinámico** (también llamado "ligadura tardía") para decidir **en tiempo de ejecución** qué versión de hablar() debe ejecutarse.

PERO OJO, aunque el código funcione sin errores, **es altamente recomendable** usar la anotación **@Override** cuando sobrescribimos métodos. Al hacer esto el compilador verifica que realmente estamos sobrescribiendo (evita errores por escribir mal el nombre o la firma del método). Además, mejora la legibilidad del código para otros programadores (o para ti en el futuro).

Ejemplo correcto con @Override:

```
public class Perro extends Animal {
    public Perro() {}

    @Override //aunque funcione sin esta palabra es muy recomendable usarla
    public void hablar() {
        System.out.println("Guau!!");
    }
}
```

¿Qué ocurre sin no usamos @Override y nos equivocamos?

```
public class Perro extends Animal {
    public Perro() {}
```

```
    // Error de tipeo: mal escrito el nombre
    public void ablar() {
        System.out.println("Guau!!");
    }
}
```

El compilador **NO daría error**, pero tampoco sobrescribimos nada. El método original hablar() del padre seguiría ejecutándose.

Si usamos @Override en ese caso, el compilador sí daría un error como este:

```
method does not override or implement a method from a supertype
```

Ejercicio 8.24. Prueba lo estudiado en el ejercicio de la clase Persona. Crea un único array que contenga personas de varios tipos y pídeles que saluden. Los métodos saludar() de la clase PersonaInglesa y PersonaItaliana estarán sobrescritos.

Ejercicio 8.25. Crea una variante del ejercicio clase Trabajador y derivadas en la que se cree un único array "de trabajadores", que contenga un objeto de cada clase, y exista un método "saludar()" que se sobrescriba en todas las clases hijas.

Ejercicio 8.26. Investiga qué hace y dónde se encuentra el método toString(). Ahora amplía el proyecto Libro. Tanto la clase Documento como la clase Libro, tendrán un método toString(), que devuelva una cadena de texto formada por título, autor y ubicación. Crea una clase Articulo, que añada el campo "procedencia". El cuerpo del programa permitirá añadir Artículos o Libros, no documentos genéricos. El método toString() deberá mostrar también el número de páginas de un libro y la procedencia de un artículo. La opción de mostrar datos llamará a los correspondientes métodos toString().

8.9. Llamando a un método de la superclase

Puede ocurrir que en un método de una clase hija no nos interese redefinir por completo las funcionalidades del método equivalente de la superclase, sino simplemente ampliarlas. En ese caso, no es necesario reescribir todo el contenido del método de la clase base, ya que podemos llamarlo directamente utilizando la palabra clave **super**.

Por ejemplo, si queremos que un **Gato Siamés** hable igual que un **Gato** normal pero que además diga “Pfff” al final, podemos hacerlo así:

```
public class GatoSiames extends Gato {
    public GatoSiames() {
        // Constructor vacío
    }

    @Override
    public void hablar() {
        super.hablar(); // Llama al método hablar() de Gato
        System.out.println("Pfff");
    }
}
```

El ejemplo completo sería:

```
public class PruebaGatos {
   public static void main(String[] args) {
      Gato gato = new Gato();
      GatoSiames gs = new GatoSiames();

      gato.hablar();
      gs.hablar();
   }
}
```

```
public class Gato extends Animal {
   public Gato() {
      // Constructor vacío
   }

   public void hablar() {
      System.out.println("Miau!!");
   }
}
```

```
public class GatoSiames extends Gato {
   public GatoSiames() {
      // Constructor vacío
   }

   @Override
   public void hablar() {
      super.hablar(); // Llama al método hablar() de la clase Gato
      System.out.println("Pfff");
   }
}
```

Y la salida sería:

```
Miau!!
Miau!!
Pfff
```

Como se puede observar, la clase GatoSiames reutiliza el comportamiento del método hablar() de su clase base (Gato) y añade algo más.

La palabra clave **super** también se puede usar para invocar explícitamente el **constructor de la clase padre** desde el constructor de una subclase. Por ejemplo:

```
// Superclase Persona
class Persona {
```

```
    Persona() {
        System.out.println("Constructor de la clase Persona");
    }
}
```

```
// Subclase Estudiante que hereda de Persona
class Estudiante extends Persona {
    Estudiante() {
        super(); // Llama al constructor de la clase padre
        System.out.println("Constructor de la clase Estudiante");
    }
}
```

```
// Clase de prueba
class Test {
    public static void main(String[] args) {
        Estudiante e = new Estudiante();
    }
}
```

Y la salida sería:

```
Constructor de la clase Persona
Constructor de la clase Estudiante
```

Esto demuestra que cuando se instancia un objeto de una subclase, primero se ejecuta el constructor de la superclase, lo que garantiza que el objeto herede correctamente las propiedades y comportamientos del nivel superior antes de aplicar sus propias características.

Habrás observado que si no escribes super() de forma explícita, Java lo añade automáticamente siempre y cuando el constructor padre **no tenga parámetros**. Esto significa que el orden de llamada de los constructores se respeta igual, **incluso si no pones super()** a mano.

Es decir, super() **NO es necesario** escribirlo si el constructor del padre **es el constructor por defecto (sin parámetros)**. Java lo llama automáticamente al comienzo. Pero **SÍ es obligatorio** escribir super(...) si el constructor del padre **requiere parámetros**. En ese caso, si no lo escribes, Java da error de compilación porque **no sabe cómo llamar al constructor padre**.

```
class A {
    A(String msg) {
        System.out.println("Mensaje: " + msg);
    }
}

class B extends A {
    B() {
        // Error si no ponemos super("Hola");
        super("Hola");
        System.out.println("Constructor B");
```

```
    }
}
```

Si te estás preguntando: “¿Por qué complicarme con super("Hola") si simplemente podría hacer las cosas en la clase hija?”, la respuesta está en el **principio de reutilización de código y diseño estructurado**. Veamos por qué **sí tiene sentido** hacerlo así en algunos casos:

```
class Vehiculo {
    Vehiculo(String tipo) {
        System.out.println("Vehículo creado de tipo: " + tipo);
    }
}
```

```
class Coche extends Vehiculo {
    Coche() {
        super("Coche"); // Tiene sentido: Coche es un tipo de Vehículo
        System.out.println("Coche creado");
    }
}
```

```
class Moto extends Vehiculo {
    Moto() {
        super("Moto"); // Moto es otro tipo de Vehículo
        System.out.println("Moto creada");
    }
}
```

Como ves, en este caso estamos evitando rescribir código.

Mira otro ejemplo donde la superclase valida o inicializa algo esencial:

```
class Usuario {
    Usuario(String nombre) {
        if (nombre == null || nombre.isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }
        System.out.println("Usuario creado: " + nombre);
    }
}
```

En este caso subclases como, por ejemplo, Cliente, no debería omitir eso. Solo llama a super(nombre) y ya se asegura de que la validación ocurra.

Ejercicio 8.27. Amplía el ejercicio de la clase Trabajador. Haz que el método "saludar" de todas las clases hijas se apoye en el de la clase "Trabajador".

Ejercicio 8.28. Ahora amplía el ejercicio de la clase Trabajador y haz que el constructor de todas las clases hijas se apoye en el de la clase "Trabajador".

8.10. La palabra reservada “this”

La palabra **this** hace referencia al **objeto actual** (la instancia actual de la clase). Se usa principalmente para **distinguir entre variables de instancia y parámetros con el mismo nombre.** Es decir, cuando los parámetros de un constructor o método se llaman igual que los atributos de la clase, usamos this para diferenciarlos. Así:

```
public class Titulo {
   private int x;
   private int y;
   private String texto;

   public Titulo(int x, int y, String texto) {
      this.x = x;        // "this.x" se refiere al atributo de la clase
      this.y = y;        // "y" es el parámetro
      this.texto = texto;
   }

   public void escribir() {
      System.out.println(x + y);
      System.out.println(texto);
   }
}
```

Si no quieres usar this puedes cambiar el nombre de los parámetros para que no se confundan. Así:

```
public Titulo(int nuevaX, int nuevaY, String nuevoTexto) {
   x = nuevaX;
   y = nuevaY;
   texto = nuevoTexto;
}
```

Además, cuando una clase tiene más de un constructor, this(...) permite **reutilizar uno dentro de otro**. Así:

```
class Test {
   int a;
   int b;

   Test() {
      this(10, 20); // Llama al otro constructor
      System.out.println("Dentro del constructor predeterminado");
   }

   Test(int a, int b) {
      this.a = a;
      this.b = b;
```

```
        System.out.println("Dentro del constructor parametrizado");
    }

    public static void main(String[] args) {
        Test obj = new Test();
    }
}
```

Y la salida sería:

```
Dentro del constructor parametrizado
Dentro del constructor predeterminado
```

Dentro del constructor sin parámetros, se llama al otro constructor con this(10, 20). Es decir, cuando creas un objeto con new Test(), en realidad se ejecuta primero el constructor con parámetros.

De esta forma **evitas duplicación de código**, **centralizas la lógica de inicialización** (lo que facilita el mantenimiento) y puedes ofrecer **diferentes formas de crear objetos**, con o sin parámetros.

Ejercicio 8.29. Utiliza el ejercicio de la clase Trabajador para mostrar la utilidad de la palabra "*this*".

8.11. Depuración de métodos: cómo seguir la ejecución paso a paso

Depurar (o *debuggear*) es el proceso de **detectar, analizar y corregir errores en un programa**. Durante la depuración, el programador examina paso a paso cómo se ejecuta el código, observa el valor de las variables y el flujo de control, y localiza los puntos en los que el programa no se comporta como se esperaba.

Un **breakpoint** (o *punto de interrupción*) es una marca que el programador coloca en una línea específica del código para que la **ejecución del programa se detenga justo en ese punto** durante el proceso de depuración. Esto permite observar el estado del programa en ese momento, examinar el valor de las variables y analizar el flujo de ejecución paso a paso.

Vamos a ver un ejercicio sencillo para aprender a depurar el código en Eclipse. Nos piden que sumemos dos números introducidos por el usuario. El código es el siguiente:

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Introduce el primer número: ");
       int num1 = sc.nextInt();
       System.out.print("Introduce el segundo número: ");
       int num2 = sc.nextInt();
       int suma = num1 + num1;
       System.out.print("El resultado de la suma es: " + suma);
    }
```

```
}
```

Este código no tiene errores de compilación, sin embargo, el resultado no es el esperado. No está sumando bien. Tenemos que encontrar el fallo. Vamos a depurar el código. Para ello:

1) Haz doble clic en la columna izquierda del editor para colocar los puntos de ruptura (breakpoints) que consideres (ver ilustración 8.1).

```
1  import java.util.Scanner;
2
3  public class depurarCodigo {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          System.out.print("Introduce el primer número: ");
7          int num1 = sc.nextInt();
8          System.out.print("Introduce el segundo número: ");
9          int num2 = sc.nextInt();
10         int suma = num1 + num1;
11         System.out.print("El resultado de la suma es: " + suma);
12     }
13 }
```

Breakpoint

Ilustración 8.1. Breakpoint. Elaboración propia.

2) Ejecuta el código pulsando el botón de “debug” (ver ilustración 8.2)

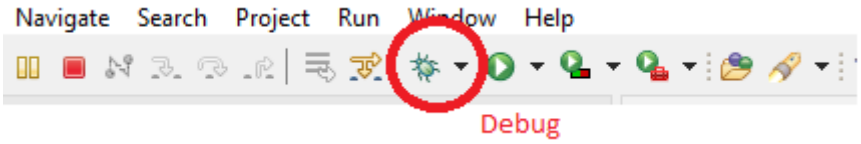

Ilustración 8.2. Botón “debug”. Elaboración propia.

3) Observa cómo la ejecución se detiene en el primer punto de ruptura (breakpoint) y se despliega una ventana llamada “Variables” en la columna de la derecha del editor (ver ilustración 8.3)

(x)= Variables | Breakpoints | Expressions

Name	Value
no method return value	
args	String[0] (id=20)
sc	Scanner (id=21)
num1	4
num2	5

Ilustración 8.3. Ventana “Variables”. Elaboración propia.

4) Usa los botones “Step Into” y “Step Over” para avanzar en el código (ver ilustración 8.4).

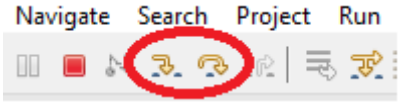

Ilustración 8.4. Botones “Step Into” y “Step Over”. Elaboración propia.

5) Cuando depurando llegamos al final del programa, vemos que se han insertado los números 4 y 5 y, sin embargo, el resultado de la suma es 8. Parece evidente dónde nos hemos equivocado… (ver ilustración 8.5)

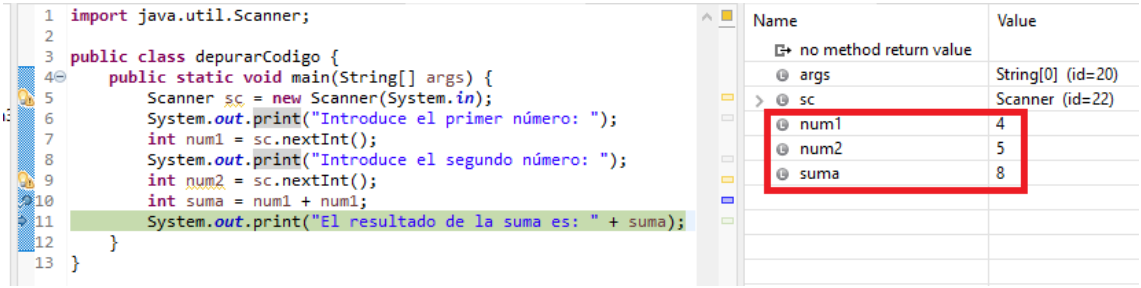

Ilustración 8.5. Valor de las variables. Elaboración propia.

Ejercicio 8.30. Copia y pega el siguiente código en Eclipse:

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Introduce una palabra: ");
       String palabra = sc.next();
       if (palabra == "") {
              System.out.print("Es un intro");
       }
       else {
              System.out.print("NO es un intro");
       }
    }
}
```

Como ves, no hay errores de compilación, pero hay algo que no está bien. Depura el código hasta que detectes el fallo.

Ejercicio 8.31. Copia y pega el siguiente código en Eclipse:

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Introduce una palabra: ");
       String palabra = sc.next();
       System.out.print("La última letra es: " +
extraerUltimaLetra(palabra));
    }
    public static String extraerUltimaLetra(String palabra) {
```

```
        return palabra.substring(palabra.length());
    }
}
```

Depura el código. ¿Dónde está el error?

Además de la opción de depurar, tienes otra gran ayuda. Si sitúas el cursor encima de un método, aparecerá un cuadro de diálogo llamado "*Intellisense*" donde podrás ver mucha información relacionada con el método. Puede que al depurar encuentres la línea donde se encuentra el fallo, pero no sepas cuál es exactamente. En estos casos el *intellisense* te puede dar pistas (ver ilustración 8.6)

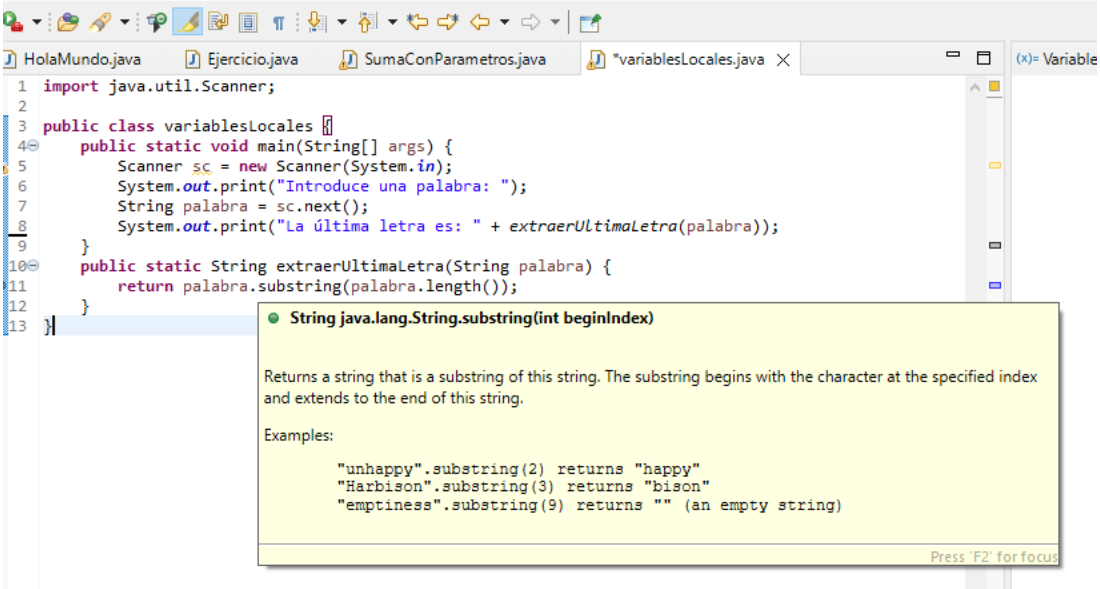

Ilustración 8.6. "*Intellisense*" desplegado al situar el cursor sobre el método "*substring*". Elaboración propia.

8.12. Ejercicios de repaso

Ejercicio 8.32. Crea una clase CuentaBancaria con los siguientes atributos **privados**:

- saldo (double)
- titular (String)

Implementa los siguientes métodos **públicos**:

1. depositar(double cantidad): Aumenta el saldo en la cantidad especificada.
2. retirar(double cantidad): Disminuye el saldo solo si hay fondos suficientes.
3. getSaldo(): Retorna el saldo actual.
4. getTitular() y setTitular(String titular): Para obtener y modificar el titular de la cuenta.

Prueba desde la clase main todos los métodos de la clase CuentaBancaria.

Ejercicio 8.33. Crea una clase Estudiante que tenga los siguientes atributos **privados**:

- nombre (String)
- edad (int)
- calificacion (double)

Implementa los siguientes métodos **públicos**:

1. getNombre() y setNombre(String nombre): Para obtener y modificar el nombre.
2. getEdad() y setEdad(int edad): La edad solo puede ser mayor o igual a 0.
3. getCalificacion() y setCalificacion(double calificacion): Debe estar entre **0 y 10**.
4. mostrarInfo(): Imprime los datos del estudiante.

Prueba desde la clase main todos los métodos de la clase Estudiante.

Ejercicio 8.34. Crea una clase base Figura con un método dibujar() que imprima "Dibujando una figura genérica".

Crea dos subclases:

- Circulo, que sobrescriba dibujar() e imprima "Dibujando un círculo".
- Cuadrado, que sobrescriba dibujar() e imprima "Dibujando un cuadrado".

En main(), crea un objeto de Circulo y otro de Cuadrado, y llama a sus métodos dibujar() individualmente.

Ejercicio 8.35. Crea una clase base Empleado con un método calcularSalario() que imprima "Salario base de un empleado".

Crea dos subclases:

- EmpleadoTiempoCompleto, que sobrescriba calcularSalario() e imprima "Salario de tiempo completo: 1000 €".
- EmpleadoMedioTiempo, que sobrescriba calcularSalario() e imprima "Salario de medio tiempo: 500 €".

En main(), crea un objeto de EmpleadoTiempoCompleto y otro de EmpleadoMedioTiempo, y llama a sus métodos calcularSalario() individualmente.

Ejercicio 8.36. Crea un sistema para gestionar productos en una tienda, utilizando modificadores de acceso. Para ello:

1. Crea una clase base Producto que tenga los siguientes atributos:
    - nombre (String) **private**

- precio (double) **private**
- cantidad (int) **protected**
- codigo (String) **default (sin modificador)**

Métodos en Producto:

- Un método **public** mostrarInfo() que imprima el nombre y el precio del producto.
- Un método **private** calcularDescuento() que calcule un 5% de descuento sobre el precio.

2. Crea una subclase ProductoElectronico que herede de Producto y tenga:
   - Un atributo adicional marca (String) **private**.
   - Un método **public** mostrarInfoElectronico() que imprima la marca y el código del producto.
3. Crea una clase Tienda con:
   - Un atributo **default** totalProductos (int) que llevará el conteo de productos en la tienda.
   - Un método **public** agregarProducto(Producto producto) que aumente el número de productos en la tienda.
   - Un método **protected** mostrarTotalProductos() que imprima la cantidad de productos en la tienda.
4. En main(), crea instancias de Producto y ProductoElectronico, usa los métodos set para asignar valores, y muestra la información.

Ejercicio 8.37. Crea un sistema para gestionar cuentas bancarias, utilizando modificadores de acceso. Para ello:

1. Crea una clase base CuentaBancaria que tenga como atributos:
   - numeroCuenta (String) **private**
   - saldo (double) **private**
   - titular (String) **protected**
   - tipoCuenta (String) **default (sin modificador)**

   Métodos en CuentaBancaria:

   - Un método **public** mostrarInformacion() que imprima el titular, el número de cuenta y el saldo.
   - Un método **private** calcularIntereses() que calcule un 2% de intereses sobre el saldo y lo imprima.
2. Crea una subclase CuentaAhorro que herede de CuentaBancaria y tenga:

- Un atributo adicional intereses (double) **private**.
- Un método **public** mostrarInformacionCuentaAhorro() que imprima el tipo de cuenta y los intereses.

3. Crea una clase Banco con:
   - Un atributo **default** totalCuentas (int) que llevará el conteo de cuentas en el banco.
   - Un método **public** agregarCuenta(CuentaBancaria cuenta) que aumente el número de cuentas en el banco.
   - Un método **protected** mostrarTotalCuentas() que imprima la cantidad de cuentas en el banco.
4. En main(), crea instancias de CuentaBancaria y CuentaAhorro, usa los métodos set para asignar valores, y muestra la información.

Ejercicio 8.38. Vamos a gestionar los productos de una tienda. Para ello, vamos a crear una clase **Producto** que tenga los atributos: nombre, precio y cantidad. Usaremos **constructores** para inicializar estos atributos al crear instancias de la clase. Además, crearemos una subclase **ProductoElectronico** que herede de **Producto** y agregue un atributo adicional para la **marca**. También, incluiremos un método para mostrar la información del producto. Así:

1. **Producto**:
   - Atributos:
     - nombre (String)
     - precio (double)
     - cantidad (int)
   - Constructor que reciba nombre, precio y cantidad como parámetros y los inicialice.
   - Método **mostrarInformacion()** que imprima los detalles del producto.
2. **ProductoElectronico** (subclase de Producto):
   - Atributo adicional: marca (String)
   - Constructor que reciba nombre, precio, cantidad, y marca como parámetros.
   - Método **mostrarInformacionElectronico()** que imprima los detalles del producto electrónico.
3. **Tienda**:
   - Atributo: productos (un arreglo de tipo Producto de tamaño fijo).
   - Método **agregarProducto()** que reciba un objeto de tipo Producto y lo agregue al arreglo.

- Método **mostrarProductos()** que imprima todos los productos de la tienda.

4. En el main(), crea instancias de Producto y ProductoElectronico usando los constructores, agrega los productos a la tienda, y muestra la información.

Ejercicio 8.39. Vamos a gestionar una biblioteca de películas. Para ello vamos a crear una clase **Pelicula** que tenga los atributos: titulo, duracion (en minutos), y anioEstreno. Luego, se creará una subclase **PeliculaDeAccion** que herede de **Pelicula** y tenga un atributo adicional para la **clasificacion**. Usaremos constructores para inicializar los atributos.

1. **Pelicula**:
   - Atributos:
     - tituloPelicula (String)
     - duracionPelicula (int)
     - anioEstrenoPelicula (int)
   - Constructor que reciba titulo, duracion y anioEstreno como parámetros y los inicialice.
   - Método **mostrarInformacion()** que imprima los detalles de la película.
2. **PeliculaDeAccion** (subclase de Pelicula):
   - Atributo adicional: clasificacionPelicula (String)
   - Constructor que reciba titulo, duracion, anioEstreno y clasificacion como parámetros.
   - Método **mostrarInformacionAccion()** que imprima los detalles de la película de acción.
3. **Biblioteca**:
   - Atributo: peliculas (un arreglo de tipo Pelicula de tamaño fijo).
   - Método **agregarPelicula()** que reciba un objeto de tipo Pelicula y lo agregue al arreglo.
   - Método **mostrarPeliculas()** que imprima todos los detalles de las películas.
4. En el main(), crea instancias de Pelicula y PeliculaDeAccion usando los constructores, agrega las películas a la biblioteca y muestra la información de todas las películas.

Ejercicio 8.40. Implementa un sistema para gestionar una tienda de vehículos. Cada vehículo tiene una marca y una descripción específica según su tipo. Así:

Clases a implementar:

- Vehiculo (Clase base)
- Coche (Subclase de Vehiculo)

- Moto (Subclase de Vehiculo)
- Bicicleta (Subclase de Vehiculo)

Polimorfismo:

- Todas las subclases deben sobrescribir el método obtenerDescripcion() para devolver información específica de cada tipo de vehículo.

Sobrecarga:

- La clase Coche debe tener una versión adicional de obtenerDescripcion() que reciba un parámetro indicando el tipo de combustible.

Pruebas:

- Crear objetos de cada clase (Coche, Moto, Bicicleta).
- Llamar al método obtenerDescripcion() en cada objeto.
- En el caso de Coche, llamar también a la versión sobrecargada con el tipo de combustible.

Ejemplo de salida esperada:

```
Coche de la marca Toyota con 4 puertas
Moto de la marca Honda con casco incluido
Bicicleta de la marca Giant con 21 cambios
Coche de la marca Toyota con 4 puertas, usa Gasolina
```

Ejercicio 8.41. Implementa un sistema para gestionar una empresa. Cada empleado tiene un nombre y un salario base, pero dependiendo del tipo de empleado, el cálculo del salario varía.

Clases a implementar:

- Empleado (Clase base)
- Gerente (Subclase de Empleado)
- Ingeniero (Subclase de Empleado)
- Asistente (Subclase de Empleado)

Polimorfismo:

Cada tipo de empleado debe sobrescribir el método calcularSalario(), aplicando su propia lógica de cálculo:

- **Gerente:** Recibe un bono adicional.
- **Ingeniero:** Tiene un extra por certificaciones.
- **Asistente:** No tiene extras, solo el salario base.

Sobrecarga:

La clase Ingeniero debe tener una versión sobrecargada de calcularSalario() que reciba el número de certificaciones y aumente el salario basándose en ellas.

Pruebas:

- Crear objetos de cada clase (Gerente, Ingeniero, Asistente).
- Llamar al método calcularSalario() en cada objeto.
- Para Ingeniero, llamar también a la versión sobrecargada con el número de certificaciones.

Ejemplo de salida esperada:

```
Salario del Gerente: 7500
Salario del Ingeniero: 5000
Salario del Asistente: 3000
Salario del Ingeniero con 3 certificaciones: 5300
```

Ejercicio 8.42. Desarrolla un sistema que modele distintos tipos de dispositivos inteligentes con diferentes niveles de herencia y constructores encadenados para analizar el **orden de ejecución de los constructores** cuando se crean objetos de la subclase más específica.

Clases a implementar:

- Dispositivo (Clase base)
- DispositivoInteligente (Subclase de Dispositivo)
- Telefono (Subclase de DispositivoInteligente)
- Smartphone (Subclase de Telefono)

Inicialización de Atributos:

- Cada clase debe tener atributos propios, como **nombre**, **conectividad**, **sistema operativo**, **cámara**, etc.
- Cada constructor debe inicializar estos atributos e imprimir un mensaje que indique su ejecución.

Crea un objeto de cada clase y observa el orden en que se ejecutan los constructores y cómo se inicializan los atributos.

Ejercicio 8.43. Desarrolla un programa que administre una flota de vehículos utilizando **arrays de objetos**. El sistema permitirá almacenar información de varios vehículos y mostrará un resumen de todos los vehículos registrados. Para ello:

- Implementa las clases Vehiculo (Clase base con atributos generales), Automovil y Camion (Subclases de Vehiculo con atributos específicos).
- Crea un array de objetos para almacenar múltiples vehículos. El array debe permitir agregar tanto automóviles como camiones.
- Implementa un método para agregar vehículos al array. Y otro método para mostrar todos los vehículos registrados.
- Para probar el sistema, crea un array de 5 vehículos y muestra la información en la consola.

Ejercicio 8.44. Desarrolla un programa que administre un **inventario de dispositivos electrónicos** utilizando **arrays de objetos**. El sistema permitirá almacenar información de varios dispositivos y mostrará un resumen de todos los dispositivos registrados. Para ello:

- Implementa las clases Dispositivo (Clase base con atributos generales), Smartphone y Laptop (Subclases de Dispositivo con atributos específicos).
- Crea un array de objetos para almacenar múltiples dispositivos. El array debe permitir agregar tanto smartphones como laptops.
- Implementa un método para agregar dispositivos al array y otro método para mostrar todos los dispositivos registrados.
- Para probar el sistema crea un array de 5 dispositivos (combinando smartphones y laptops) y muestra la información de los dispositivos en la consola.

Ejercicio 8.45. Crea un sistema que maneje diferentes tipos de vehículos usando herencia y sobreescritura de métodos:

1. Crea una clase base Vehiculo con:
    - Atributos marca y modelo.
    - Un constructor que reciba la marca y el modelo.
    - Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
    - Un método mostrarInfo() que imprima la información del vehículo.
2. Crea una clase Coche que extienda Vehiculo con:
    - Añade el atributo puertas (número de puertas).
    - Un constructor que reciba la marca, el modelo y las puertas, usando super para inicializar los atributos de la clase base.
    - Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
    - Sobrescribe (@Override) el método mostrarInfo() para incluir el número de puertas.
3. Crea una clase Moto que extienda Vehiculo con:
    - Añade el atributo cilindrada.
    - Un constructor que reciba la marca, el modelo y la cilindrada, usando super.
    - Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
    - Sobrescribe mostrarInfo() para incluir la cilindrada.
4. En la función main, crea un array de 4 Vehiculos con instancias de Coche y Moto. Luego llama a mostrarInfo() con cada objeto.

Ejercicio 8.46. Crea un sistema de empleados con diferentes tipos (Gerente y Desarrollador). Cada uno tiene atributos específicos y un método calcularSalario(), que será sobrescrito en cada clase derivada.

1. **Crea la clase base Empleado** con:
    - Atributos nombre, salarioBase y id.
    - Un constructor que use this() para asignar valores por defecto.
    - Un método calcularSalario() que devuelva salarioBase.
    - Un método mostrarInfo() que muestre el nombre y salario.
2. **Crea la clase Gerente que extienda Empleado** con:
    - Atributo bonoGerencial.
    - Un constructor que use super() para llamar al constructor de Empleado.
    - Sobrescribe calcularSalario() para sumar bonoGerencial.
3. **Crea la clase Desarrollador que extienda Empleado** con:
    - Atributo horasExtras y pagoPorHoraExtra.
    - Un constructor que use super().
    - Sobrescribe calcularSalario() para sumar el pago por horas extras.
4. En la función main, crea un array de 4 Empleados con instancias de Gerente y Desarrollador. Luego llama a mostrarInfo() con cada objeto.

Ejercicio 8.47. Copia y pega el siguiente código en Eclipse y ejecútalo. El programa **compila sin errores**, pero **lanza una excepción** si introduces una palabra. Usa el depurador para detectar el fallo.

```
import java.util.Scanner;

public class depurarCodigo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Introduce una palabra: ");
      String palabra = sc.next();
      System.out.print("La primera letra es: " + obtenerPrimeraLetra(palabra));
   }

   public static String obtenerPrimeraLetra(String texto) {
      return texto.substring(0, 0);
   }
}
```

TEMA 9. DIAGRAMAS DE CLASE

El diagrama de clases probablemente sea el diagrama más común en cualquier proceso de desarrollo software. Permite planificar nuestro proyecto describiendo las clases de las que se compone el sistema que vamos a desarrollar y las relaciones existentes entre ellas. Así, es muy recomendable tener un diagrama de clases antes de empezar a escribir código.

El diagrama lo dibujaremos siguiendo las pautas del lenguaje UML (Unified Modeling Language –Lenguaje de modelado unificado-). Se trata de símbolos que usaremos para representar las clases con sus atributos y métodos y las relaciones entre ellas. Estos símbolos están estandarizados para que todos los desarrolladores puedan entender su significado. Hay varias herramientas para crear diagramas UML que puedes encontrar por internet (draw.io, gliffy, lucidchart...). Utiliza la que más te guste.

Pero ¿por qué hay que dividir un proyecto en clases? Porque cuando tenemos que desarrollar un proyecto, es conveniente descomponerlo para poder repartir tareas fácilmente entre el equipo de trabajo. Además, de esta forma, es más sencillo localizar errores en el código.

Pero ¿cómo puedo saber cuántas clases va a tener mi programa? Después de haber establecido cuáles serán los requisitos del sistema o de nuestro juego, podremos hacer una **descripción** general del programa que tenemos que implementar. Esta descripción nos ayudará a saber el número de clases que tendremos que desarrollar. Para ello, señalaremos los nombres y los verbos. Los nombres pasarán a ser objetos (clases) y los verbos serán sus métodos (acciones).

Vamos a ver un ejemplo sencillo para aclarar conceptos. Tenemos que desarrollar una tienda de artículos online. Después de establecer los requisitos del sistema, podemos hacer la siguiente descripción: “un **vendedor** quiere vender un **artículo** y un **comprador** lo puede adquirir a través de un **pago**. Después se realizará el **envío**.”

De momento, solo subrayamos los nombres de la descripción para esclarecer cuáles van a ser los objetos que vamos a necesitar (las clases).

Para representar una clase en lenguaje UML solo hay que dibujar un rectángulo con 3 estancias. La de arriba será para escribir el nombre de la clase; la del medio será para los atributos de la clase; y la de abajo será para los métodos (acciones) que desarrollará el objeto.

Vamos a representar la primera clase: Articulo. Piensa, ¿cuáles son los atributos de un artículo? ¿Qué debe tener cualquier artículo? ¿Qué información esperarías que pusiese en la web? A modo de ejemplo, pondremos como atributos el nombre y el precio del artículo. Del mismo modo, vamos a pensar en el resto de los atributos que tendrían las demás clases.

La clase “Articulo”, sin sus métodos, quedaría así (ver ilustración 9.1):

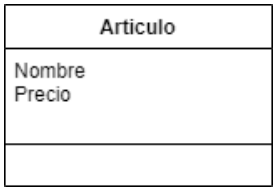

Ilustración 9.1. La clase "artículo". Elaboración propia.

Ahora, vamos a representar las otras clases de nuestro sistema (ver ilustración 9.2):

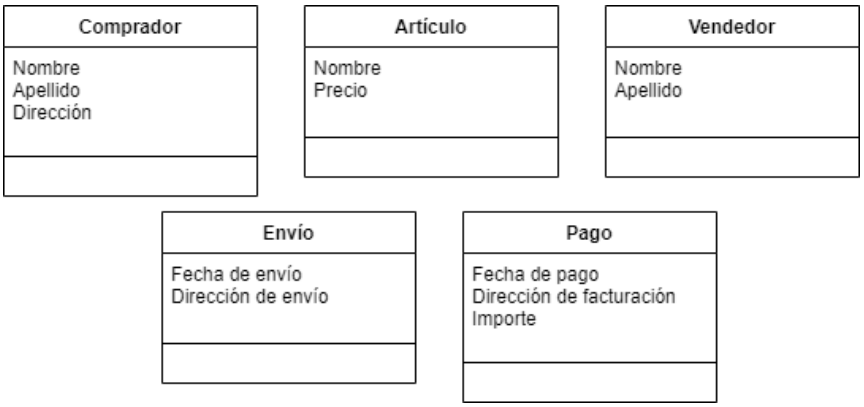

Ilustración 9.2. Clases extraídas de la descripción. Elaboración propia.

Ya tenemos representadas todas las clases que hemos extraído de la descripción que hicimos del sistema.

9.1. Multiplicidad o cardinalidad

Las clases que acabamos de representar se asocian entre sí, es decir, algunos objetos representados por esas clases tienen relaciones con otros objetos. La multiplicidad o cardinalidad establece cuántos posibles objetos intervienen en cada asociación, una para cada extremo de la relación. Además, para que quede todo mucho más claro, vamos a añadir qué acción se establece en cada asociación (ver ilustración 9.3).

En la tabla 1 puedes ver los distintos tipos de multiplicidad:

Multiplicidad	**Significado**
1	Uno y solo uno
0..1	Cero o uno
*	Cero o varios
0..*	Cero o varios
1..*	Uno o varios

Tabla 1. Tipos de multiplicidad. Elaboración propia.

Para nuestro ejemplo utilizaremos:

- Multiplicidad **de uno a varios**. Tienes que preguntarte, ¿cuántos artículos habrá en cada envío? Como mínimo, un envío tendrá un artículo, pero no podemos saber el número máximo. Esta relación se representa **1..*** en la línea que va a unir las clases artículos y envíos, cerca de la representación de la clase “articulo”.
- Multiplicidad **de varios a varios**. Del mismo modo, ¿cuántos artículos puede vender un vendedor? Puede que ninguno, es decir, el mínimo sería 0. En esta ocasión, tampoco podemos saber el máximo número de artículos que se pueden vender. Esta relación se representa escribiendo **0..*** en el lado de la clase “articulo”. Otra cuestión es, ¿cuántos vendedores pueden vender un mismo artículo? Si consideramos que varios (depende si los artículos son únicos o si varias personas pueden venderlos. Esta cuestión tendría que quedar clara en la fase de la toma de requisitos) representaremos esta relación escribiendo de nuevo 0..* pero esta vez en el lado de la clase “vendedor”.
- Multiplicidad **de uno a uno**. Piensa en la relación entre la clase “envio” y la clase “pago”. Parece claro que a cada envío le corresponde un único pago y viceversa.

En la ilustración 3 se puede apreciar el diagrama completo.

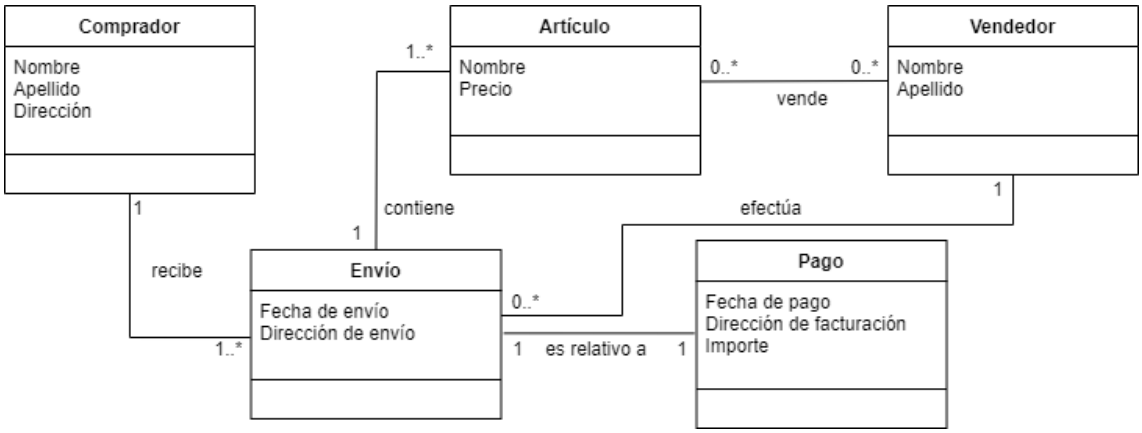

Ilustración 9.3. Diagrama de clases con las cardinalidades de las relaciones. Elaboración propia.

Como puedes observar, no todas las clases (entidades) tienen relaciones directas con las otras. No tienen por qué tenerlas. Es decir, si un objeto no se prevé que se relacione con otro, no habrá "comunicación" entre ambos.

<u>Pero ojo, ¡éste solo es uno de los muchos modelados posibles del sistema!</u> No hay una única solución al problema del diseño de nuestro programa. Depende de lo que se establezca en la toma de requisitos del sistema, el diseño final será más o menos parecido a este. Eso sí, no olvides que UML es un lenguaje estandarizado, es decir, el significado de cada uno de sus símbolos (cajas, líneas, relaciones...) es específico y, por lo tanto, inmutable.

9.2. Tipos de relaciones entre clases

Las uniones entre clases nos indican cómo se relacionan los objetos que estas representan entre sí. Estas relaciones se pueden clasificar en tres tipos:

- Herencia
- Asociación
    - Bidireccional
    - Unidireccional
    - Involutiva
    - Agregación
    - Composición
- Dependencia

9.2.1. Herencia

Siguiendo nuestro ejemplo, piensa que los vendedores y los compradores son usuarios. Podemos crear una clase "usuario" de la que hereden ambas clases (ver ilustración 9.4). Así, escribiremos en esta nueva clase los atributos comunes a todos los usuarios, como sus nombres y apellidos. ¿Y esto para qué sirve? Bueno, para ahorrarnos escribir más código. El código que esté en la clase "usuario" te servirá para cualquier clase derivada, es decir, no tendrás que volver a escribirlo.

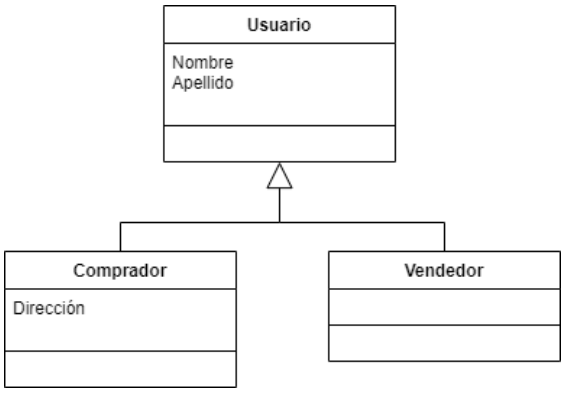

Ilustración 9.4. Ejemplo de herencia. Elaboración propia.

9.2.2. Asociación

Una asociación describe una relación entre objetos. La asociación en UML se representa con una línea continua que une las clases que se pretende relacionar. Así (ver ilustración 9.5):

Ilustración 9.5. Asociación entre clases. Elaboración propia.

Este tipo de asociaciones se dice que son **bidireccionales**, es decir, funcionan en ambos sentidos, la clase A se relaciona con B y viceversa. Son las asociaciones más habituales, pero en ocasiones se pueden ver asociaciones en un único sentido, lo que llamaremos **unidireccional**. En este caso, en su representación UML, la línea termina en una punta de flecha que indica el sentido de la asociación (ver ilustración 9.6).

Ilustración 9.6. Asociación unidireccional (izquierda) y bidireccional (derecha). Elaboración propia.

Cuando una clase aparece en los dos extremos de la asociación, se dice que es una relación **involutiva** (ver ilustración 9.7).

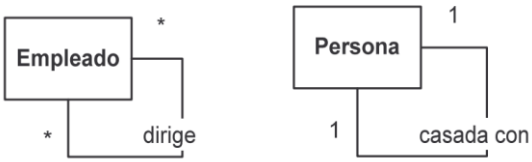

Ilustración 9.7. Relación involutiva. Elaboración propia.

Cuando los objetos de una clase se componen de objetos de otra clase se llama **relación de composición**. Piensa, por ejemplo, en la clase “Equipo” y la clase “Jugadores” (el equipo está compuesto de jugadores), o la clase “Parking” y la clase “Aparcamiento” (el parking está

compuesto de aparcamientos). Esta relación de composición, en lenguaje UML, se representa con un diamante negro del lado del contenedor (ver ilustración 9.8).

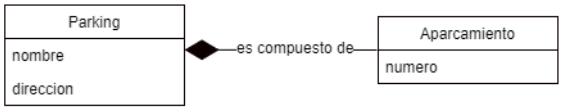

Ilustración 9.8. Ejemplo de una relación de composición. Elaboración propia.

Si los objetos de una clase pueden ser componentes de otra clase, se trata de una relación de **agregación**. Imagina la clase "Avión" y la clase "Pasajero", o la clase "Parking" y la clase "Coche" (el parking puede estar compuesto de coches, pero un coche puede no tener nada que ver con un parking). Esta relación de agregación, en lenguaje UML, se representa con un diamante blanco del lado del contenedor (ver ilustración 9.9).

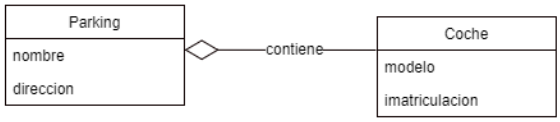

Ilustración 9.9. Ejemplo de una relación de agregación. Elaboración propia.

9.2.3. Dependencias

Una asociación de dependencia muestra la relación entre dos objetos: un objeto que solicita un servicio y otro objeto que provee el servicio solicitado.

La dependencia, en lenguaje UML, se representa como una línea discontinua con una punta de flecha (ver ilustración 9.10). Por ejemplo, para resolver una operación matemática compleja (raíz cuadrada), tendremos que invocar alguna función de la clase Math (en este caso la función sqrt()).

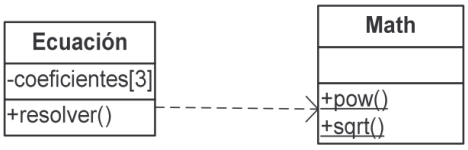

Ilustración 9.10. Relación de dependencia. Elaboración propia.

9.3. Afianzando conceptos

Un cliente nos ha pedido crear una app para Android que gestione el alquiler de bicicletas por la ciudad. Después de la toma de requisitos, podemos llegar a una descripción del sistema. En esta ocasión también vamos a señalar las acciones que tendrían cada uno de nuestros objetos.

Descripción: "El usuario tendrá que registrarse en la aplicación móvil y pagar una suscripción mensual. Una vez se haya confirmado el registro, todas las opciones de la app estarán disponibles para el usuario. Las bicicletas tendrán GPS para localizarlas en un mapa. De esta forma, el usuario puede saber dónde se encuentra la bici más cercana. Además, las bicicletas tienen que llevar un candado que se bloquee o desbloquee con la app. Cuando el usuario esté cerca de la bicicleta, desbloquea el candado con la app y la puede usar. Al terminar su uso, el usuario tiene que bloquear el candado de la bicicleta y confirmar su nueva ubicación."

Según esta descripción del sistema, podemos hacer un diagrama de clases como el siguiente (ver ilustración 9.11):

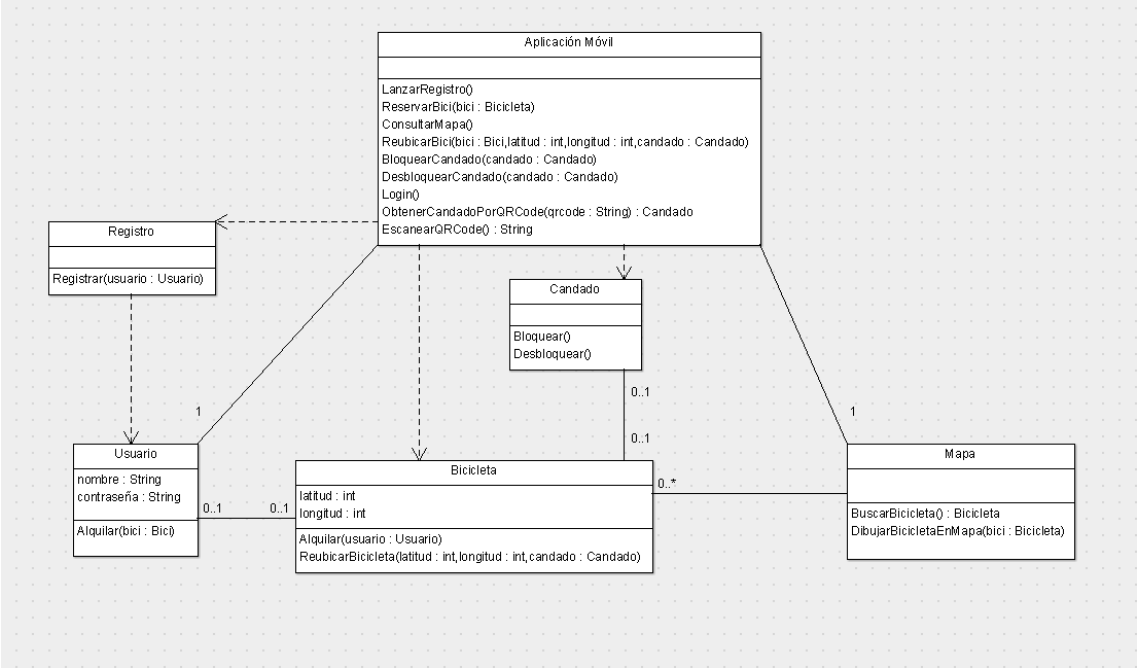

Ilustración 9.11. Diagrama de clases del ejercicio propuesto. Elaboración propia.

Fíjate que hay relaciones de dependencia entre la clase “Aplicación móvil” y las clases “Registro”, “Candado” y “Bicicleta”. También hay una relación de dependencia entre las clases “Registro” y “Usuario”. El resto de las asociaciones son bidireccionales.

Observa también cómo representamos las clases con cajetines con 3 compartimentos: uno para el nombre de la clase, otro para los atributos (aunque no haya, como en la clase “Registro”) y otro para los métodos de la clase (sus acciones).

Mira cómo hay invocaciones (llamadas) en métodos en los que directamente se ponen los parámetros que se van a enviar (p.ej: ReubicarBicicleta (latitud:int, longitud:int, candado:Candado).

Te recuerdo que este diagrama **no es una solución única**. Puedes haber pensado en otra alternativa al sistema perfectamente válida. Solo depende del punto de vista del diseñador.

9.4. Ejercicios de repaso

Ejercicio 9.1. Una biblioteca necesita un sistema para gestionar los libros y los préstamos a los usuarios. El sistema debe permitir registrar libros, usuarios y manejar los préstamos de libros. Los requisitos son:

- Cada **Libro** tiene un título, un autor, un ISBN único y un estado de disponibilidad.
- Un **Usuario** tiene un nombre, un ID único, un correo electrónico y una lista de libros prestados.
- La **Biblioteca** debe gestionar un catálogo de libros y una lista de usuarios.
- Un usuario puede **pedir prestado** un libro si está disponible y **devolverlo** posteriormente.

- La biblioteca debe permitir **agregar** y **eliminar** libros de su catálogo.
- Debe existir una función para **buscar libros** por título.

Diseña el **diagrama de clases UML**.

Ejercicio 9.2. Se necesita modelar un sistema para gestionar las facultades, departamentos y profesores de una universidad. Los requisitos son:

- Una universidad está compuesta por múltiples facultades.
- Si la universidad deja de existir, todas sus facultades también desaparecen. (**Composición**).
- Cada facultad pertenece a una universidad y puede tener varios departamentos.
- Si una facultad se elimina, sus departamentos pueden seguir existiendo de manera independiente. (**Agregación**).
- Cada departamento pertenece a una facultad y tiene profesores.
- Un profesor solo puede pertenecer a un departamento.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.

Ejercicio 9.3. Se necesita modelar un sistema para gestionar los vehículos de una empresa de transporte. Los requisitos son:

- Todos los vehículos tienen un **número de serie**, **marca** y **modelo**.
- Existen diferentes tipos de vehículos: **Coche**, **Moto** y **Camión**, que heredan de **Vehículo**.
- **Coche**: Tiene un atributo adicional **cantidad de puertas**.
- **Moto**: Tiene un atributo **cilindrada**.
- **Camión**: Tiene un atributo **capacidad de carga**.
- Cada vehículo puede estar asignado a **un Conductor**.
- Un **Conductor** tiene un **nombre** y un **número de licencia**.
- La relación entre conductor y vehículo es **unidireccional**.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.

Ejercicio 9.4. Se necesita modelar un sistema donde los empleados pueden enviar documentos a una impresora para ser impresos. Los requisitos son:

- **Empleado**: Representa a una persona que trabaja en la oficina y necesita imprimir documentos.
- **Documento**: Representa un archivo que el empleado quiere imprimir.
- **Impresora**: Se encarga de imprimir los documentos enviados.
- Un **Empleado depende** de la clase **Impresora** porque necesita una impresora para imprimir.
- Un **Documento depende** de la clase **Empleado**, ya que sin un empleado no hay documentos que imprimir.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.

TEMA 10. ESTRUCTURAS DINÁMICAS DE DATOS

Hasta ahora hemos trabajado con variables y arrays estáticos, es decir, con datos que tienen un tamaño fijo desde que se declaran. Por ejemplo, un int siempre ocupa lo mismo en memoria, y un array como int[3] tiene un tamaño que no cambia durante la ejecución.

Este enfoque es fácil de entender y eficiente cuando manejamos datos cuyo tamaño no varía, pero se vuelve limitado y poco práctico cuando necesitamos estructuras que puedan crecer o reducirse dinámicamente, como listas de elementos cuyo número no conocemos de antemano.

Por ejemplo, si queremos crear una agenda, necesitaremos ir añadiendo nuevos datos. Si reservamos espacio para un máximo de 10 personas, no podremos llegar a añadir la número 11. Una solución podría ser sobredimensionar, es decir, preparar una agenda contando con 1000 personas, aunque luego nos quedemos en 200 contactos. Esto tiene varios inconvenientes: se desperdicia memoria, obliga a conocer bien los datos con los que vamos a trabajar, sigue pudiendo verse sobrepasado, etc.

Para solucionar este tipo de problemas se crearon las **estructuras dinámicas.** Estas estructuras se agrupan bajo el nombre de ***colecciones***. Una colección, como su nombre indica, es simplemente un grupo de elementos. En Java podemos encontrar varias colecciones en el paquete **java.util**, como ArrayList, LinkedList, HashMap, entre muchas otras. Cada una tiene su propia forma de funcionar y está pensada para situaciones distintas.

Trabajar con colecciones es muy útil porque nos ahorra tiempo y nos da herramientas para organizar y manipular datos de forma flexible.

Además, existen otras estructuras dinámicas más complejas que nosotros **no** trataremos aquí, como los conjuntos (Set, que permiten almacenar elementos sin repetición), las colas de prioridad (que ordenan los elementos según su importancia), árboles más especializados (que se usan para búsquedas rápidas o bases de datos), grafos (para modelar relaciones complejas) y muchas más.

Estas estructuras avanzadas suelen estudiarse en cursos o libros de nivel avanzado, y su conocimiento abre la puerta a áreas como la optimización, la inteligencia artificial o el desarrollo de software más eficiente.

10.1. No genéricos vs genéricos

Cuando hablamos de colecciones o estructuras dinámicas, podemos distinguir entre dos grandes tipos: las **estructuras no genéricas** y las **genéricas**.

Las **estructuras no genéricas** son aquellas que no especifican ningún tipo de dato al declararlas, por ejemplo, List lista = new ArrayList();. En este caso, la estructura puede contener cualquier tipo de dato, ya que Java las trata como si guardaran objetos del tipo más general: Object. Esto puede parecer más flexible, pero también **es más peligroso, porque no hay control en tiempo de compilación**. Puedes meter cualquier cosa, y cuando intentes usar los datos, tendrás que hacer conversiones manuales (casts). Si te equivocas, el programa puede lanzar excepciones en tiempo de ejecución, como ClassCastException.

Por otro lado, las **genéricas** son aquellas que especifican el tipo de dato que van a almacenar usando los símbolos < >. Por ejemplo, List<String> solo admite cadenas de texto. Esto ofrece

muchas ventajas: el compilador puede detectar errores antes de que el programa corra, no necesitas hacer conversiones manuales de tipos, y el código es más claro y seguro.

Es decir, usar genéricos es una forma de evitar problemas y hacer que tu código sea más robusto, mantenible y legible. **Siempre que puedas, usa estructuras genéricas**.

10.2. Pilas

Una de las estructuras dinámicas más conocidas son **las pilas**. Una pila (o *stack*, en inglés) es una colección que sigue el principio de **LIFO**, que significa *Last In, First Out* o, en otras palabras, el último que entra es el primero que sale. Piensa en una pila de platos donde solo puedes quitar o poner elementos desde la parte de arriba. No puedes sacar uno del medio sin que todo se desarme.

Las pilas se usan en situaciones como deshacer acciones en un editor de texto o realizar algunas operaciones matemáticas. Son muy útiles cuando necesitas llevar el control de lo que ha pasado y volver atrás paso a paso.

Podemos trabajar con pilas de varias formas. Una de las más directas es usando la clase Stack, que forma parte del paquete java.util.

Los métodos más usados de la clase Stack son:

- **push()** para apilar un elemento.
- **pop()** para desapilar (quitar el de arriba).
- **peek()** para mirar lo que hay en la cima sin quitarlo.
- **size()** que devuelve un número entero con la cantidad de elementos actuales en la pila.
- **isEmpty()** que devuelve *true* si la pila está vacía, o *false* en caso contrario.

Vamos a ver un ejemplo: creamos una pila, introducimos 5 números y luego los mostramos por pantalla:

```
import java.util.Scanner;
import java.util.Stack;

public class Pila {

	public static void main(String args[]) {
		Stack<Integer> pila = new Stack<>();

		Scanner in = new Scanner (System.in);

		for (int i=0; i<5; i++) {
			System.out.print("Introduce un número: ");
			int num = in.nextInt();
			pila.push(num);
		}
		int total = pila.size();
		for (int i=0; i<total; i++) {
			System.out.println("El contenido de la pila es: " +
pila.pop());
		}
```

```
        }
}
```

NOTA: El método size() devuelve el número actual de elementos que tiene una estructura dinámica (una pila en este ejemplo). Hay un detalle importante que debes tener en cuenta cuando usas size() dentro de un bucle. Cada vez que haces pop() estás quitando un elemento y la pila se hace más pequeña. Esto hace que el valor de size() cambie y, por lo tanto, el número de vueltas que dará el bucle, que va a ser menor de lo que esperas. Para solucionar este problema es recomendable saber con antelación la cantidad de elementos que hay en la estructura. Así:

```
int total = pila.size();
```

Ahora, este total, ya lo puedes utilizar dentro del bucle for y tendrás el número de iteraciones esperado.

Vamos a ver un ejemplo con estructura de pila no genérica para que veas los posibles problemas que puedes tener:

```
Stack pila = new Stack();
pila.push(3.14f);  // float
pila.push("hola");  // String

float x = pila.pop();  // Error de compilación
```

En este caso, pila.pop() devuelve un objeto *Object*. Se puede imprimir como texto gracias al método toString() de la clase, pero no es ni un float ni un String. Es decir, podríamos hacer esto sin problema:

```
System.out.println("El contenido de la pila es: " + pila.pop());
```

Pero, aunque **sí puedas imprimir los elementos sin hacer casting**, eso no significa que puedas **usarlos correctamente en operaciones sin hacer la conversión de tipos**.

RECUERDA: El uso de estructuras no genéricas obliga a hacer conversiones explícitas (casting), lo cual puede provocar errores en tiempo de ejecución, dificulta la legibilidad del código y elimina la comprobación de tipos en tiempo de compilación. Por este motivo, **siempre que sea posible se recomienda usar estructuras genéricas**.

Ejercicio 10.1. Crea un programa que pida al usuario 3 números reales y 3 cadenas de texto y lo guarde en una pila **no** genérica. Después muestra el contenido en orden contrario. ¿Has tenido algún problema? ¿Por qué?

Ejercicio 10.2. La "notación polaca inversa" es una forma de expresar operaciones que consiste en indicar los operandos antes del correspondiente operador. Por ejemplo, en vez de "3+4" se escribiría "3 4 +". Es una notación que no necesita paréntesis y que se puede resolver usando una pila: si se recibe un dato numérico, éste se guarda en la pila; si se recibe un operador, se obtienen los dos operandos que hay en la cima de la pila, se realiza la operación y se apila su

resultado. El proceso termina cuando sólo hay un dato en la pila. Por ejemplo, "3 4 +" se convierte en: apilar 3, apilar 4, sacar dos datos y sumarlos, guardar 7, terminado. Impleméntalo y comprueba si el resultado de "3 4 6 5 - + * 6 +" es 21.

10.3. Colas

Las **colas** son otra estructura dinámica muy común y fácil de entender. Una cola (o *queue* en inglés) funciona siguiendo el principio **FIFO**, que significa *First In, First Out*. Es decir, el primer elemento en entrar es el primero en salir. Imagina una fila en una tienda o en una parada de autobús: quien llega primero es atendido primero. Justo lo contrario que una pila.

Las colas son muy útiles cuando necesitas procesar elementos en el orden en que llegaron. Por ejemplo, una impresora que va sacando trabajos de impresión uno a uno, o un sistema que atiende solicitudes de usuarios según el orden de llegada. También se usan mucho en algoritmos de búsqueda y en programación concurrente.

En Java, no hay una clase llamada Queue que puedas usar directamente como se hace con Stack, pero sí existe la **interfaz** Queue (el concepto interfaz se verá con profundidad en un tema posterior), y una de las implementaciones más usadas es LinkedList, que permite trabajar con colas de forma muy sencilla. Así:

```
Queue<Integer> cola = new LinkedList();
```

Aquí usamos algunos métodos importantes:

- **add()** agrega un elemento al final de la cola.
- **peek()** mira el primer elemento sin quitarlo.
- **poll()** extrae y devuelve el primer elemento.
- **size()** indica cuántos elementos hay en la cola.
- **isEmpty()** devuelve *true* si la cola está vacía o *false* en caso contrario.

Aquí tienes un ejemplo donde se añaden números a una cola y luego se muestren por pantalla:

```
import java.util.LinkedList;
import java.util.Queue;

public class Cola {
    public static void main(String[] args) {
        Queue<Integer> cola = new LinkedList();
        cola.add(3);
        cola.add(14);
        cola.add(12);
        cola.add(7);
        cola.add(10);
        int total = cola.size();
        for (int i=0; i<total; i++) {
            System.out.println("La posición " + i + " de la cola es: " +
cola.poll());
        }
    }
}
```

Ejercicio 10.3. Crea un programa que pida al usuario 5 números reales de doble precisión, los guarde en una cola y luego los muestre en pantalla.

Ejercicio 10.4. Crea un programa que pida frases al usuario, hasta que introduzca una frase vacía. En ese momento, mostrará todas las frases que se habían introducido.

NOTA: Si escribes:

```
Queue<Double> cola = new LinkedList();  // Falta el parámetro de tipo <>
```

El compilador permite que funcione, pero **genera una advertencia de tipo "unchecked assignment"**, porque estás asignando una LinkedList sin tipo a una variable que espera una lista de Double.

Para evitar riesgos de tipo, se recomienda **usar también los genéricos en el constructor**. Así:

```
Queue<Double> cola = new LinkedList<Double>();
```

O, a partir de Java 7, puedes usar el **operador diamante (<>)** para que el compilador infiera el tipo:

```
Queue<Double> cola = new LinkedList<>();
```

10.4. Listas

Las listas son una de las estructuras dinámicas más útiles y comunes en Java porque te permiten almacenar elementos en orden, cambiar su tamaño mientras el programa está en ejecución y acceder fácilmente a cualquier posición. Son ideales cuando no sabes cuántos elementos vas a manejar o cuando necesitas modificar la colección sobre la marcha.

Para crear una lista necesitamos la interfaz **List**, que define un conjunto de métodos para trabajar con colecciones ordenadas. Esta interfaz especifica qué operaciones debe tener una lista (como añadir, eliminar, obtener elementos, etc.), pero no tiene implementación propia. Es decir, no puedes crear directamente una lista con new List() porque no existe una implementación real ahí. Para que esta lista funcione, debes usar una clase que implemente esa interfaz. Las implementaciones más comunes son:

- **ArrayList**, que usa un array interno que crece automáticamente. Ofrece acceso rápido a los elementos por índice y es eficiente para añadir elementos al final, pero insertar o borrar elementos en medio puede ser más lento porque hay que mover otros elementos.
- **LinkedList**, que está formada por nodos enlazados donde cada uno apunta al siguiente. Es más eficiente para insertar y eliminar elementos en cualquier posición, pero acceder a un elemento específico puede ser más lento porque hay que recorrer la lista.

Es decir, si vas a acceder mucho por posición, como en una lista de resultados, usa ArrayList. Sin embargo, si vas a insertar o eliminar elementos en lugares intermedios con frecuencia (como en un editor de texto o una cola de reproducción), LinkedList es mejor opción.

Este tipo de decisión es importante en programas reales donde el rendimiento importa, aunque para muchos ejercicios básicos ambas implementaciones funcionarán igual.

Por ejemplo, para declarar una lista de String con la implementación ArrayList, escribirías:

```
List<String> miLista = new ArrayList<>();
```

Fíjate en que la variable está declarada como List<String>, que es la interfaz, y la instancia se crea con new ArrayList<>(), que es la implementación concreta.

De esta forma puedes cambiar la implementación fácilmente sin modificar el resto del código. Por ejemplo, si quieres pasar a la implementación de LinkedList, solo cambiarías esto en el código:

```
List<String> miLista = new LinkedList<>();
```

Así, un ejemplo que pidiese 3 palabras al usuario para almacenarlas en una lista de Strings sería:

```
import java.util.List;
import java.util.ArrayList;
import java.util.Scanner;

public class EjemploLista {
    public static void main(String[] args) {
      List<String> palabras = new ArrayList<>();
      Scanner sc = new Scanner(System.in);
      for (int i=0; i<3; i++) {
            System.out.print("Introduce una palabra: ");
            palabras.add(sc.next());
      }
    }
}
```

Si declaras directamente:

```
ArrayList<String> miLista = new ArrayList<>();
```

tu variable queda atada a esa implementación, y cambiar a otra lista más adelante puede ser más complicado.

Una vez tienes la lista, puedes añadir o insertar cosas con el método add(). Por ejemplo, lista.add(String) o lista.add(posición, String). Si quieres saber cuántos elementos tiene la lista en un momento dado, usas size(), que te devuelve el número actual.

Si necesitas sacar algo, puedes pedir el elemento que está en una posición específica con get(indice). O cambiar lo que hay en esa posición con set(indice, nuevoValor). Y si ya no quieres un elemento, lo puedes eliminar con remove(indice).

¡OJO! Los métodos add() y set() tienen funciones muy diferentes y es importante saber cuándo usar cada uno:

- **add()** se usa para **insertar** elementos en la lista. Puedes añadir un nuevo elemento al final con add(elemento) o insertar en una posición específica con add(indice, elemento). Cuando usas add(), la lista crece porque estás **añadiendo un nuevo elemento**, y si lo haces en medio, **los demás elementos se desplazan para dejar espacio**.
- **set()**, en cambio, se usa para **reemplazar** un elemento que ya está en la lista. Es decir, no añade un nuevo elemento ni cambia el tamaño de la lista, simplemente cambia el valor que está en la posición indicada por el que tú le pases.

```
List<String> lista = new ArrayList<>();
lista.add("A");       // lista: ["A"]
lista.add("B");       // lista: ["A", "B"]
lista.add(1, "C");    // lista: ["A", "C", "B"] -> insertamos "C" en la posición 1

lista.set(1, "D");    // lista: ["A", "D", "B"] -> reemplazamos "C" por "D" en la posición 1
```

Para ordenar listas puedes usar el método sort(). Este método, desde Java 8, necesita siempre un argumento: un comparador (Comparator). Si quieres ordenar una lista con su **orden natural** (números de menor a mayor, cadenas en orden alfabético), puedes hacer:

```
miLista.sort(null); // ordena usando el orden natural
```

Si prefieres una sintaxis más clara o usas versiones anteriores a Java 8, puedes hacer:

```
Collections.sort(miLista); // también ordena por el orden natural
```

Ambas opciones hacen lo mismo, así que puedes elegir la que te resulte más cómoda o legible.

Si quieres invertir el orden de los elementos de una lista (de mayor a menor, o alfabético descendente), puedes usar el método reverse() de la clase Collections:

```
Collections.reverse(miLista);
```

Este método **no necesita parámetros adicionales** y modifica la lista directamente. Pero cuidado, a diferencia de sort(), **reverse() no ordena**, solo **invierte el orden actual** de la lista. Es decir, para ordenar de mayor a menor o alfabéticamente descendente tienes que hacer primero Sort() y luego revertir el orden.

La clase Collections está en el paquete java.util, que hay que importar al comienzo del programa. Mira este ejemplo para ordenar de mayor a menor:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EjemploOrdenDescendente {
   public static void main(String[] args) {
```

```
        List<Integer> miLista = new ArrayList<>();
        miLista.add(5);
        miLista.add(2);
        miLista.add(9);
        miLista.add(1);
        miLista.add(7);

        // Ordena de menor a mayor (orden natural)
        miLista.sort(null);

        // Invierte la lista para tener orden de mayor a menor
        Collections.reverse(miLista);

        System.out.println("Lista ordenada de mayor a menor: " + miLista);
    }
}
```

Ejercicio 10.5. Escribe un programa que:

1. Cree una lista para almacenar nombres (String).
2. Pida al usuario que introduzca 5 nombres y los añada a la lista.
3. Muestre todos los nombres almacenados.
4. Pida al usuario que introduzca un índice y muestre el nombre que está en esa posición (validando que el índice sea válido).
5. Pida al usuario un índice para eliminar el nombre en esa posición y lo elimine de la lista (validando el índice).
6. Finalmente, muestre la lista actualizada con los nombres que quedan.

10.5. Mapas

Imagina una agenda de contactos. Tienes nombres (las claves) y números de teléfono (los valores). Esto es justo lo que hace un **mapa** (Map), es decir, te permite guardar y buscar datos usando una **clave** para acceder a un **valor**.

En lugar de usar índices como en una lista, usas claves que pueden ser de casi cualquier tipo: String, Integer, Character, etc. Y lo mismo con los valores. Por ejemplo, puedes tener un mapa que asocie un número de matrícula con el nombre del dueño de un coche, o un mapa que guarde el precio de productos por nombre.

Al igual que las listas y las colas, los mapas en Java son una **interfaz**, lo que significa que no se pueden usar directamente. En su lugar, usamos clases que la implementan, como HashMap, que es la más común. Usamos genéricos para indicar el tipo de clave y el tipo de valor. Así:

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> edades = new HashMap<>();
```

Los métodos más usados en mapas son:

- **put(clave, valor)** añade un par clave–valor. Si la clave ya existe, reemplaza el valor.
- **get(clave)** devuelve el valor asociado a esa clave.
- **remove(clave)** elimina la entrada con esa clave.
- **replace(clave, nuevoValor)** reemplaza el valor asociado a la clave, si la clave ya existe.
- **containsKey(clave)** devuelve true si la clave está en el mapa.
- **size()** dice cuántos pares clave–valor hay.
- **isEmpty()** indica si el mapa está vacío.
- **putIfAbsent()** agrega un valor si la clave no existe. Por ejemplo:

```
map.put("A", 1);  // {"A": 1}
map.putIfAbsent("A", 2); // {"A": 1} (no cambia)
map.put("A", 3);  // {"A": 3} (sobrescribe)
```

Ejemplo:

```java
import java.util.HashMap;
import java.util.Map;

public class EjemploMap {
    public static void main(String[] args) {
      Map<String, Integer> map = new HashMap<>();

      map.put("valor1", 15);
      map.put("valor2", 20);
      map.put("valor3", 1000);
      map.put("valor4", 1500);
      map.put("valor5", 2);

      System.out.print(map.get("valor1"));
    }
}
```

NOTA: Cada **clave es única**. Si haces put() con una clave que ya existe, el valor anterior se reemplaza, pero **los valores sí pueden repetirse**.

10.5.1. Principales implementaciones de Map: HashMap, LinkedHashMap, TreeMap

En un **HashMap** no hay un orden garantizado en los elementos, es decir, los pares clave–valor no se almacenan ni se recorren en el mismo orden en que los agregaste. Esto ocurre porque internamente el HashMap **no usa índices ni listas**, sino algo diferente: una **tabla hash**, que es una estructura especial que organiza los datos usando una **función de dispersión** (o función hash). Esta función toma la clave y la convierte en un número, que sirve para decidir **en qué "casilla" interna se va a guardar el valor**. Java calcula el **hash** de la clave y decide en qué parte

del mapa colocar el valor. Este proceso no sigue un orden visible ni predecible para ti como programador, porque su objetivo no es ordenar, sino **ser muy rápido para buscar y guardar datos**. La principal ventaja de esta “ordenación” es que la velocidad de acceso es muy alta y usa muy poca memoria.

LinkedHashMap mantiene el orden de inserción. Es decir, si primero añades la clave "A" y luego la "B", al recorrer el mapa verás primero "A" y luego "B". Internamente, es como un HashMap con una lista enlazada que recuerda el orden.

En un **TreeMap**, el orden es alfabético o numérico (según el tipo de clave). Es ideal si necesitas que los elementos estén siempre ordenados por clave.

Mira la comparativa de la tabla 10.1 para más detalles.

Característica	HashMap	LinkedHashMap	TreeMap
Orden de los elementos	No garantizado	Orden de inserción	Orden natural de las claves
Velocidad de acceso	Muy rápida	Rápida	Más lenta
Uso de memoria	Baja	Moderada	Alta

Tabla 10.1. Comparativa entre las clases que implementan los mapas. Elaboración propia.

Ejercicio 10.6. Crea un programa que, cuando el usuario introduzca el nombre de un número del 1 al 5 en inglés (por ejemplo, "two"), diga su traducción en español ("dos").

Ejercicio 10.7. Crea un programa que introduzca 5 elementos clave-valor en un mapa. Después mostrará por consola el mapa completo ¿Qué orden establece? ¿Por qué? Más tarde eliminará uno de los elementos, reemplazará uno de los valores del mapa con replace() y volverá a mostrar el mapa por consola. Luego pedirá al usuario que introduzca valores y se buscarán en el mapa. Si no se encuentran mostrará el mensaje “no encontrado”. Se saldrá del bucle cuando el usuario pulse 0.

Ejercicio 10.8. Crea un programa que haga lo siguiente:

1. Cree un Map<String, String> para almacenar nombres y números de teléfono.
2. Pida al usuario que introduzca 3 contactos (nombre y número).
3. Después, pregunte al usuario por un nombre y muestre el número de teléfono asociado.
4. Si el nombre no está en la agenda, debe indicarlo con un mensaje.
5. Finalmente, muestra todos los contactos almacenados.

10.6. Ejercicios de repaso

Ejercicio 10.9. Desarrolla un programa que permita almacenar palabras en una lista hasta que el usuario pulse Intro. Después muestra el contenido de la lista.

Ejercicio 10.10. A partir del ejercicio anterior, ordena la lista por orden alfabético y vuelve a mostrarla.

Ejercicio 10.11. Desarrolla un programa que use una pila para invertir una cadena dada por el usuario. Usa funciones y control de excepciones try-catch.

Ejercicio 10.12. Verifica si una palabra se lee igual de izquierda a derecha y de derecha a izquierda (palíndromo) usando una pila. Usa funciones y control de excepciones.

Ejercicio 10.13. Haz el ejercicio anterior usando una cola y una pila.

Ejercicio 10.14. Mueve los primeros n elementos de una cola al final. Por ejemplo, si la entrada es: [1, 2, 3, 4, 5] y n = 2, la salida sería [3, 4, 5, 1, 2].

Ejercicio 10.15. Crea un ArrayList de números enteros y ordénalo de menor a mayor. Utiliza la clase Collections para ordenar. Echa un vistazo al resto de métodos que te ofrece esta clase para trabajar con listas.

Ejercicio 10.16. Dado un ArrayList de enteros, recorre la lista con un bucle foreach y calcula la suma de todos los elementos.

Ejercicio 10.17. Crea un Map<String, Double> donde las claves sean los nombres de productos y los valores sus precios. El usuario introducirá una clave y el programa verifica si el producto específico está en el mapa y dice su precio. Utiliza distintas funciones y controla las posibles excepciones.

Ejercicio 10.18. Dado un Map<String, String>, elimina un elemento a través de su clave y luego imprime el mapa resultante especificando su tamaño. Controla las posibles excepciones.

Ejercicio 10.19. Crea un programa que gestione un mapa de productos y sus precios (clave-valor). El programa debe mostrar el mapa original con algunos productos y precios predefinidos, luego debe pedir al usuario que ingrese el nombre de un producto que desea **actualizar o agregar**. Si el producto ya existe en el mapa, se debe solicitar al usuario un nuevo precio para ese producto y actualizar el valor correspondiente en el mapa. Si el producto no existe en el mapa, se debe agregar el producto con un precio predeterminado de 5 euros. Muestra el mapa antes y después de la actualización o adición. La función que realiza la actualización o adición debe estar separada en una función distinta. Controla las posibles excepciones.

Ejercicio 10.20. Crea una lista de cadenas de texto, luego elimina un elemento dado por el usuario. Después, ordénala alfabéticamente.

Ejercicio 10.21. Dada una lista de países, desarrolla un programa que permita al usuario buscar un país, reemplazar uno por otro o eliminar todos los países de la lista. Utiliza distintas funciones para realizar las acciones.

Ejercicio 10.22. Implementa un programa que muestre un menú para:

1. **Agregar una nueva tarea**
    - Solicitar al usuario una **categoría** (ej: "Trabajo", "Estudio", "Personal").
    - Solicitar una **descripción** de la tarea (ej: "Hacer informe").
    - Almacenar la tarea en:
        - La **lista general** de tareas.

        - El **mapa** bajo su categoría correspondiente.
        - La **cola** de tareas pendientes.

2. **Completar la siguiente tarea pendiente**
    - Extraer la tarea más antigua de la **cola** (FIFO).
    - Moverla a la **pila** del histórico (para consultarla luego).
3. **Mostrar la última tarea completada**
    - Consultar la **pila** del histórico (sin eliminar la tarea).
4. **Listar tareas por categoría**
    - Mostrar todas las tareas de una categoría específica usando el **mapa**.
5. **Mostrar todas las tareas registradas**
    - Recorrer la **lista general** de tareas.
6. **Salir del programa**

Gestiona las tareas utilizando:

- Una **pila** (stack) para el histórico de tareas completadas (LIFO).
- Una **cola** (queue) para manejar tareas pendientes (FIFO).
- Un **mapa** (map) para organizar tareas por categoría.
- Una **lista** (list) para almacenar todas las tareas registradas.

TEMA 11. CLASES ABSTRACTAS E INTERFACES

La **abstracción** consiste en definir **qué hace un objeto**, sin entrar en los detalles de **cómo lo hace**. Permite diseñar estructuras más limpias y reutilizables, enfocadas en el comportamiento común que deben compartir ciertas clases, sin forzar una implementación específica.

Java implementa este principio a través de dos estructuras: **clases abstractas** e **interfaces.**

1. Clases abstractas

Una **clase abstracta** es una clase que **no se puede instanciar directamente**. Su propósito no es crear objetos concretos, sino **servir de modelo base para otras clases**. Puede contener **métodos abstractos** (sin cuerpo), que deben ser implementados por las clases hijas, y también **métodos concretos** (con cuerpo), que pueden heredarse tal cual o sobrescribirse. Además, puede tener atributos y constructores, igual que cualquier otra clase.

Vamos a desarrollar este concepto un poco más: como hemos visto, en una clase tratamos de **definir un objeto** a través de sus atributos y las acciones que puede realizar (sus métodos). Por ejemplo, si hablamos de una puerta, podríamos establecer atributos como la altura, la anchura, el color o el peso, ya que todas las puertas comparten estas características. También podríamos establecer que las puertas pueden abrirse y cerrarse. Y no hay margen de duda porque todas las puertas hacen esto. Por lo tanto, la clase Puerta tendría sentido como **clase concreta**.

Ahora bien, si intentamos definir un objeto más **genérico**, como un animal, surgen problemas. ¿Cómo defines un animal? Algunos atributos o métodos no están del todo claros. ¿Tiene plumas? ¿Nada? ¿Hace sonidos? La realidad es que **hay muchos tipos de animales**, con características muy distintas entre sí. Al intentar definir una clase Animal, nos damos cuenta de que **algunas propiedades o comportamientos no se pueden precisar** sin más información. Por eso, los métodos como hacerSonido() o desplazarse() serían **abstractos**, es decir, no tendrían implementación en la clase base, porque cada animal lo hace de forma distinta.

Otra forma de entenderlo: piensa en la palabra "animal" e imagina uno. ¿Qué ves? Estarás visualizando un perro, un gato o un pájaro, pero no un animal “genérico”. Lo que imaginas es una **concreción**, no una abstracción. Por tanto, Animal sería un concepto **abstracto**, y su representación en Java sería una **clase abstracta** que otras clases concretas (como Perro, Gato o PezEspada) heredan y completan con sus propios detalles.

Para afianzar el concepto, piensa en esto: **hay acciones comunes a todos los animales**, como nacer(). Pero ¿deberíamos implementar este método en la superclase Animal? ¿Y qué pondríamos? No tiene sentido. Un mamífero nace de una manera, un ave de otra, y un insecto de otra distinta. Por lo tanto, **lo útil es declarar el método nacer() como abstracto** en la clase Animal, y que cada subclase lo implemente según corresponda.

Esta estrategia tiene varias ventajas:

- **Evita duplicación de código** al reunir en la superclase atributos y métodos comunes.
- **Fuerza a las subclases a definir comportamientos concretos**, asegurando que el programa refleje fielmente la realidad.
- **Facilita la organización y mantenimiento del código**, especialmente en jerarquías amplias.

Vamos a seguir este ejemplo. El método abstracto nacer() se declara así:

```
public abstract void nacer();
```

Fíjate que el método acaba con punto y coma (;) y no abre y cierra llaves. Esto quiere decir que no va a ser implementado.

Mientras, en la subclase "pato", por ejemplo, implementaríamos el método heredado. Para ello, es conveniente usar la palabra reservada por el lenguaje "override" (sobrescribir). Así:

```
@Override
public void nacer() {
    //un huevo eclosiona y nace el pato
}
```

Ahora, como la clase Animal va a ser abstracta, hay que añadirle el modificador "abstract". Así:

```
public abstract class Animal {

}
```

El ejemplo completo quedaría así:

Clase Animal

```
public abstract class Animal {
    public abstract void nacer();
}
```

Clase Pato

```
public class Pato extends Animal {
    @Override
    public void nacer() {
        //un huevo eclosiona y nace el pato
    }
}
```

Vamos a ver otro ejemplo. Los *sprites* son objetos gráficos 2D. Cuando trabajamos en consola, podemos asumir que los conjuntos de caracteres que vamos a utilizar para representar objetos son *sprites*. Así, es muy probable que en un juego que desarrollemos para consola haya muchos *sprites* distintos.

En este caso, resultaría lógico definir una superclase Sprite que contuviese todas las características comunes a todos los *sprites* que pueda haber en el juego como, por ejemplo, una coordenada X, una coordenada Y, una imagen, un color, una longitud…

Aunque en un juego todos los *sprites* fuesen de un solo píxel, lo habitual sería que tuviesen una imagen formada por un tipo y un número de caracteres distinto. Piensa, por ejemplo, en un disparo y una nave. El disparo tendría una imagen como esta |, con una longitud de un píxel, mientras que la nave podría ser así <-> con una longitud de 3 píxeles. Entonces, ¿cómo implementaríamos en la superclase Sprite los métodos EstablecerImagen() y EstablecerLongitud()?

No tiene sentido implementarlos. Tienen que ser abstractos y dejar que las subclases se encarguen de la implementación.

El código quedaría de la siguiente manera:

Clase Sprite

```
public abstract class Sprite {
    protected String imagen;
    protected int longitud;
    public abstract void EstablecerImagen();
    public abstract void EstablecerLongitud();
}
```

Clase Nave

```
public class Nave extends Sprite {

    @Override
    public void EstablecerImagen() {
        imagen = "<->";
    }

    @Override
    public void EstablecerLongitud() {
        longitud = 3;
    }
}
```

Clase Disparo

```
public class Disparo extends Sprite {

    @Override
    public override void EstablecerImagen() {
        imagen = "|";
    }

    @Override
    public void EstablecerLongitud() {
        longitud = 1;
    }
}
```

Ejercicio 11.1. Define las clases cocina, dormitorio y aseo. Todas tendrán atributos superficie y volumen. Crea una clase abstracta que contenga un método para escribir por consola estos atributos.

Ejercicio 11.2. Haz una clase abstracta llamada "Shape" que defina dos métodos abstractos *area*() y *perimeter*(). Las subclases "Circle" y "Rectangle" implementarán estos métodos para calcular el área y el perímetro de un círculo y un rectángulo. Crea una instancia de las clases Circle y Rectangle y utiliza los métodos area() y perimeter() para calcular y mostrar el área y el perímetro de cada figura.

11.2 Interfaces

La interfaz de una clase define su comportamiento, es decir, lo que podemos y no podemos hacer con los objetos instanciados de la misma. Observa este ejemplo para verlo más claro:

```
public class Botella {
    public int vaciar();
```

```
    public void llenar();
    private String otraCosa();
}
```

En este caso, un objeto de la clase Botella, podría invocar a los métodos v*aciar* y ll*enar*, pero no al método o*traCosa,* ya que es privado.

Es decir, la interfaz de la clase Botella está formada por los métodos *vaciar* y *llenar*.

Podemos encontrar clases que tengan la misma interfaz o funcionalidad (por ejemplo, un libro y una ventana se pueden abrir), pero la implementación es distinta. Es decir, **las interfaces o funcionalidades son comportamientos y las clases representan sus posibles implementaciones.**

Si una interfaz o funcionalidad es implementada de forma distinta por dos objetos (clases), Java nos permite explicitar la interfaz y escribirla en otro archivo distinto. En este caso usaremos la palabra reservada por el lenguaje *interface*. Así:

```
public interface IBotella {
        int vaciar();
        void llenar();
}
```

NOTA: Por convención, en muchos lenguajes de programación, las interfaces se nombran empezando con la letra *i* en mayúscula junto al nombre que le quieras poner. No es el caso de Java, que no hay una regla que obligue a que todas las interfaces tengan que comenzar por "I". Sin embargo, muchas empresas y desarrolladores siguen esta convención como una práctica de estilo de código. Esto puede ser útil para distinguir las interfaces de las clases concretas y hacer que el código sea más legible y fácil de entender. En última instancia, la elección de si utilizar o no la convención de nombrado de interfaces que comienzan con "I" es una decisión que debe tomar el equipo de desarrollo en función de sus propias preferencias y prácticas de codificación. Lo más importante es que se siga una convención coherente en todo el código para que sea fácil de entender y mantener. En este texto sí utilizaré la norma y llamaré a las interfaces comenzando por 'I'.

Como ves, la interfaz *IBotella* declara los métodos *vaciar* y *llenar*. Fíjate que no he añadido modificadores de acceso en los métodos porque obligatoriamente en una interfaz todos sus miembros tienen que ser públicos, aunque, si quieres, se pueden utilizar para mejorar la legibilidad del código. Observa también cómo los métodos no se implementan, sino que se cierran con punto y coma.

Una vez creada la interfaz, en la clase indicamos que vamos a implementar la interfaz IBotella añadiendo la palabra reservada 'implements'. Así:

```
public class Botella implements IBotella {

      @Override
      public int vaciar() {
            // TODO Auto-generated method stub
            return 0;
      }

      @Override
      public void llenar() {
            // TODO Auto-generated method stub
```

```
        }
}
```

En este caso estamos obligados a implementar todos y cada uno de los métodos y propiedades que estén declarados en la interfaz. En el caso de no hacerlo el compilador arrojaría un error. Por esta obligación se dice que las interfaces son **contratos**.

En el siguiente ejemplo, al implementar las clases Ventana y Cuaderno, podríamos instanciar objetos de ventana y de cuaderno a través de la propia interfaz:

Interfaz IAbrir

```
public interface IAbrir {
    void abrir();
}
```

Clase Cuaderno

```
public class Cuaderno implements IAbrir {

        @Override
        public void abrir() {
                System.out.println("Estamos abriendo un cuaderno");
        }
}
```

Clase Ventana

```
public class Ventana implements IAbrir {

        @Override
        public void abrir() {
                System.out.println("Estamos abriendo una ventana");
        }
}
```

Clase PruebaInterfaz

```
public class PruebaInterfaz {

        public static void main(String[] args) {

                IAbrir ventana = new Ventana();
                ventana.abrir();
                IAbrir cuaderno = new Cuaderno();
                cuaderno.abrir();
        }
}
```

11.2.1. Segregación de interfaces

Como hemos visto, cuando preveas que vas a compartir una funcionalidad entre distintos objetos, usa interfaces. Ahora bien, imagina que queremos trabajar con vehículos. Vamos a tener objetos camión y avión. Entonces pensamos que es buena idea declarar una interfaz como esta:

```java
public interface IVehiculo {
    void acelerar(int kmh);
    void frenar();
    void girar(int angulo);
    void despegar();
    void aterrizar();
}
```

Vamos a definir la clase Avión:

```java
public class Avion implements IVehiculo{

	@Override
	public void acelerar(int kmh) {
		System.out.println("Estamos acelerando " + kmh + " km/h");
	}

	@Override
	public void frenar() {
		System.out.println("Estamos frenando");
	}

	@Override
	public void girar(int angulo) {
		System.out.println("Estamos girando " + angulo + " grados");
	}

	@Override
	public void despegar() {
		System.out.println("Estamos despegando");
	}

	@Override
	public void aterrizar() {
		System.out.println("Estamos aterrizando");
	}
}
```

Ahora la clase Camión. Aquí surge un problema. Observa:

```java
public class Camion implements IVehiculo {

	@Override
	public void acelerar(int kmh) {
		System.out.println("Estamos acelerando " + kmh + " km/h");
	}

	@Override
	public void frenar() {
		System.out.println("Estamos frenando");
	}

	@Override
	public void girar(int angulo) {
		System.out.println("Estamos girando " + angulo + " grados");
	}

	@Override
```

```
        public void despegar() {
                // ¿Cómo implementamos este método?
        }

        @Override
        public void aterrizar() {
                // ¿Cómo implementamos este método?
        }
}
```

Observa que los métodos de la interfaz IVehiculo no están definiendo bien las funcionalidades de todos los vehículos. Los camiones no pueden despegar. Es preferible utilizar varias interfaces con menos métodos para definir comportamientos. En este ejemplo, resultaría óptimo dividir la interfaz en dos. Mira:

```
public interface IVehiculo {
    void acelerar(int kmh);
    void frenar();
    void girar(int angulo);
}
```

```
public interface IVehiculoVolador extends IVehiculo {
    void despegar();
    void aterrizar();
}
```

Observa cómo *IVehiculoVolador* hereda de *IVehiculo*. A efectos prácticos, esta es una buena solución, ya que una clase que implemente *IVehiculoVolador* también está implementando *IVehiculo* al haberlo heredado. Por lo tanto, la clase *Camion* puede implementar *IVehiculo* y la clase *Avion* puede implementar *IVehiculoVolador*.

Aunque haya segregación, no tiene por qué haber herencia de interfaces. Observa el siguiente ejemplo. Vamos a definir otra interfaz que sirva para objetos que se utilicen como armas de guerra:

```
public interface IArmasDeGuerra {
    void apuntar();
    void disparar();
}
```

Ahora podríamos tener objetos que fuesen armas de guerra, como una pistola:

```
public class Pistola implements IArmasDeGuerra{

        @Override
        public void apuntar() {
                System.out.println("Apuntando con la pistola");
        }

        @Override
        public void disparar() {
                System.out.println("Disparando con la pistola");
        }
}
```

En este ejemplo, se da el caso de que existen vehículos que pueden ser armas de guerra, como un tanque. En Java, las clases pueden implementar tantas interfaces como necesiten. Así, al declarar la clase Tanque implementaríamos dos interfaces distintas. Observa:

```
public class Tanque implements IVehiculo, IArmasDeGuerra {

	@Override
	public void apuntar() {
		System.out.println("Apuntando con el tanque");
	}

	@Override
	public void disparar() {
		System.out.println("Disparando con el tanque");
	}

	@Override
	public void acelerar(int kmh) {
		System.out.println("Estamos acelerando " + kmh + " km/h");
	}

	@Override
	public void frenar() {
		System.out.println("Estamos frenando");
	}

	@Override
	public void girar(int angulo) {
		System.out.println("Estamos girando " + angulo + " grados");
	}
}
```

Como ves, es muy importante segregar bien las interfaces. En caso contrario podrías tener problemas a la hora de implementar sus métodos.

También es posible definir atributos (variables miembro) en una interfaz de Java, pero solo se pueden declarar como constantes estáticas y finales. Es decir, las variables definidas en una interfaz son siempre constantes, lo que significa que su valor no se puede cambiar después de que se hayan inicializado. Por ejemplo, se puede definir una interfaz que contenga una constante para representar un valor máximo:

```
public interface Interfaz {
    int VALOR_MAXIMO = 100;
}
```

La constante VALOR_MAXIMO se puede utilizar en cualquier clase que implemente la interfaz *Interfaz*. Además, al ser una constante, su valor no se puede cambiar durante la ejecución del programa.

Ejercicio 11.3. Piensa en una funcionalidad como acelerar. ¿Qué objetos pueden acelerar? Declara una interfaz y haz que las clases Coche, Guepardo y Humano la implementen. Después instancia un objeto de cada clase a través de la interfaz e invoca el método Acelerar.

Ejercicio 11.4. Supongamos que tenemos una interfaz llamada Ave que tiene varios métodos para modelar el comportamiento de las aves, tales como volar(), ponerHuevos(), cantar(), etc. Sin embargo, no todas las aves son capaces de realizar todas estas acciones. Por ejemplo, los

avestruces no pueden volar, pero sí ponen huevos. Por lo tanto, sería deseable tener interfaces más específicas para representar los comportamientos de las aves.

Segrega la interfaz Ave en varias interfaces más específicas y coherentes, utilizando el principio de segregación de interfaces. Cada nueva interfaz debe representar un comportamiento específico de las aves. Luego, define una clase Avestruz que implemente las interfaces necesarias que correspondan a los comportamientos que esta ave puede realizar. Genera una función main para instanciar objetos y probar el programa.

11.3. Conclusiones

- Un método abstracto es un método sin implementar, vacío. Estos métodos sirven para definir qué se debe hacer, pero no cómo hacerlo.
- De la implementación del método abstracto se encargarán las subclases.
- Al heredar de una clase abstracta, es obligatorio implementar todos sus métodos abstractos.
- Las clases abstractas no pueden ser instanciadas.
- Una interfaz se compone de métodos en los que se define un comportamiento o funcionalidad, pero no se especifica cómo llevarlo a cabo.
- Una interfaz puede contener variables, pero solo declaradas constantes estáticas.
- Al igual que ocurre con las clases abstractas, serán las clases que implementen la interfaz las que se encargarán de implementar todos los métodos y propiedades. Por eso se dice que una interfaz es un contrato.
- A diferencia de una clase abstracta, una interfaz no puede hacer nada por sí sola.
- Si tenemos objetos que tienen poco o nada en común, pero que comparten una funcionalidad (por ejemplo, un cuaderno y una ventana se abren) es conveniente utilizar interfaces.
- La herencia múltiple no es posible en Java, aunque podemos implementar tantas interfaces como sean necesarias. Ambas opciones son compatibles, es decir, en una clase podemos heredar otra clase y, además, implementar todas las interfaces que queramos. Así:

```
public class MiClase extends ClasePadre implements Interfaz1, Interfaz2 {
    // implementación de métodos de ClasePadre, Interfaz1 y Interfaz2
}
```

11.4. Ejercicios de repaso

Ejercicio 11.5. Crea una jerarquía de clases para representar figuras geométricas. Debes implementar una clase abstracta *Figura* con un método abstracto calcularArea(). Luego, implementa las clases Circulo y Rectangulo que hereden de Figura y calculen su área correspondiente. Usa el método main para crear objetos de estas clases y mostrar el área.

Ejercicio 11.6. Crea una interfaz Sonido con el método hacerSonido(). Luego crea las clases Perro, Gato y Vaca que implementen esta interfaz. Cada clase debe tener su propia versión del sonido que emite. En el main, crea un array de Sonido y recórrelo para ejecutar hacerSonido() en cada objeto.

Ejercicio 11.7. Crea una interfaz Conducible con los métodos acelerar() y frenar(). Define una clase abstracta Vehiculo con atributos marca, modelo y velocidad. Luego crea las clases Coche y Moto que hereden de Vehiculo e implementen la interfaz Conducible. En el main, crea instancias y simula una secuencia de acciones.

Ejercicio 11.8. Define una interfaz MetodoPago con el método pagar(double monto). Crea las clases TarjetaCredito, Paypal y Efectivo que implementen esta interfaz con una lógica de ejemplo. Luego crea una clase Carrito con un método realizarPago(MetodoPago metodo, double monto). Prueba diferentes métodos de pago en el main.

Ejercicio 11.9. Crea un sistema que distinga entre empleados administrativos y técnicos. Define:

- Una clase abstracta Empleado con nombre, id, y método calcularSalario().
- Una interfaz Reportable con el método generarReporte().
- Una interfaz Mantenible con el método realizarMantenimiento().

Luego, la clase Administrativo hereda de Empleado e implementa Reportable, y la clase Tecnico hereda de Empleado e implementa Mantenible.

Agrega un main que cree una lista de empleados y calcule sus salarios. Luego llamará a los métodos de mantenimiento o reporte según el tipo.

Ejercicio 11.10. Define una clase abstracta CuentaBancaria con atributos titular, saldo y métodos:

- depositar(double cantidad)
- retirar(double cantidad)
- consultarSaldo()

Crea la interfaz Transferible con método transferir(CuentaBancaria destino, double monto)

Implementa: CuentaCorriente y CuentaAhorro, que heredan de CuentaBancaria e implementan Transferible.

Simula en el main depósitos, retiros y transferencias entre cuentas. Incluye validaciones de saldo insuficiente y montos inválidos.

TEMA 12. MANEJO DE FICHEROS

Hasta ahora, todos los programas que hemos desarrollado almacenaban la información únicamente en memoria durante su ejecución. Al cerrar el programa, todos los datos desaparecían. Pero en el mundo real, la mayoría de las aplicaciones necesitan guardar información de forma persistente: registros de usuarios, configuraciones, resultados de cálculos, historiales, entre otros.

Para lograrlo, es fundamental aprender a trabajar con **ficheros**.

Java proporciona un conjunto de clases muy completo para la lectura y escritura de datos en archivos, tanto de texto como binarios. Gracias a estas herramientas, podemos leer el contenido de un fichero línea a línea, escribir nuevas líneas, modificar ficheros existentes o incluso manipular rutas y estructuras de directorios.

Dominar el manejo de ficheros te abrirá la puerta a desarrollar aplicaciones mucho más útiles, como procesadores de texto, lectores de configuración, sistemas de log o simples bases de datos locales.

12.1. Escritura en un fichero de texto

Para escribir en un fichero de texto hay que tener en cuenta varias cosas. Para empezar, vamos a utilizar las clases *FileWriter* y *BufferedWriter* que se encuentran en la biblioteca de clases java.io.

Para crear un archivo de texto, accedemos al constructor de la clase *FileWriter* y enviamos como parámetro el nombre del archivo y su extensión. Si no especificas una ruta, el fichero se guardará en la carpeta del proyecto. El fichero no solo se crea, también se abre.

Para escribir en el fichero, utilizaremos el método *write()* de la clase *BufferedWriter*.

Aunque no es obligatorio utilizar *BufferedWriter*, es altamente recomendable, especialmente si se trata de archivos grandes o si se requiere un alto rendimiento en la escritura de archivos.

En el caso de no querer utilizar la clase *BufferedWriter*, usaríamos el método *write()* de la clase *FileWriter* para escribir en el archivo.

Después, no olvides cerrar el fichero cuando termines de escribir.

Mira este ejemplo. Vamos a crear un fichero de texto y a escribir algo en él:

```
import java.io.*;

public class EjemploFileWriter {
    public static void main(String[] args) throws IOException {
        // Crear el objeto FileWriter para escribir en el archivo
        FileWriter archivo = new FileWriter("archivo.txt");

        // Crear el objeto BufferedWriter para escribir en el archivo
        BufferedWriter escritor = new BufferedWriter(archivo);

        // Escribir una cadena en el archivo
        escritor.write("Hola mundo!");

        // Cerrar el escritor y el archivo
```

```
        escritor.close();
        archivo.close();

        System.out.println("El archivo se ha escrito correctamente.");
    }
}
```

Observa cómo se controlan las posibles excepciones con la cláusula '*throw*'. También se podría hacer con los bloques '*try*' y '*catch*'.

Para localizar el archivo creado rápidamente puedes pulsar el botón secundario del ratón sobre el fichero de la clase. Ahí selecciona propiedades y aparecerá una ventana como la de la ilustración 12.1. Después, haz clic sobre el icono de la localización del proyecto (ver ilustración 12.1) y se abrirá directamente en el explorador de archivos del sistema.

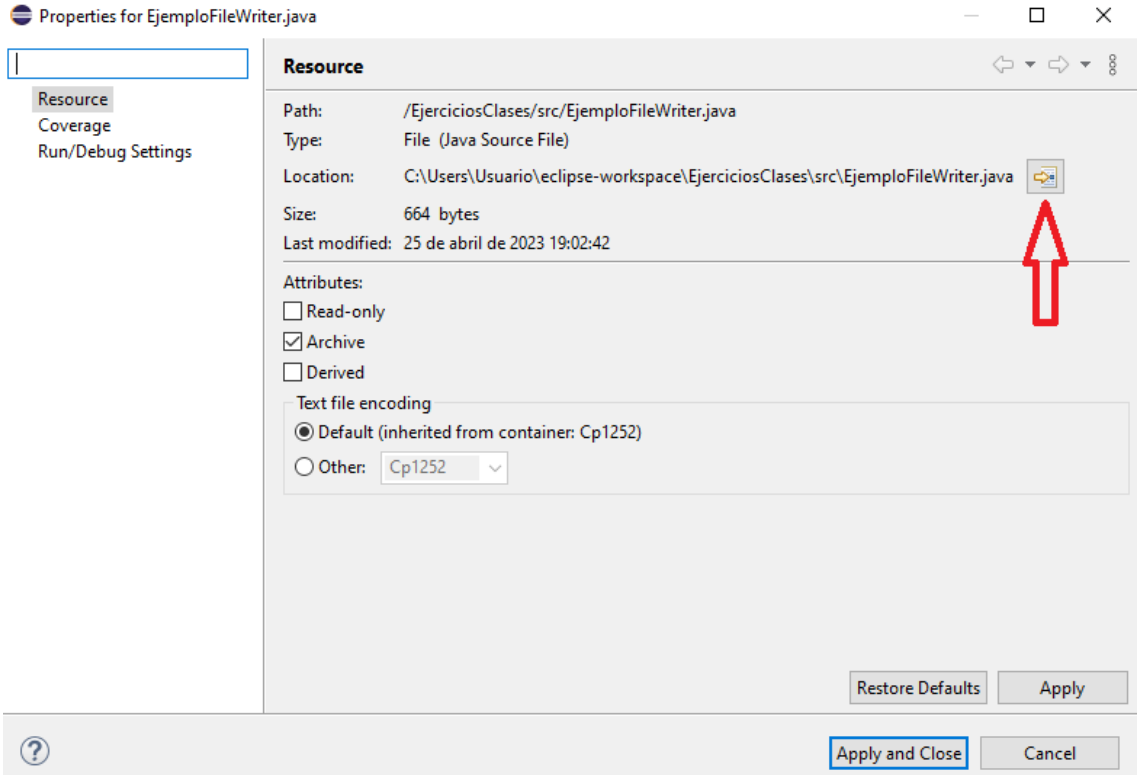

Ilustración 12.1. Ruta del proyecto. Elaboración propia.

Ejercicio 12.1. Desarrolla un programa que guarde las frases que escriba el usuario en un fichero. El programa acabará cuando el usuario escriba "fin".

12.2. Lectura de un fichero de texto

En este caso tenemos que instanciar un objeto de la clase *FileReader* que, al igual que *FileWriter*, se encuentra en la biblioteca de clases java.io. Por otro lado, del mismo modo que utilizamos la clase *BufferedWriter* para optimizar la escritura, instanciaremos un objeto de la clase *BufferedReader*. Además, utilizaremos el método *readLine()* en lugar de *write()* puesto que vamos a leer.

Tampoco en esta ocasión puedes olvidar cerrar el fichero después de usarlo con el método *close()*.

Vamos a ver un ejemplo:

```
import java.io.*;

public class EjemploLecturaArchivo throws IOException{
    public static void main(String[] args){

        // Crear un objeto FileReader para leer el archivo
        FileReader lector = new FileReader("archivo.txt");

        // Crear un objeto BufferedReader para leer el archivo de forma
eficiente
        BufferedReader buffer = new BufferedReader(lector);

        // Leer el archivo línea por línea y mostrarlo en la consola
        String linea = buffer.readLine();
        while (linea != null) {
            System.out.println(linea);
            linea = buffer.readLine();
        }

        // Cerrar el buffer y el lector para liberar los recursos del
sistema
        buffer.close();
        lector.close();
    }
}
```

Ejercicio 12.2. Desarrolla un programa que lea el fichero escrito en el ejercicio anterior y lo muestre por consola.

Por otro lado, habrá ocasiones en las que te interesará buscar datos en un texto. Vamos a ver un ejemplo que nos diga cuántas veces aparece un texto determinado por el usuario dentro de un archivo:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class EjemploBuscarTexto {

    public static void main(String[] args) {
        String fileName = "archivo.txt";
        String textoBuscado = "mundo";
        int cantidad = 0;
        try {
            cantidad = contarTexto(fileName, textoBuscado);
        } catch (IOException e) {
            System.out.println("Se produjo un error al leer el archivo: " +
e.getMessage());
        }
        System.out.println("La cantidad de veces que aparece \"" +
textoBuscado + "\" en el archivo es: " + cantidad);
```

```
    }

    public static int contarTexto(String fileName, String textoBuscado)
throws IOException {
        FileReader fileReader = new FileReader(fileName);
        BufferedReader bufferedReader = new BufferedReader(fileReader);
        String line;
        int cantidad = 0;
        while ((line = bufferedReader.readLine()) != null) {
            if (line.contains(textoBuscado)) {
                cantidad++;
            }
        }
        bufferedReader.close();
        fileReader.close();
        return cantidad;
    }
}
```

Ejercicio 12.3. Desarrolla un programa que pida al usuario el nombre de un fichero de texto y un fragmento de un texto que quiera buscar. El programa mostrará por consola todas las líneas que contengan el texto que se está buscando.

12.3. Añadir datos a un fichero existente

Para poder añadir datos a un fichero existente, solo habría que enviar el parámetro *true* al constructor de la clase *FileWriter*. Mira cómo quedaría:

```
import java.io.FileWriter;
import java.io.IOException;

public class EjemploAgregarTexto {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("archivo.txt", true);
            writer.write("Texto a agregar al archivo");
            writer.close();
        } catch (IOException e) {
             System.out.println("Se produjo un error al leer el archivo: "
+ e.getMessage());
        }
    }
}
```

Ejercicio 12.4. Desarrolla un programa que cree un fichero donde se guardará una línea de texto. Luego se cerrará. Después, pedirá al usuario que introduzca frases para añadirlas al fichero creado. El proceso seguirá hasta que el usuario escriba “fin”. Utiliza funciones estáticas.

12.4. Especificar la ruta del fichero

Hasta el momento, nos hemos limitado a ubicar los ficheros de lectura o escritura en la misma carpeta donde se encuentra el proyecto, pero no tiene por qué ser así. Podemos indicar la ruta que queramos.

Una ruta tiene esta sintaxis:

```
"C:\Users\Usuario\Desktop\archivo.txt"
```

Recuerda que las barras invertidas se usan en Java como secuencias de escape, es decir, para que Java las considere como barra invertida, tienes que duplicarlas. Para que el programa no lance un error, la ruta habría que escribirla así:

```
"C:\\Users\\Usuario\\Desktop\\archivo.txt"
```

También podrías utilizar la barra diagonal. Así:

```
"C:/Users/Usuario/Desktop/archivo.txt"
```

12.5. Control de excepciones de entrada y salida

Podemos encontrarnos algunos métodos para controlar las posibles excepciones de nuestro programa, como el método *exists* de la clase *File*. Este método se interpreta de manera literal, es decir, nos permite saber si el fichero existe, en cuyo caso, sentencia. Mira este ejemplo:

```
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File archivo = new File("prueba.txt");

        if (archivo.exists()) {
            System.out.println(archivo + " existe");
        } else {
            System.out.println(archivo + " no existe");
        }
    }
}
```

Hay que tener en cuenta que el método *exists* también devuelve true si encuentra un directorio. Para evitar este problema se puede utilizar el método *isDirectory*. Mira este ejemplo:

```
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File archivo = new File("prueba.txt");

        if (archivo.exists() && !archivo.isDirectory()) {
            System.out.println(archivo + " existe");
        } else {
            System.out.println(archivo + " no existe");
        }
```

```
    }
}
```

Otra forma de comprobar si un archivo existe en Java es usando la función isFile() del paquete java.io.File. Esta función comprueba si el archivo especificado es un archivo o no. La ventaja de usar isFile() sobre exists() es que no se necesita comprobar si el archivo especificado es un directorio o no. Mira este ejemplo:

```
import java.io.File;

public class Main {
    public static void main(String[] args) {
        File archivo = new File("prueba.txt");

        if (archivo.isFile()) {
            System.out.println(archivo + " existe");
        } else {
            System.out.println(archivo + " no existe");
        }
    }
}
```

Pero podemos encontrarnos más excepciones de entrada y salida de los ficheros que no dispongan de métodos específicos como en el ejemplo anterior (no tener permiso para acceder a un fichero, que el fichero sea de solo lectura, etcétera). Para estos casos, vamos a controlar las posibles excepciones con los bloques *try* y *catch*.

Como se dijo en un tema anterior, puedes utilizar varios bloques "*catch*" con excepciones distintas. En el código podemos encontrarnos errores de entrada o salida del fichero (*IOException*) o de archivos no encontrados (*FileNotFoundException*), por ejemplo. Recuerda que debes ordenar estos bloques desde el más específico hasta el más genérico.

Vamos a ver un ejemplo:

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        FileWriter fichero;
        String nombre;
        String line;
        Scanner scanner = new Scanner(System.in);
        System.out.print("Introduce el nombre del fichero: ");
        nombre = scanner.nextLine();
        System.out.print("Introduce la frase a guardar: ");
        line = scanner.nextLine();
        try {
            fichero = new FileWriter(nombre);
            fichero.write(line);
            fichero.close();
        }
        catch (FileNotFoundException e) { //Excepción y mensaje específico
```

```
            System.out.println("El archivo no pudo ser encontrado." +
e.getMessage());
        }
        catch (IOException e2) { //Excepción y mensaje más genérico que el
anterior
            System.out.println("No se ha producido un error en la
entrada/salida del fichero: " + e2.getMessage());
        }
        catch (Exception e3) { //Excepción y mensaje genérico
            System.out.println("Ha habido un error: " + e3.getMessage());
        }
    }
}
```

Ejercicios 12.5. Desarrolla un programa que pida al usuario dos nombres de ficheros. El primero existirá y tendrá contenido, y el segundo se creará y estará vacío. Después, volcará todo el contenido del primer fichero al segundo, pasando todo el texto a mayúsculas. Además, se controlarán todas las posibles excepciones del programa.

Ejercicio 12.6. Desarrolla un programa que pida al usuario dos números y la operación que quiere realizar con ellos. Este proceso se seguirá realizando hasta que el usuario escriba en la operación "fin". Todos los cálculos se guardarán en un fichero. Además, el programa controlará todas las posibles excepciones, no solo las referidas a ficheros, sino de los propios datos (se introducen caracteres en lugar de números, se divide entre cero, etcétera). En el caso de saltar alguna excepción, no se debe interrumpir el programa.

12.6. Ejercicios de repaso

Ejercicio 12.7. Desarrolla un programa que pida una palabra al usuario. Después tendrá que leer un archivo de texto creado previamente y contar cuántas veces aparece la palabra. Utiliza el método Split() para dividir las frases en palabras. El resultado lo escribirá en un nuevo fichero de texto. El programa debe gestionar las posibles excepciones.

NOTA: Cuando usas throws IOException en la declaración de un método, delegas la responsabilidad de manejar la excepción a quien llame al método. En este caso, debe ser el propio método quien gestione las posibles excepciones, es decir, debes usar bloques de try-catch.

Ejercicio 12.8. Desarrolla un programa que lea un archivo de texto y escriba todo el contenido en otro fichero de texto en mayúsculas (método toUpperCase). El programa debe gestionar las posibles excepciones. Además, cada línea en el archivo de salida debe comenzar con su número de línea.

Ejercicio 12.9. Desarrolla un programa que permita al usuario escribir mensajes por consola. Estos mensajes se añadirán al final de un archivo de texto existente junto con la fecha en cada línea (utiliza el método now() de la clase LocalDateTime). El proceso termina cuando el usuario escriba la palabra "salir". Gestiona las posibles excepciones.

Ejercicio 12.10. Desarrolla un programa que lea un archivo de texto que contiene una lista de estudiantes y sus calificaciones. El formato del archivo es el siguiente:

```
Juan Pérez, 8.5
```

María García, 7.0 Carlos López, 9.2 Ana Sánchez, 6.8

El programa debe:

1. Leer el archivo de texto y almacenar la información de las calificaciones en una lista.
2. Calcular la **nota media** de todos los estudiantes.
3. Crear un **nuevo archivo** con las calificaciones de los estudiantes, ordenadas de menor a mayor.
4. Añadir al final del archivo una línea que indique la **nota media** de todos los estudiantes.

El programa debe gestionar las excepciones que puedan ocurrir durante la lectura y escritura de archivos.

Ejercicio 12.11. Desarrolla un programa que lea un archivo y calcule el número total de caracteres, de palabras y de líneas que tiene. Escribe los resultados en un nuevo archivo de texto.

Ejercicio 12.12. Desarrolla un programa que lea un archivo. Luego, tiene que escribir esas líneas en un archivo nuevo en orden inverso (última línea primero).